Marketing Analytics

Lecture Notes

Marketing Decisions and AI-Assisted Coding

By Nathan Yang
October 26, 2026

Contents

1 Marketing Decisions, Data, and AI-Assisted Python **1**

1.1 From a Marketing Question to an Analytical Object . . . 1
1.2 The Unit of Observation Determines the Calculation . . . 2
1.3 Python Objects and Reproducible Calculations . . . 3
1.4 Case Study: A Coupon Campaign Specification . . . 4
1.5 AI-Assisted Coding as a Sequence of Verifiable Steps . . . 5
1.6 Interpreting and Communicating a Result . . . 5
1.7 Readings . . . 6
1.8 mHealth: From Food Entries to a Decision Table . . . 6
1.9 Python Calculations for the Synthetic Cases . . . 13
1.10 mHealth: An Auditable Monthly Business Review . . . 15

2 Customer Data, Measurement, and Visualization **18**

2.1 Define the Population Before the Metric . . . 18
2.2 Build a Customer Table Without Losing Customers . . . 18
2.3 Case Study: A Reversal Caused by Customer Mix . . . 19
2.4 Cohorts, Retention, and Customer Age . . . 20
2.5 Visual Encodings Should Match the Question . . . 21
2.6 Case Study: Online Grocery Baskets and Repeated Purchases . . . 21
2.7 From Evidence to a Management Presentation . . . 22
2.8 AI-Assisted Workflow and Practice . . . 23
2.9 Readings . . . 23
2.10 mHealth: Recorded Variety and Missing Days . . . 23
2.11 mHealth: Nutrition Knowledge and Sample Restrictions . . . 25
2.12 Python Calculations for the Synthetic Cases . . . 30
2.13 mHealth: Recency and Frequency at a Fixed Date . . . 31
2.14 mHealth: Chart Choice, Denominators, and Customer Differences . . . 33
2.15 Interactive Charts Must Preserve the Comparison . . . 38
2.16 McDonald's: Measuring Ice-Cream Availability . . . 42
2.17 Public Signals of Demand for Generative AI . . . 49

3 Regression, Interpretation, and Marketing Decisions **53**

3.1 A Conditional Mean and a Regression Line . . . 53
3.2 Multiple Regression and the Comparison It Uses . . . 54

3.3 Case Study: Attractive Markets and Apparent Competitive Benefits 54
3.4 Transformations and Economically Meaningful Units 55
3.5 Interactions Describe How a Relationship Changes 56
3.6 Uncertainty, Dependence, and Practical Importance 56
3.7 Diagnostics and Panel Data 58
3.8 AI-Assisted Implementation and Practice 58
3.9 Readings 59
3.10 mHealth: Breadth and Next-Day Recorded Calories 59
3.11 Python Calculations for the Synthetic Cases 62
3.12 Price Elasticity When Prices Respond to Demand 62

4 Product Preferences, Customer Segments, and Lifetime Value 66
4.1 Product Design Requires Tradeoffs 66
4.2 Utility, Choice Probabilities, and Willingness to Pay 66
4.3 Case Study: A New Bundle Can Cannibalize Existing Sales 67
4.4 Estimate and Validate Preferences 68
4.5 Segmentation Begins With a Definition of Similarity 68
4.6 Choosing and Evaluating Segments 69
4.7 Customer Lifetime Value Depends on Timing and Margin 70
4.8 Subscription and Noncontractual Relationships 71
4.9 Integrated Case: Which Segment Should Receive Retention Support? 72
4.10 AI-Assisted Implementation and Readings 72
4.11 mHealth: Behavioral Segments and Product Economics 72
4.12 Python Calculations for the Synthetic Cases 75
4.13 mHealth: Time Until a Logging Gap 76
4.14 Subscription Retention and Category Participation 79

5 Text, Social Media, and Customer Networks 84
5.1 Define the Construct and the Document 84
5.2 Representing Language With Counts and Weights 84
5.3 Embeddings and Semantic Similarity 85
5.4 Sentiment, Aspect Labels, and Topic Models 86
5.5 Case Study: A Complaint Detector Under Changing Prevalence 86
5.6 Human and LLM Annotation 87
5.7 Networks: Connections, Reach, and Brokerage 88
5.8 Social-Media Case: Engagement and Commercial Value 88
5.9 AI-Assisted Workflow, Practice, and Readings 89
5.10 mHealth: Classifying New Food Descriptions 89

5.11 Python Calculations for the Synthetic Cases . . . 92
5.12 mHealth: Text Topics and Their Denominators . . . 93

6 Causal Effects and Marketing Experiments 97
6.1 Define the Treatment, Outcome, and Estimand . . . 97
6.2 Why the Treated-Untreated Difference Can Be Biased . . . 97
6.3 Designing an Experiment Around the Decision . . . 98
6.4 Case Study: Estimate an Email Assignment Effect . . . 99
6.5 Power, Precision, and the Minimum Useful Effect . . . 99
6.6 Implementation Checks That Protect the Comparison . . . 100
6.7 Observational Adjustment and Its Limits . . . 100
6.8 Difference-in-Differences and a Counterfactual Trend . . . 103
6.9 Other Designs: Discontinuities and Instruments . . . 104
6.10 Spillovers and Experiment Units . . . 104
6.11 AI-Assisted Workflow and Practice . . . 105
6.12 Readings . . . 106
6.13 mHealth: Planning a Randomized Suggestion Test . . . 106
6.14 Python Calculations for the Synthetic Cases . . . 108
6.15 mHealth: Prior Activity and Experiment Precision . . . 110
6.16 McDonald's: Repair Policy and the Choice of a Comparison Group . . . 113

7 Prediction, Machine Learning, and Marketing Actions 117
7.1 Specify the Forecast Origin and Target . . . 117
7.2 Baselines, Loss Functions, and Regularization . . . 118
7.3 Logistic Regression, Trees, and Ensembles . . . 118
7.4 Validation, Cross-Validation, and Leakage . . . 119
7.5 Case Study: Accuracy Can Conceal a Useless Model . . . 119
7.6 Ranking and Calibration Answer Different Questions . . . 120
7.7 Thresholds Should Reflect the Decision . . . 120
7.8 Case Study: Targeting the Most Responsive Customers Can Lose Money . . . 121
7.9 Interpretation, Fairness, and Deployment . . . 122
7.10 Recommendation Models and the Candidate Set . . . 123
7.11 Ranking Metrics Depend on Position and Available Labels . . . 124
7.12 Case Study: Food Recommendations, Variety, and Customer Goals . . . 124
7.13 LLM Explanations Need Factual and Behavioral Evaluation . . . 126
7.14 AI-Assisted Workflow, Practice, and Readings . . . 127
7.15 mHealth: Predicting a Returning User's Next Basket . . . 127
7.16 Python Calculations for the Synthetic Cases . . . 131

7.17 mHealth: Basket Associations and Future Validation . . . 133
7.18 McDonald's: Forecasting Availability With Missing Future Labels . . . 137

8 Current Topics and Integrated Marketing Decisions 141
8.1 Recommendation Systems Learn From Selected Exposure . . . 141
8.2 Case Study: Relevance, Variety, and Customer Goals . . . 144
8.3 Marketing-Mix Models Combine Carryover and Saturation . . . 146
8.4 Case Study: Allocate the Next Dollar Using Marginal Returns . . . 147
8.5 Generative AI Changes the Cost of Producing and Checking Work . . . 148
8.6 Case Study: Evaluate an AI-Assisted Analytics Workflow . . . 148
8.7 Integrated Case: A Retail Subscription Launch . . . 149
8.8 Practice and Further Reading . . . 150
8.9 mHealth: Familiarity, Novelty, and the Decision Objective . . . 151
8.10 Python Calculations for the Synthetic Cases . . . 153
8.11 Evidence Checks for Generated Marketing Explanations . . . 154
8.12 An Executable Chart Can Still Misstate the Evidence . . . 156
8.13 Competition, Demand Measurement, and Generative AI . . . 159

Chapter 1

Marketing Decisions, Data, and AI-Assisted Python

A retailer must decide which customers should receive a coupon, how much inventory to place in each store, and which product descriptions should appear online. Each decision needs a different analytical object. A forecast of next week's purchases can inform inventory. A comparison of purchases with and without a coupon can inform targeting. A summary of product complaints can identify a service problem. An analysis becomes useful when its output corresponds to the decision the manager can actually change.

1.1 From a Marketing Question to an Analytical Object

Consider a subscription service deciding whether to offer a renewal discount. The service observes account tenure, past payments, recent use, and prior contacts. A descriptive analysis reports renewal rates by tenure. A predictive analysis estimates the probability that an account renews under the conditions represented in the training data. A causal analysis estimates how an offer changes renewal. A decision analysis combines that change with margins, contact costs, and operational constraints. These objects can use overlapping data while answering different questions.

The timing of the decision determines what information is available. If an account must be selected on Monday morning, a support call made on Wednesday cannot be a predictor for that selection. A variable can be strongly associated with an outcome and still be unusable at the decision time. Write down a forecast origin, an outcome window, and a feature window before constructing a table. An account-week table and an account-lifetime table support different forecasts because their rows and time horizons differ.

For a renewal offer, a useful decision quantity is incremental expected contribution:

$$\Delta\pi_i = m_i\{p_i(1) - p_i(0)\} - c_i.$$

Here i identifies an account, $p_i(1)$ and $p_i(0)$ are its renewal probabilities with and without the offer, m_i is contribution in dollars per retained account over the specified horizon, and c_i is the expected incremental cost of making the offer. This compact expression assumes that the offer's costs can be summarized by c_i and that m_i already reflects relevant variable costs. When every renewing recipient receives a discount, discount expense depends on renewal under treatment and should be modeled explicitly.

The difference between the two renewal probabilities matters because many accounts would renew without assistance. Suppose contribution is \$100 and contact cost is \$3. Then an account whose renewal probability rises from 0.90 to 0.91 has incremental expected contribution of \$-2. An account whose probability rises from 0.40 to 0.50 has contribution of \$7. Ranking by treated

renewal probability chooses the first account; ranking by incremental contribution chooses the second. Observing only treated customers would not reveal the untreated probabilities.

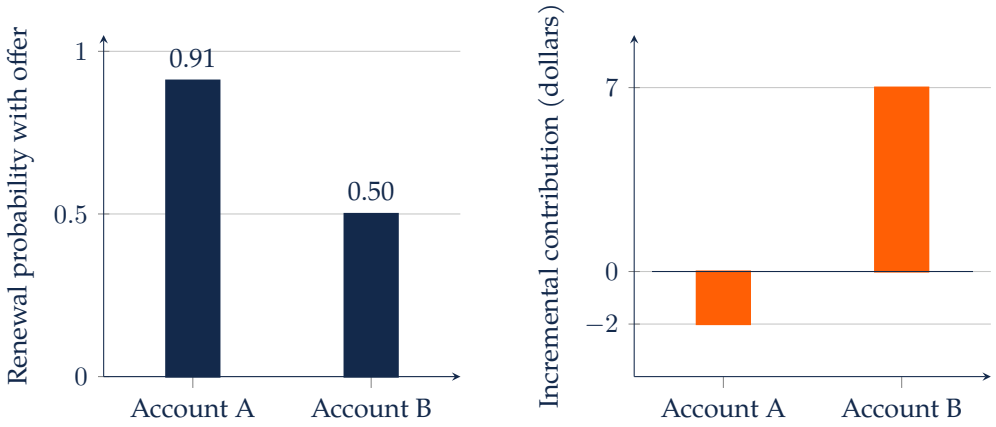


Figure 1.1. High response and high incremental value can identify different customers.

With contribution and contact cost held fixed, the first account's high baseline renewal probability reverses its ranking under incremental value. A response model and a targeting policy therefore need separate evaluation.

1.2 The Unit of Observation Determines the Calculation

A transaction system often stores one row per product line, while the marketing question concerns orders or customers. Suppose order O1 contains two product lines, O2 contains one, and O3 contains three. There are six rows, three orders, and perhaps only two customers. Counting rows as purchases would report six orders. Averaging customer attributes over product lines would give more weight to customers with larger baskets.

A key uniquely identifies the intended unit. At the line level, the key might be the combination of order ID and line number. At the order level, it is order ID. At the customer level, it is customer ID. A useful data contract states the key, allowed duplicates, measurement units, missing-value codes, and date conventions. It also states whether a negative quantity represents a return, whether a price includes tax, and whether canceled orders belong in the analysis.

For line ℓ in order o, gross line sales are quantity times unit price, $s_{o\ell} = q_{o\ell}p_{o\ell}$. Order sales are $S_o = \sum_\ell s_{o\ell}$, and customer-period sales are the sum across that customer's eligible orders during the period. These are identities once the population and variables have been defined. They do not require a behavioral assumption. The substantive choices enter through eligibility, returns, discounts, and the observation window.

Joining data can change the population or its weights. A many-to-one join from order lines to a customer table should add customer attributes without multiplying the original rows. If the customer table accidentally contains two records for a customer, every matching order line appears twice. A program can finish without an error while overstating sales. Verify the relationship between keys before joining, and compare row counts and totals afterward.

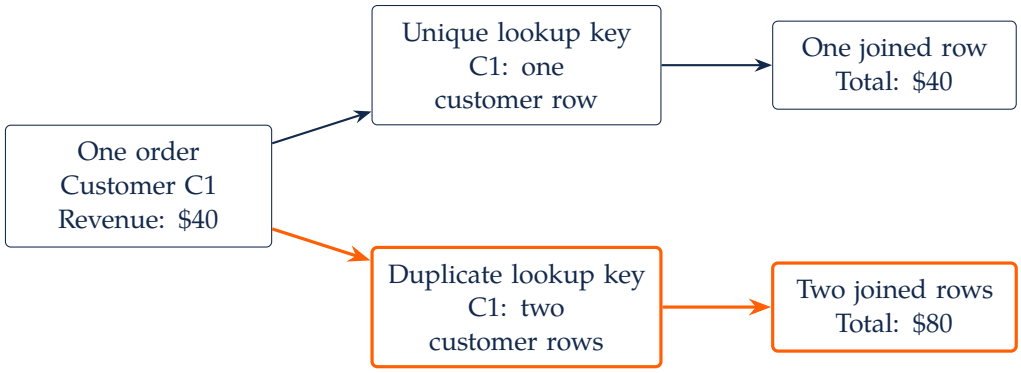


Figure 1.2. A duplicate customer key multiplies transactions in a join.

For a single $40 order, a customer lookup with two matching records produces two joined rows and an apparent $80 total. This error arises from the structure of the tables. Increasing the sophistication of the subsequent model cannot repair it.

1.3 Python Objects and Reproducible Calculations

Python stores individual values in objects such as integers, floating-point numbers, strings, and booleans. A list stores an ordered collection; a dictionary associates names with values. A function packages a calculation with explicit inputs and an output. These distinctions matter when an assistant supplies code: a column of numeric-looking strings does not behave like a numeric column, and an empty string does not automatically represent a missing value.

Suppose two orders contain three product lines. Multiplying quantity by unit price gives line sales; summing within each order gives order sales. Returning each calculation as a number allows its result to be checked and reused.

```
lines = [
    {"order": "O1", "quantity": 2, "unit_price": 10.0},
    {"order": "O1", "quantity": 1, "unit_price": 5.0},
    {"order": "O2", "quantity": 3, "unit_price": 4.0},
]

def line_sales(line):
    return line["quantity"] * line["unit_price"]

order_sales = {}
for line in lines:
    key = line["order"]
    order_sales[key] = order_sales.get(key, 0.0) + line_sales(line)

assert order_sales == {"O1": 25.0, "O2": 12.0}
assert sum(order_sales.values()) == 37.0
```

The total is $37 across two orders, so average order value is $18.50. Dividing by the three product lines would instead give average line sales, $12.33. Neither denominator is inherently wrong; they describe different objects. The mistake would be assigning one calculation the other's interpretation.

A DataFrame provides a tabular interface for larger data. In pandas, selecting columns, filtering rows, creating variables, grouping observations, and joining tables are distinct operations. Use explicit intermediate names when those operations change the unit of observation. For example, `eligible_lines`, `orders`, and `customer_months` communicate more than repeatedly overwriting `df`. The pandas tutorials provide syntax references; the analytical specification should determine which operations are needed.

Notebook execution order can hide dependencies. A cell may work because an earlier interactive command created a variable that is absent from the saved notebook. Before describing a notebook as reproducible, restart the kernel and run all cells in order. Store file paths relative to the project, record package versions, and distinguish source data from derived outputs. A random seed helps reproduce a simulation or split, but it does not guarantee identical results across every software version or computing platform.

1.4 Case Study: A Coupon Campaign Specification

A retailer has a monthly contact budget for 5,000 customers. The proposed campaign offers a discount on the next purchase. The manager initially asks for "a model to find our best customers." That request leaves the action, outcome, and objective unresolved. The highest-spending customers may receive the offer without changing their behavior; customers likely to respond may buy products with low margins.

A usable specification would identify the decision date, eligible customer population, offer, evaluation horizon, and objective. One possibility is to maximize incremental contribution during the next 30 days, subject to the contact limit. Eligibility might require marketing consent and an active account. The analysis should define contribution after discounts, returns, fulfillment, and contact costs. It should identify which customer information existed before assignment and how untreated outcomes will be measured.

The initial dataset might include customer-level histories and a randomized pilot. Without such a pilot or another credible design, a response model can help describe or predict behavior, but the incremental contribution calculation remains unresolved. The next operational step may therefore be a randomized test, rather than immediate large-scale optimization.

A specification also needs acceptance criteria. The eligible-customer table must have one row per customer, no post-assignment features, and the intended date coverage. A campaign summary must reconcile assigned counts, contact costs, purchases, and contribution. These checks are small enough to state before asking an AI assistant to implement the analysis. They protect the meaning of the result while allowing flexibility in the code.

1.5 AI-Assisted Coding as a Sequence of Verifiable Steps

An effective coding request names the inputs, output, definitions, and checks. For the transaction example, ask for a function that accepts product-line records, validates required fields, aggregates sales by order, and returns both order totals and an exception report. Provide the three-line example and its expected $37 total. Ask the assistant to explain what happens with a return, a missing order ID, or a duplicated line.

Generate one analytical transformation at a time when the next step depends on a definition. Inspect the proposed code before executing it, especially commands that install software, delete files, upload data, or invoke an external API. Read error messages as evidence about the code's assumptions. A type error may reveal numeric values stored as strings; a merge error may reveal an incorrect key. Repairing the error without understanding its cause can preserve the underlying analytical mistake.

Verification should use both identities and meaningful counterexamples. Order totals must sum to the line total for the same eligible rows. A single known customer should produce a manually calculable summary. An empty input should have an explicit result. A missing customer lookup should not silently remove sales. These checks constrain what the implementation is allowed to do without forcing one particular coding style.

Keep a short record of consequential AI contributions: the task requested, important changes to the plan or code, and the checks performed. A saved conversation alone does not establish correctness. The record should make it possible to explain why a transformation was accepted.

1.6 Interpreting and Communicating a Result

A useful recommendation names the decision, states the relevant estimate with units and uncertainty, and explains how it changes the action. "The model has high accuracy" leaves the manager's decision unresolved. "At a $3 contact cost and the stated contribution assumptions, contact the customers with positive estimated incremental contribution, subject to capacity" connects the analysis to the objective. The recommendation still needs evidence about how the probabilities were estimated and whether the policy was evaluated on new customers or periods.

Results should distinguish observation from assumption. A transaction total is calculated from recorded eligible rows. A renewal probability is an estimate. A contribution margin may be supplied by finance. An untreated outcome for an offered customer is counterfactual. Listing these roles prevents a spreadsheet or notebook from giving every input the appearance of equal certainty.

If one customer places four orders and another places one, an order-weighted mean gives the first customer four times the weight. If both customers should count equally, calculate each customer's mean first. If an AI-generated program reports six purchases from six product lines, inspect whether purchase was defined as a line or an order. If an offer has a high response rate, ask for the response rate under no offer before interpreting the response as an effect.

1.7 Readings

The Python marketing research companion repository provides examples tied to marketing applications. Its notebooks are reference material and require inspection of dependencies and data paths before adaptation. The pandas tutorials support tabular implementation.

1.8 mHealth: From Food Entries to a Decision Table

An mHealth product manager must decide which food categories to suggest when a customer returns to the app. Recent logs reveal repeated categories, variation across days, and gaps in observation. The immediate analytical task is to construct a reliable customer history. A recommendation based on duplicated daily totals or an accidental change in the customer population can look precise while describing the wrong behavior.

The released files contain actual 2016 records. User identifiers have been replaced, birthdays have been replaced with age on January 1, 2016, and weight fields contain documented random perturbations. The food file has 13,874,280 entries from 5,908 users. The separate daily file covers a smaller set of users and dates. Food categories, descriptions, and calorie fields retain their source values, apart from standardizing provider references to mHealth. The data dictionary defines the available fields and unresolved coding conventions. In particular, `Weightchange`, `Goalplan`, and `Gender` have undocumented code meanings. None establishes a treatment, a behavioral goal, or a validated sex variable for an energy equation.

The working cohort retains every entry for users whose first four hexadecimal pseudonym digits have a value below 4,096. This fixed rule reduces memory requirements and preserves within-user histories. Its realized sample contains 917,153 entries, 370 users, and 76,804 observed user-days. Ages at the reference date range from 17 to 76. The cohort is a subset of app users who appear in the released file; it is not a representative population sample. The adult nutrition sensitivity analysis in M2 uses a separate age restriction.

A food-entry row records an item, while `Foodcalories` repeats the source daily total. On the first eligible user-day selected by the code below, five entries on July 4 have item calories summing to 1,132.7852. Each row repeats a daily total of 1,132.79. Summing that repeated field gives 5,663.95, exactly five times the reported daily total. The small difference between the item sum and one copy of the daily total is compatible with rounding in this case. Larger discrepancies elsewhere require investigation.

The daily key is `(Userid, date)`. A left join preserves all 76,804 cohort days, with 37,634 matched days and 39,170 unmatched days in the separate daily file. Replacing it with an inner join would remove more than half the cohort days. An unmatched row is therefore a sample-definition decision, even if the software reports no error.

The two branches retain the same matched records. Only the left-join branch preserves the unmatched days needed to describe the food-log cohort. Neither branch supplies unobserved calendar days.

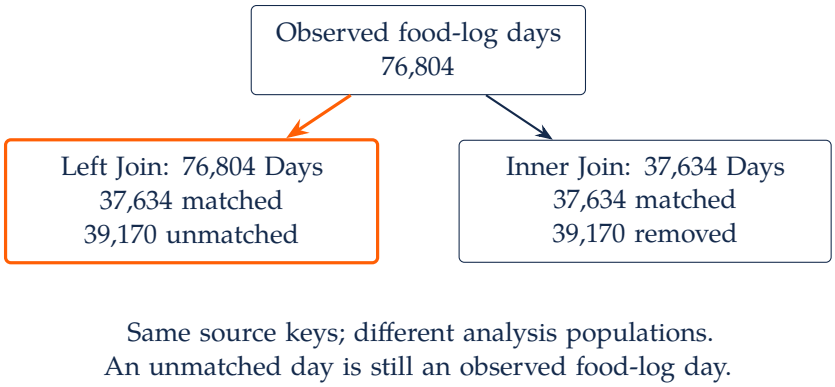


Figure 1.3. A left join preserves observed days while an inner join changes the analysis population.

1.8.1 Loading Data and Constructing Daily Measures

Use Python 3.13 with NumPy 2.1.3, pandas 2.2.3, PyArrow 19.0.0, SciPy 1.15.3, scikit-learn 1.6.1, and statsmodels 0.14.4 to reproduce the saved calculations. From the repository root, install the pinned packages into an environment, verify the release files, and run the analyses in dependency order:

```
python -m pip install -r 03_Data_and_Code/mHealth/recurring_examples/requirements.txt
python 03_Data_and_Code/mHealth/download_data.py
python 03_Data_and_Code/mHealth/recurring_examples/run_all.py
```

The downloader checks the published SHA-256 hashes. The first run reads the compressed food file in chunks and saves a local Parquet cache; subsequent runs reuse it. The cache is excluded from Git. Delete the `recurring_examples/cache` directory to force a fresh derivation. `common.py` supplies the same sample and measurement functions to every module. Paths are relative to the script location, so the saved programs can also be run from another working directory.

For each observed day, category breadth counts distinct recorded categories, assigning missing labels to `Unclassified`. Calorie-share variety uses the sum of included item calories as its denominator. A negative or missing item calorie value invalidates that day's share calculation; a nonpositive item sum also makes shares undefined. These conditions affect calorie-based variety, while the day's existence can still be used to measure logging. The first reported daily total is retained only after checking that its nonmissing values agree within the day.

Python source: common.py. From its directory, run `python common.py`.

```
"""Shared input and measurement functions for the eight mHealth examples."""

from pathlib import Path
import hashlib
import json

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parent
```

```
DATA = ROOT.parent / "data"
CACHE = ROOT / "cache"
OUTPUT = ROOT / "results"
SEED = 534
COHORT_THRESHOLD = 4096


def save_result(module, result):
    OUTPUT.mkdir(exist_ok=True)
    (OUTPUT / f"{module}.json").write_text(
        json.dumps(result, indent=2, allow_nan=False) + "\n"
    )


def load_food():
    """Keep all rows for users whose first four pseudonym hex digits are <4096."""
    CACHE.mkdir(exist_ok=True)
    cached = CACHE / "food.parquet"
    metadata = CACHE / "source.json"
    manifest = json.loads((ROOT.parent / "manifest.json").read_text())
    signature = {
        "sha256": manifest["files"]["food_entries.csv.gz"]["sha256"],
        "threshold": COHORT_THRESHOLD,
        "preparation_version": 2,
    }
    if (
        cached.exists()
        and metadata.exists()
        and json.loads(metadata.read_text()) == signature
    ):
        return pd.read_parquet(cached)
    columns = [
        "Userid",
        "Fancydate",
        "Name",
        "Category",
        "Calories",
        "Foodcalories",
        "Budgetcalories",
        "Age_at_2016_01_01",
    ]
    parts = []
    for chunk in pd.read_csv(
        DATA / "food_entries.csv.gz", usecols=columns, chunksize=250000
    ):
        code = chunk.Userid.str.slice(1, 5).map(lambda value: int(value, 16))
        parts.append(chunk.loc[code < COHORT_THRESHOLD].copy())
    food = pd.concat(parts, ignore_index=True)
    food["date"] = pd.to_datetime(food.Fancydate)
    food["category"] = food.Category.fillna("Unclassified")
    food["description"] = (
```

```
        food.Name.fillna("")
        .str.casefold()
        .str.replace(r"\s+", " ", regex=True)
        .str.strip()
    )
    food.to_parquet(cached, index=False)
    metadata.write_text(json.dumps(signature, indent=2) + "\n")
    return food


def make_daily(food):
    """One row per observed user-day; missing calendar days are not created."""
    keys = ["Userid", "date"]
    grouped = food.groupby(keys, observed=True)
    daily = grouped.agg(
        entries=("Calories", "size"),
        category_count=("category", "nunique"),
        item_calories=("Calories", "sum"),
        food_calories=("Foodcalories", "first"),
        budget_calories=("Budgetcalories", "first"),
        daily_total_versions=("Foodcalories", "nunique"),
        budget_versions=("Budgetcalories", "nunique"),
    )
    daily.loc[daily.daily_total_versions != 1, "food_calories"] = np.nan
    daily.loc[daily.budget_versions != 1, "budget_calories"] = np.nan
    invalid = food.Calories.isna() | food.Calories.lt(0)
    daily["invalid_item_calories"] = invalid.groupby([food.Userid, food.date]).sum()
    categories = food.groupby(keys + ["category"], observed=True).Calories.sum()
    category_totals = categories.groupby(level=keys).transform("sum")
    concentration = (
        (categories / category_totals.where(category_totals > 0))
        .pow(2)
        .groupby(level=keys)
        .sum(min_count=1)
    )
    daily["variety"] = (1 - concentration).where(
        (daily.item_calories > 0) & (daily.invalid_item_calories == 0)
    )
    daily = daily.reset_index().sort_values(keys).reset_index(drop=True)
    assert not daily.duplicated(keys).any()
    assert daily.variety.dropna().between(-1e-10, 1 + 1e-10).all()
    return daily


def make_baskets(food):
    """Distinct observed categories; the missing-category label is not a candidate."""
    selected = food.loc[
        food.category != "Unclassified", ["Userid", "date", "category"]
    ].drop_duplicates()
    return (
        selected.groupby(["Userid", "date"], observed=True)
```

```
        .category.agg(lambda values: tuple(sorted(values)))
        .reset_index(name="basket")
    )


def description_holdout(text):
    """A fixed description-level split independent of labels or outcomes."""
    return int(hashlib.sha256(text.encode()).hexdigest()[:8], 16) % 5 == 0


def load_inputs():
    food = load_food()
    return food, make_daily(food), make_baskets(food)


if __name__ == "__main__":
    food, daily, baskets = load_inputs()
    daily.to_parquet(CACHE / "daily.parquet", index=False)
    baskets.to_parquet(CACHE / "baskets.parquet", index=False)
    result = {
        "rows": len(food),
        "users": int(food.Userid.nunique()),
        "user_days": len(daily),
        "categories": int(food.category.nunique()),
        "missing_category_rows": int(food.Category.isna().sum()),
        "invalid_item_calorie_rows": int(
            (food.Calories.isna() | food.Calories.lt(0)).sum()
        ),
        "days_with_inconsistent_daily_total": int(
            daily.daily_total_versions.ne(1).sum()
        ),
        "days_with_inconsistent_budget": int(daily.budget_versions.ne(1).sum()),
        "nonpositive_recorded_daily_totals": int(daily.food_calories.le(0).sum()),
        "variety_eligible_days": int(daily.variety.notna().sum()),
        "item_total_matches_recorded_total_within_one": int(
            (daily.item_calories - daily.food_calories).abs().le(1).sum()
        ),
    }
    save_result("cohort", result)
    print(json.dumps(result, indent=2))
```

1.8.2 Join Validation and a Hand-Checkable Day

The join declares a one-to-one relationship and records unmatched keys. The small day-level example checks both candidate aggregations against the repeated total. Keeping these checks beside the join prevents a later model from silently inheriting a different denominator.

Python source: m01.py. From its directory, run `python m01.py`.

```
"""Audit the cohort, repeated totals, and the separate daily file."""

import json
import pandas as pd
from common import DATA, load_inputs, save_result

food, daily, baskets = load_inputs()
other = pd.read_csv(DATA / "daily_records.csv.gz", usecols=["Userid", "Fancydate"])
other["date"] = pd.to_datetime(other.Fancydate)
assert not other.duplicated(["Userid", "date"]).any()
joined = daily.merge(
    other[["Userid", "date"]],
    how="left",
    on=["Userid", "date"],
    validate="one_to_one",
    indicator=True,
)
assert len(joined) == len(daily)
example = daily.loc[(daily.entries.between(3, 6)) & daily.food_calories.gt(0)].iloc[0]
rows = food.loc[food.Userid.eq(example.Userid) & food.date.eq(example.date)]
result = {
    "rows": len(food),
    "users": int(food.Userid.nunique()),
    "days": len(daily),
    "matched_days": int(joined._merge.eq("both").sum()),
    "unmatched_days": int(joined._merge.eq("left_only").sum()),
    "example_date": str(example.date.date()),
    "example_entries": len(rows),
    "example_item_sum": float(rows.Calories.sum()),
    "example_daily_total": float(example.food_calories),
    "example_wrong_sum": float(rows.Foodcalories.sum()),
}
save_result("M01", result)
print(json.dumps(result, indent=2))
```

1.8.3 Data Cleaning as a Sequence of Decisions

Structural checks stop calculations that violate their stated observation unit. A duplicate key in a daily lookup can multiply food records, so it must be resolved before joining. Review flags serve a different purpose: they identify observations whose meaning needs investigation. The cohort contains 52,838 rows equal on the selected fields of user, day, normalized description, category, and calories. Those fields omit event timestamps and meal identifiers. Repeated servings can therefore produce equal rows. Automatically dropping them would impose an unsupported definition of duplication.

The audit finds 206 food rows with missing categories, one negative item calorie value, and 23 days with a nonpositive sum of item calories. The calorie-share calculation consequently excludes 24 days. By contrast, 319 days have a nonpositive reported daily total. These counts differ because

the item sum and reported total are separate measurements. Their discrepancy exceeds one calorie unit on 23,729 days. These flags can overlap and must not be added as if they represented disjoint exclusions.

A value outside a chosen review range is evidence for inspection, not proof of an error. The reported daily calorie median is 1,475.63, and the range is 0 to 9,190. A review flag below 100 or above 5,000 marks 651 days. Those thresholds are imposed sensitivity choices. They do not establish nutritional adequacy or identify incorrect reports. Preserve the released values, add flags, document the rule for each analysis, and report the resulting sample. M2 compares nutrition-informed alternatives; M3 repeats a regression after a stated restriction.

A useful AI coding request is: "Create separate structural checks and review flags. Stop on a duplicate daily key, preserve repeated food entries, count overlapping flags separately, and return the retained sample for each calculation." Verify the proposed implementation with a two-entry day whose repeated total is 100 and whose calorie shares are 0.9 and 0.1. It must produce a daily total of 100 and variety of 0.18. A gap from January 1 to January 3 must remain a missing January 2 observation. The executable counterexamples are in `test_logic.py`.

Python source: audit.py. From its directory, run `python audit.py`.

```
"""Separate structural failures, review flags, and analysis-specific exclusions."""

import json
import numpy as np
import pandas as pd
from common import DATA, load_inputs, save_result


def require_unique(frame, keys):
    if frame[keys].isna().any().any():
        raise ValueError("Missing join key")
    if frame.duplicated(keys).any():
        raise ValueError("Duplicate key at the declared observation unit")


def daily_flags(daily):
    flags = pd.DataFrame(index=daily.index)
    flags["invalid_item"] = daily.invalid_item_calories.gt(0)
    flags["nonpositive_item_sum"] = daily.item_calories.le(0)
    flags["nonpositive_daily_total"] = daily.food_calories.le(0)
    flags["missing_daily_total"] = daily.food_calories.isna()
    flags["total_discrepancy_over_one"] = (
        (daily.food_calories - daily.item_calories).abs().gt(1)
    )
    flags["daily_total_below_100_or_above_5000"] = daily.food_calories.lt(
        100
    ) | daily.food_calories.gt(5000)
    return flags


def run_audit():
```

```
    food, daily, baskets = load_inputs()
    assert food.Userid.notna().all() and food.date.notna().all()
    assert food.date.between("2016-01-01", "2016-12-31").all()
    require_unique(daily, ["Userid", "date"])
    # Repeated entries can be legitimate. Flag equality on these fields without deletion.
    fields = ["Userid", "date", "description", "category", "Calories"]
    duplicate_flags = food.duplicated(fields, keep=False)
    flags = daily_flags(daily)
    health = pd.read_csv(
        DATA / "daily_records.csv.gz",
        usecols=["Userid", "Fancydate", "Age_at_2016_01_01", "Currentweight"],
    )
    health = health.loc[health.Userid.isin(food.Userid.unique())]
    require_unique(health, ["Userid", "Fancydate"])
    age_versions = health.groupby("Userid").Age_at_2016_01_01.nunique(dropna=False)
    assert age_versions.eq(1).all()
    result = {
        "food_rows": len(food),
        "daily_rows": len(daily),
        "missing_category_rows": int(food.Category.isna().sum()),
        "rows_equal_on_selected_entry_fields": int(duplicate_flags.sum()),
        "missing_item_calorie_rows": int(food.Calories.isna().sum()),
        "negative_item_calorie_rows": int(food.Calories.lt(0).sum()),
        "flags_by_user_day": {key: int(value) for key, value in flags.sum().items()},
        "recorded_calorie_quantiles": {
            str(q): float(value)
            for q, value in daily.food_calories.quantile(
                [0, 0.01, 0.5, 0.99, 1]
            ).items()
        },
        "matched_daily_file_rows": len(health),
        "fixed_age_min": int(health.Age_at_2016_01_01.min()),
        "fixed_age_max": int(health.Age_at_2016_01_01.max()),
        "nonpositive_weight_rows": int(health.Currentweight.le(0).sum()),
    }
    save_result("audit", result)
    print(json.dumps(result, indent=2))


if __name__ == "__main__":
    run_audit()
```

1.9 Python Calculations for the Synthetic Cases

The inputs below are imposed teaching values. Running the program creates the labeled synthetic CSV files and prints the corresponding calculations. They are separate from the mHealth estimates. Keep the script beside `example_helpers.py` in `03_Data_and_Code/Lecture_Examples`.

Python source: example_helpers.py. From its directory, run `python example_helpers.py`.

```
"""File output and arithmetic helpers for synthetic lecture examples."""

import csv
import itertools
import json
import math
from pathlib import Path

ROOT = Path(__file__).resolve().parent


def write_csv(name, fields, rows):
    with (ROOT / name).open("w", newline="", encoding="utf-8") as f:
        out = csv.writer(f)
        out.writerow(fields)
        out.writerows(rows)


def ols_slope(points):
    xbar = sum(x for x, y in points) / len(points)
    ybar = sum(y for x, y in points) / len(points)
    slope = sum((x - xbar) * (y - ybar) for x, y in points) / sum(
        (x - xbar) ** 2 for x, y in points
    )
    return {"intercept": ybar - slope * xbar, "slope": slope}


def shares(utilities):
    weights = {key: math.exp(val) for key, val in utilities.items()}
    return {key: weight / sum(weights.values()) for key, weight in weights.items()}
```

Python source: m01_synthetic.py. From its directory, run `python m01_synthetic.py`.

```
"""Synthetic numerical examples for M1; run from any working directory."""

import itertools
import json
import math
from example_helpers import write_csv, ols_slope, shares

r = {}
lines = [("O1", "C1", 2, 10), ("O1", "C1", 1, 5), ("O2", "C2", 3, 4)]
write_csv(
    "M01_order_lines_synthetic.csv",
    ["order_id", "customer_id", "quantity", "unit_price_dollars"],
    lines,
)
orders = {}
for order, customer, qty, price in lines:
```

```
    orders[order] = orders.get(order, 0) + qty * price
r["M01"] = {
    "revenue": sum(orders.values()),
    "order_count": len(orders),
    "average_order_value": sum(orders.values()) / len(orders),
    "average_line_revenue": sum(orders.values()) / len(lines),
    "renewal_incremental_value_A": 100 * (0.91 - 0.90) - 3,
    "renewal_incremental_value_B": 100 * (0.50 - 0.40) - 3,
}
assert orders == {"O1": 25, "O2": 12}
print(json.dumps(r, indent=2))
```

1.10 mHealth: An Auditable Monthly Business Review

An mHealth manager needs to decide whether a decline in food entries warrants attention to customer engagement. Entry counts combine the number of eligible customers, the length of the reporting month, the frequency of logging, and the intensity of logging on observed days. A decline can arise from any of these components. The comparison becomes interpretable when the eligible population is fixed before the reporting periods and every component has an explicit denominator.

Define the population as users in the teaching cohort with at least one observed day before May 1, 2016. All members remain in the May and June denominators, including those with no logs in a month. A missing day contributes zero to the indicator for an observed log; its food consumption remains unobserved. For month t, the number of entries satisfies the accounting identity

$$L_t = ND_t p_t a_t,$$

where N is the fixed number of eligible users, D_t is calendar days in the month, p_t is observed user-days divided by ND_t, and a_t is entries per observed user-day. The identity separates calendar exposure from logging behavior. It uses counts of records, with no assumption that an entry represents a purchase or a complete meal.

Quantity	May	June
Eligible Users	340	340
Calendar User-Days	10,540	10,200
Observed User-Days	7,082	6,331
Observed Logging Fraction	0.6719	0.6207
Entries per Observed Day	11.7995	11.6910
Food Entries	83,564	74,016

The table identifies a 5.12 percentage-point decline in the logging fraction, alongside a shorter month and a modest decline in entries per observed day. Changing the denominator to users who happened to log during June would remove inactive members from the comparison. The fixed-population result supports investigation of engagement, recording practices, and seasonality. It supplies no estimate of a campaign effect or change in nutrition.

An AI-assisted business review should retrieve numerical claims from a computed evidence file. The file below records the source checksum, membership rule, observation unit, month, numerator, and denominator. A useful coding request is: "Construct the fixed cohort before May, count distinct user-dates, retain members with zero logs, and verify the accounting identity separately for each month." The independent calculation must finish before an assistant writes the managerial explanation.

The claude-ecom reference separates numerical computation from a narrative layer for ecommerce reviews. The mHealth calculation applies that separation to logging data. Its descriptive evidence file also supplies the structured-claim checks in M8.

Python source: reporting.py. From its directory, run `python reporting.py`.

```
"""Compute an auditable, fixed-cohort mHealth business review locally."""

import json
import pandas as pd
from common import ROOT, load_food, save_result


def monthly_review(food):
    # Fix membership before either reporting month; retain zero-log users.
    members = sorted(food.loc[food.date < "2016-05-01", "Userid"].unique())
    rows = []
    for month in ["2016-05", "2016-06"]:
        period = pd.Period(month, freq="M")
        part = food.loc[
            food.Userid.isin(members) & food.date.dt.to_period("M").eq(period)
        ]
        days = part[["Userid", "date"]].drop_duplicates()
        denominator = len(members) * period.days_in_month
        assert len(days) <= denominator
        rows.append(
            {
                "period": month,
                "eligible_users": len(members),
                "calendar_user_days": denominator,
                "observed_user_days": len(days),
                "users_with_logs": int(days.Userid.nunique()),
                "food_entries": len(part),
                "logging_fraction": len(days) / denominator,
                "entries_per_observed_day": len(part) / len(days),
            }
        )
    manifest = json.loads((ROOT.parent / "manifest.json").read_text())
    return {
        "source_sha256": manifest["files"]["food_entries.csv.gz"]["sha256"],
        "membership": "At least one observed day before 2016-05-01",
        "unit": "eligible user-calendar-day",
        "metric": "logging_fraction",
        "interpretation": "descriptive",
```

```
        "periods": rows,
    }


if __name__ == "__main__":
    result = monthly_review(load_food())
    save_result("reporting", result)
    print(json.dumps(result, indent=2))
```

Chapter 2

Customer Data, Measurement, and Visualization

A grocery retailer sees larger online baskets and asks whether moving customers to its app will raise sales. The raw comparison combines the shopping channel with who uses it, why they shop, and which trips are observed. A household may order online for a planned weekly restock and visit a store for a forgotten ingredient. Measurement and visualization determine whether the analysis reveals that distinction or conceals it.

2.1 Define the Population Before the Metric

A customer dataset is a record of an observation process. Loyalty accounts exclude unidentified shoppers. Online logs record sessions that may contain repeated visits by the same person. Orders exclude browsing without purchase. App events reflect tracking rules, consent, device settings, and instrumentation changes. Before calculating a metric, identify the target population and the mechanism by which its members enter the table.

Suppose monthly revenue per customer is intended to describe all eligible accounts. Dividing total revenue by purchasing accounts produces revenue per purchasing account. If inactivity changes, that measure can increase while revenue per eligible account falls. The denominator should therefore be written into the metric name or its definition. For customer acquisition cost, specify whether the denominator is new registered accounts, first-time purchasers, or customers with positive contribution over a defined horizon.

A conversion rate requires an eligible exposure count. If b eligible sessions produce at least one purchase among n eligible sessions, session conversion is b/n. Customer conversion instead uses customers in both numerator and denominator. A customer with ten sessions should not receive ten times the weight in a customer-level rate. When the same customer contributes repeated observations, statistical uncertainty must also account for that dependence.

Returns require a substantive rule. Gross sales, net sales after returns, and contribution after variable costs answer different questions. A comparison that deducts online returns but observes only gross offline sales mixes measurement rules. Currency, tax, fulfillment fees, and discounts should be aligned across channels before interpreting differences.

2.2 Build a Customer Table Without Losing Customers

Begin with a customer roster that defines eligibility. Aggregate eligible transactions to customer-period totals, then left-join those totals to the roster. A customer with no transactions can receive zero sales if the transaction system fully captures activity during the observation period. Zero should not replace a value that is missing because the customer was unobserved or the data feed failed.

A join should have a documented relationship: one-to-one, many-to-one, or one-to-many. Validate key uniqueness in the lookup table and inspect unmatched records. A match rate of 99% can still conceal a material problem if the unmatched 1% contains the highest-value customers or one entire channel. Report both counts and relevant value totals for unmatched observations.

Dates need similar attention. Convert timestamps to a consistent timezone before assigning day or week. Determine whether the date represents ordering, shipment, payment, or return. A promotion announced late in a period may affect orders immediately but recognized revenue later. Aligning metrics on an inappropriate date can create an apparent lag that comes from accounting rather than customer behavior.

Outliers should be investigated before removal. A $10,000 order could be a business customer, a data-entry error, or a legitimate event. A rule that trims all orders above a percentile changes the estimand to a restricted population. State that restriction and compare conclusions with and without the rule when it can affect the decision. Robust summaries such as medians and quantiles often reveal more than one mean with an undocumented exclusion.

2.3 Case Study: A Reversal Caused by Customer Mix

A marketing manager compares campaigns A and B using conversion. Both campaigns reach new and returning customers, but their audience composition differs. Assume the following customer counts and outcomes, with each customer belonging to one segment and appearing once.

Campaign	New Customers	New Conversions	Returning Customers	Returning Conversions	Overall Conversion
A	100	9	100	80	89/200 = 44.5%
B	200	20	100	81	101/300 = 33.7%

Campaign B has higher conversion among new customers, 10% versus 9%, and among returning customers, 81% versus 80%. Campaign A nevertheless has the higher aggregate rate because half its audience consists of returning customers, compared with one-third for B. Returning customers convert much more frequently in both campaigns.

The aggregate rate is a weighted average of segment rates:

$$\bar{p}_j = \sum_g w_{jg} p_{jg}, \qquad \sum_g w_{jg} = 1.$$

Here j indexes campaigns, g indexes customer segments, p_{jg} is the conversion fraction within a segment, and w_{jg} is that segment's share of the campaign audience. The weights are part of the comparison. Holding them at 50% new and 50% returning gives standardized rates of 44.5% for A and 45.5% for B. This adjustment describes performance for the chosen common mix. It establishes a causal campaign effect only if the design supports that interpretation.

B's small advantages within both segments coexist with A's aggregate advantage. A manager deciding where to send the next campaign needs to specify the future audience mix and whether customers were assigned comparably. An aggregate bar chart alone cannot answer that question.

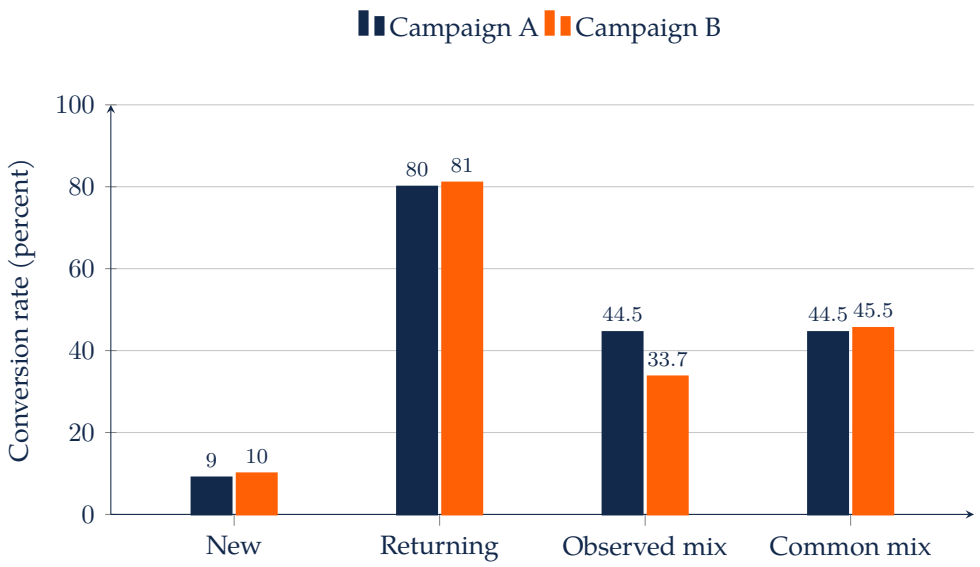


Figure 2.1. Audience composition reverses the aggregate campaign ranking.

2.4 Cohorts, Retention, and Customer Age

A cohort groups customers by a common starting event, such as first purchase or subscription activation. Retention at customer age k is the fraction of the original eligible cohort meeting a stated activity criterion at that age. A cohort table separates calendar time from customer age: an October signup observed one month later is not comparable with an April signup observed seven months later unless the question explicitly concerns those different ages.

For cohort c, define N_c as the number of eligible customers at entry and $A_{c,k}$ as the number active at age k. The descriptive retention rate is $R_{c,k} = A_{c,k}/N_c$. A later cohort may not yet have reached age k; that cell is unobserved, not zero. Omitting inactive customers from the denominator would turn retention into a different conditional measure.

Signup Cohort	Entry Customers	Active at Month 1	Active at Month 2	Active at Month 3
January	100	60	45	40
February	100	70	50	Not yet observed
March	100	80	Not yet observed	Not yet observed

Month-one retention rises from 60% for the January cohort to 80% for the March cohort. March customers have not yet reached the later observation windows, so their long-run retention remains unobserved. Comparing March's month one with January's month three would combine cohort quality with customer age. The right comparison depends on whether the decision concerns onboarding, long-term value, or a calendar-time change affecting all existing customers.

Unobserved future periods require forecasts from an explicit model. Coding those periods as zero would treat unavailable outcomes as observed failures and bias retention downward.

	At Entry	Month 1	Month 2	Month 3
January	100%	60%	45%	40%
February	100%	70%	50%	Not yet observed
March	100%	80%	Not yet observed	Not yet observed

Each cohort begins with 100 customers.
Only observed customer ages enter the retention denominator.

Figure 2.2. A cohort's unobserved future is different from zero retention.

2.5 Visual Encodings Should Match the Question

Use position on a common scale when precise comparison matters. A dot plot can compare segment conversion rates more clearly than area-sized bubbles. A line chart is appropriate when the ordering of time is meaningful. A histogram or empirical cumulative distribution reveals skewness that a mean hides. A scatterplot supports examination of a relationship, with transparency or aggregation when observations overlap.

The scale changes the interpretation. A logarithmic sales axis emphasizes proportional differences; a linear axis emphasizes dollar differences. A truncated bar-chart axis exaggerates differences because bar length encodes magnitude from its baseline. A carefully labeled narrow axis can be useful for a line or dot plot when the scientific question concerns small differences. The choice should preserve the intended comparison and disclose the scale.

Uncertainty should be shown for an estimated comparison when it affects a decision. For an independent Bernoulli sample, a common large-sample standard error for a proportion is $\sqrt{\hat{p}(1-\hat{p})/n}$. Near zero or one, or with small samples, use an interval appropriate to binomial data. With repeated observations per customer, treating every session as independent generally understates uncertainty. Resample or cluster at the unit that captures the relevant dependence.

Color can encode categories or highlight the comparison that matters. Use line styles, markers, and labels alongside color, and keep small text dark enough for contrast. A figure should remain interpretable when printed in grayscale or described to a student who cannot see it. Explain the axes, units, sample, and principal comparison in its caption or accompanying text.

2.6 Case Study: Online Grocery Baskets and Repeated Purchases

Chintala, Liaukonytė, and Yang's grocery-shopping study compares online and offline baskets using household shopping records. It studies basket composition and repeated purchasing, with attention to differences in shopping-trip types. Larger baskets need not contain a wider range of categories, and a household's choice of channel can be related to the purpose of its trip.

Suppose one household buys the sets {milk, bread, apples, yogurt} and {milk, bread, apples, coffee} on successive trips. Each basket contains four distinct items. Their Jaccard similarity is

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{3}{5} = 0.60.$$

The numerator counts shared distinct items and the denominator counts all distinct items appearing in either trip. A different pair of four-item baskets with one shared item has similarity $1/7 \approx 0.143$. Basket size is identical in the two examples, while repeated purchasing differs substantially. Whether quantities or spending weights should enter the measure depends on the decision; set-based Jaccard treats one unit and ten units of an item alike.

A defensible descriptive study would define eligible grocery trips, compare channels within households when appropriate, and examine basket size, distinct categories, repeat-item overlap, and spending. It would inspect whether findings are driven by large restocking trips, small convenience trips, or particular retailers. Channel labels should not silently determine the definition of a "typical" trip when the objective is a comparable classification. Clustering can classify trips using their observed composition.

A retail recommendation might distinguish discovery from replenishment. High repetition can support convenient reordering while making it harder for unfamiliar products to receive attention. A proposed interface change should therefore specify both the desired outcome and the possible tradeoff. Its effect requires evaluation under an appropriate design, even when the descriptive pattern provides a strong reason to test it.

2.6.1 Variety Depends on Both Breadth and Concentration

Two customers can purchase the same number of categories while allocating spending very differently. One divides a basket equally between two categories; another assigns 90% to one and 10% to the other. A category count assigns both customers breadth two. To measure how evenly spending is distributed, define $V = 1 - \sum_c s_c^2$, where s_c is category c's share of the customer's spending within a stated basket or period. The shares must use a common denominator and sum to one. The two baskets have $V = 0.50$ and $V = 0.18$, respectively.

Nielsen and coauthors (2022) use item counts and a reverse-coded concentration measure based on calorie shares to study food-consumption variety. The underlying unit matters: spending shares, calorie shares, and purchase counts can rank the same baskets differently. A wider category definition also changes measured variety without changing consumption. Report the category mapping, observation window, and share denominator before interpreting a trend or segment difference. M7, Section 7.12, connects these measurements to recommendation design.

2.7 From Evidence to a Management Presentation

The Pyramid Principle organizes a recommendation around a decision-relevant answer, followed by a small number of supporting reasons and the evidence for each reason. This structure works when the answer's strength matches the analysis. A descriptive study might recommend a test or a closer diagnostic comparison, rather than immediate deployment.

For the campaign example, a clear opening is: "Compare the campaigns at a common customer mix before reallocating the budget." The supporting reasons are that returning customers convert

more often, the campaigns reached different mixes, and the within-segment ranking differs from the aggregate ranking. The supporting evidence should quantify each reason and connect it to the allocation decision.

An effective chart title can state the comparison, such as "Campaign A's aggregate advantage reflects its returning-customer share." The caption should define the conversion denominator and identify the data as synthetic.

2.8 AI-Assisted Workflow and Practice

Ask an assistant to construct a customer-period table from a specified roster and transaction table, preserving customers with no observed purchases. Require a join audit, a missing-data report, and a reconciliation of total sales. Next ask it to calculate the campaign table above and standardized rates under a 50/50 audience mix. Verify those small numbers before applying the same functions to a larger file.

For a chart request, specify the question, observation unit, axis units, ordering, and uncertainty representation. Ask the assistant to describe what visual pattern would change the recommendation. Compare its description with the plotted data. If it labels an observational channel difference as an effect, revise the interpretation and identify the design that would be needed to support the stronger claim.

For practice, change campaign B's audience to 100 new and 100 returning customers while keeping its segment conversion rates fixed. Its aggregate rate becomes 45.5%, and the reversal disappears. Next calculate month-two retention for the January and February cohorts: 45% and 50%. March has no observed month-two rate. Finally, compare a four-item basket with itself: Jaccard similarity is one, regardless of whether the quantities have changed. Each exercise tests a definition that should remain explicit in an automated analysis.

2.9 Readings

Wilke's visualization text offers a visual reference for distributions, comparisons, and uncertainty. The pandas grouping documentation supports implementation.

2.10 mHealth: Recorded Variety and Missing Days

The manager wants to know whether customers with broad recorded diets need different suggestions from customers who repeatedly log a few categories. Category count measures breadth. A calorie-share measure also distinguishes an even distribution from a day dominated by one category. For user i, day t, and recorded category c, define k_{ict} as the sum of item calories in that category. On days with nonnegative observed item values and a positive item total, define

$$s_{ict} = \frac{k_{ict}}{\sum_h k_{iht}}, \qquad V_{it} = 1 - \sum_c s_{ict}^2.$$

The shares sum to one across recorded categories, including `Unclassified` where necessary. V_{it} approaches zero when one category accounts for almost all recorded item calories. It rises when calorie shares become more evenly distributed across categories. Multiplying every item calorie value by the same positive constant leaves these shares unchanged. This scale invariance helps distinguish composition from recorded quantity, although omitted foods can still change composition.

Among the 76,780 eligible user-days, mean variety is 0.7834 when each day has equal weight and 0.7855 when each user first contributes a personal average. The small difference in these means does not make the weighting choices interchangeable. A day-weighted comparison gives frequent loggers more influence. A user-weighted comparison answers a question about the average observed user.

Recorded Categories	Eligible Days	Mean Variety
1–5	11,861	0.5306
6–10	30,258	0.7895
11–20	32,492	0.8618
21 or More	2,169	0.9046

Breadth and calorie-share variety rise together in these groups, partly because adding categories expands the possible distribution of shares. This is a descriptive relation among recorded days. It does not show that the app caused broader consumption or that greater category variety improved health. Source categories are not a validated diet-quality index.

The complete 2016 calendar contains 135,420 potential user-days for the selected users. Only 76,804 have food entries; 58,616 are unobserved. A zero-filled calorie panel would interpret every absent record as no consumption. For a logging outcome, zero can instead mean that no entry was observed on a specified calendar day. The variable's definition determines whether filling a missing cell with zero is appropriate. These users are identified from a full-year file, so the calendar is a descriptive grid rather than a verified account-enrollment roster.

The item sum is within one calorie unit of the reported daily total on 53,075 observed days. Using the reported total as the denominator of category shares would fail to make shares sum to one on many other days. The calculation below preserves both measures, constructs the observed-day summary, and builds the calendar separately.

Python source: m02.py. From its directory, run `python m02.py`.

```
"""Measure recorded variety and separate observed days from calendar days."""

import json
import pandas as pd
from common import OUTPUT, load_inputs, save_result

food, daily, baskets = load_inputs()
eligible = daily.loc[daily.variety.notna()].copy()
eligible["breadth_group"] = pd.cut(
    eligible.category_count,
    [0, 5, 10, 20, 1000],
    labels=["1-5", "6-10", "11-20", "21+"],
```

```
)
table = eligible.groupby("breadth_group", observed=True).agg(
    days=("variety", "size"),
    mean_variety=("variety", "mean"),
    mean_recorded_calories=("food_calories", "mean"),
)
OUTPUT.mkdir(exist_ok=True)
table.to_csv(OUTPUT / "M02_breadth.csv")
assert int(table.days.sum()) == len(eligible)
calendar = pd.MultiIndex.from_product(
    [sorted(food.Userid.unique()), pd.date_range("2016-01-01", "2016-12-31")],
    names=["Userid", "date"],
)
panel = daily.set_index(["Userid", "date"]).reindex(calendar)
observed = panel.entries.notna()
result = {
    "eligible_days": len(eligible),
    "excluded_days": len(daily) - len(eligible),
    "calendar_user_days": len(panel),
    "observed_days": int(observed.sum()),
    "unobserved_days": int((~observed).sum()),
    "mean_variety_day_weighted": float(eligible.variety.mean()),
    "mean_variety_user_weighted": float(
        eligible.groupby("Userid").variety.mean().mean()
    ),
    "daily_total_equal_item_sum_within_one": int(
        (daily.food_calories - daily.item_calories).abs().le(1).sum()
    ),
    "daily_total_comparison_days": len(daily),
    "breadth": table.reset_index().to_dict(orient="records"),
}
save_result("M02", result)
print(json.dumps(result, indent=2))
```

2.11 mHealth: Nutrition Knowledge and Sample Restrictions

An mHealth analyst evaluating suggestions about diet must determine whether the records measure food selection, complete daily intake, or usual intake over a longer period. A day with one logged snack supplies evidence about that snack. It does not establish the person's total intake for the day. The National Cancer Institute's food-record guidance identifies recording burden and reactivity as features of the measurement process. Its usual-intake guidance distinguishes day-to-day variation from a person's longer-run average. Repeated logs reduce some sampling variation while selective omissions can persist.

Nutrition studies often collect information that makes filtering more defensible. The NHANES dietary codebook documents recall reliability through DR1DRSTZ. mHealth has no corresponding completeness indicator. Several food entries or a positive calorie total cannot be substituted for that validation. NHANES also permits reliable records involving fasting or only water, so a zero total

cannot universally be classified as erroneous. A ratio requiring a positive denominator nevertheless excludes zero totals by mathematical necessity.

For an adult descriptive analysis, the base sample requires age of at least 18 on January 1, positive finite reported daily calories, and a defined calorie-share variety measure. The fixed-age restriction removes users who were 17 at the reference date even if they turned 18 later in the year. It retains older adults; age 65 is not a general nutritional validity boundary. The combined restrictions retain 75,855 days from 367 users. They define the population and the calculations; they do not certify complete intake.

The National Cancer Institute's outlier guidance supports examining extreme observations and comparing analyses under alternative exclusions. Short-term intake can genuinely vary substantially. A population percentile screen is therefore a sensitivity analysis. The 1st and 99th percentiles of positive adult daily totals in January through June are 209.01 and 3,534.11. Fixing those cutoffs using the earlier period prevents later observations from choosing their own reference range. A second screen uses an explicitly imposed 100–5,000 range; a third requires item totals and reported totals to agree within one calorie unit.

Rule Applied to the Adult Base	Days Retained	Mean Reported Calories	Mean Variety
Adult Positive Base	75,855	1,541.82	0.7837
Imposed 100–5,000 Range	75,523	1,543.04	0.7863
Earlier-Period 1st–99th Percentiles	74,224	1,531.07	0.7916
Item/Day Agreement Within One	52,707	1,493.19	0.7671

Each row applies one screen independently to the same base. The agreement rule removes 23,148 days, far more than the percentile rule's 1,631. Its lower mean variety could reflect the kinds of records retained, including differences in logging intensity. Agreement is useful evidence about accounting consistency, but it does not prove nutritional completeness. A preference for cleaner-looking totals can therefore change the population being described.

The horizontal movement measures retention of adult base days; the vertical movement measures their mean recorded variety. The accounting-consistency restriction produces the largest movement in both dimensions. Treating it as a harmless technical correction would obscure a substantive change in the sample.

2.11.1 Energy Plausibility Requires Physiological Inputs

Energy-plausibility methods compare reported intake with expenditure or a model of requirements. Banna, McCrory, Fialkowski, and Boushey (2017) compare fixed cutoffs with individualized methods. Black (2000), especially Table 8, updates the Goldberg approach. For an individual under the method's energy-balance and measurement conditions, bounds for mean reported energy intake divided by basal metabolic rate are

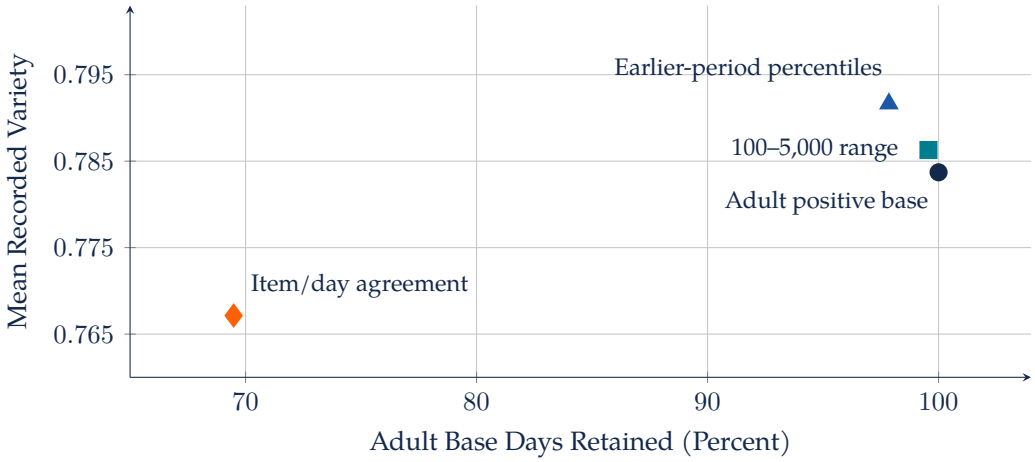


Figure 2.3. Alternative screens retain different samples and change measured variety.

$$L, U = \mathrm{PAL} \exp\left(\mp 2 \sqrt{\frac{(0.23)^2}{d} + (0.085)^2 + (0.15)^2} \right).$$

Here d is the number of assessment days and PAL is total energy expenditure divided by basal metabolic rate. The coefficients represent the variation assumptions for intake, predicted basal metabolic rate, and activity. The factor 2 gives the paper's approximate 95% limits for an individual; the sample-size term is one. These are conditional screening bounds, not an individual's recommended intake. Appropriate PAL and basal-metabolic-rate inputs are essential.

With an imposed basal metabolic rate of 1,500 kcal/day and PAL of 1.55, the bounds are approximately 1,308–4,131 kcal/day for one assessment day and 1,580–3,421 for seven days. Raising PAL to 1.80 raises the seven-day bounds to 1,835–3,973. A hypothetical seven-day mean of 1,600 would fall within the first interval and below the second. The same intake can receive a different flag because of the activity assumption. Additional days reduce the intake-variation component while the other variance components remain.

The code calculates these hypothetical bounds separately from the real-data screens. mHealth does not supply validated activity, basal metabolic rate, complete assessment days, or a documented mapping of `Gender` to the sex variable required by common prediction equations. No participant is assigned an individualized Goldberg label. Weight-loss goals and perturbed weights also complicate an energy-balance interpretation. Applying the equations to invented physiological inputs would turn an assumption into an apparent observation.

An AI coding request should therefore name the assessment instrument, population, units, necessary inputs, and output flags. Require the program to preserve raw values, distinguish overlapping exclusions, and return the analysis with and without the screen. Ask it to explain why one-day outliers, genuine fasting, and incomplete logging require different interpretations. The nutrition resource guide supplies the original methods and downloadable readings.

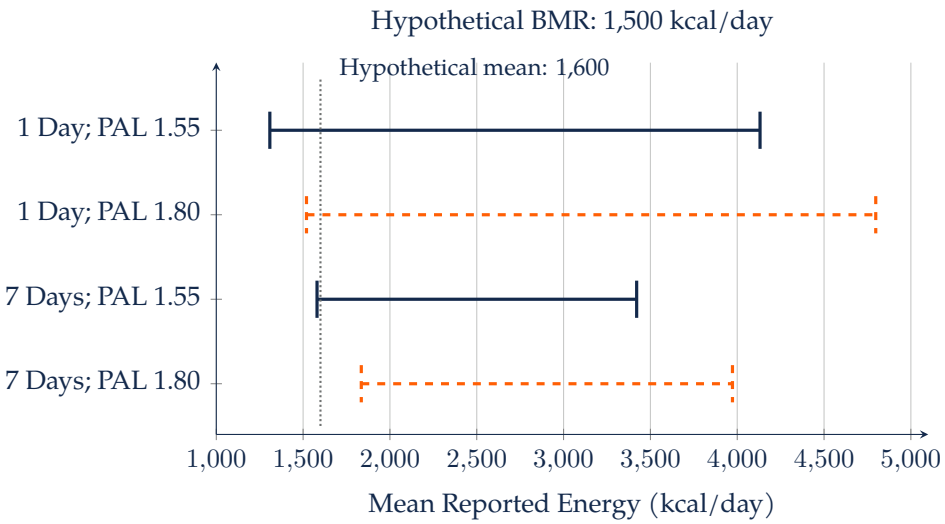


Figure 2.4. Assessment duration narrows an energy-plausibility interval while assumed activity shifts it.

Python source: nutrition.py. From its directory, run `python nutrition.py`.

```
"""Domain-informed sample definitions and explicitly hypothetical energy screens."""

import json
import math
import numpy as np
from common import load_inputs, save_result


def goldberg_bounds(days, pal, cv_intake=23, cv_bmr=8.5, cv_pal=15):
    """Individual EI:BMR bounds with Black (2000) coefficients, z=2, n=1."""
    if days < 1 or pal <= 0:
        raise ValueError("Positive assessment duration and PAL are required")
    variation = math.sqrt(cv_intake**2 / days + cv_bmr**2 + cv_pal**2) / 100
    return pal * math.exp(-2 * variation), pal * math.exp(2 * variation)


def run_analysis():
    food, daily, baskets = load_inputs()
    age_counts = food.groupby("Userid").Age_at_2016_01_01.nunique(dropna=False)
    assert age_counts.eq(1).all()
    ages = food.groupby("Userid").Age_at_2016_01_01.first()
    daily["age"] = daily.Userid.map(ages)
    # Nonnegative category calories and a positive denominator define valid shares.
    base = daily.loc[
        daily.variety.notna()
        & daily.food_calories.gt(0)
        & np.isfinite(daily.food_calories)
        & daily.age.ge(18)
    ].copy()
    training = base.loc[base.date.lt("2016-07-01")]
```

```
lower, upper = training.food_calories.quantile([0.01, 0.99])
masks = {
    "adult_positive_base": base.food_calories.gt(0),
    "fixed_100_to_5000_sensitivity": base.food_calories.between(100, 5000),
    "prior_period_1st_to_99th_percentile": base.food_calories.between(lower, upper),
    "daily_item_agreement_within_one": (base.food_calories - base.item_calories)
    .abs()
    .le(1),
}
summaries = []
for name, mask in masks.items():
    selected = base.loc[mask]
    summaries.append(
        {
            "rule": name,
            "days": len(selected),
            "users": int(selected.Userid.nunique()),
            "removed_from_base": len(base) - len(selected),
            "mean_recorded_calories": float(selected.food_calories.mean()),
            "mean_variety": float(selected.variety.mean()),
        }
    )
# Hypothetical complete records, stable energy balance, and predicted BMR.
# These BMR/PAL inputs are imposed; no mHealth participant is classified.
scenario = []
for days in [1, 7]:
    for pal in [1.55, 1.80]:
        lo, hi = goldberg_bounds(days, pal)
        scenario.append(
            {
                "days": days,
                "assumed_pal": pal,
                "bmr_kcal": 1500,
                "lower_ratio": lo,
                "upper_ratio": hi,
                "lower_kcal": 1500 * lo,
                "upper_kcal": 1500 * hi,
                "illustrative_1600_kcal_flag": 1600 / 1500 < lo,
            }
        )
result = {
    "source_observed_days": len(daily),
    "base_days": len(base),
    "base_removed_days": len(daily) - len(base),
    "age_min": float(ages.min()),
    "age_max": float(ages.max()),
    "training_days": len(training),
    "quantile_lower": float(lower),
    "quantile_upper": float(upper),
    "sensitivity": summaries,
    "hypothetical_goldberg": scenario,
```

```
    }
    save_result("nutrition", result)
    print(json.dumps(result, indent=2))


if __name__ == "__main__":
    run_analysis()
```

2.12 Python Calculations for the Synthetic Cases

The inputs below are imposed teaching values. Running the program creates the labeled synthetic CSV files and prints the corresponding calculations. They are separate from the mHealth estimates. Keep the script beside `example_helpers.py` in `03_Data_and_Code/Lecture_Examples`.

Python source: m02_synthetic.py. From its directory, run `python m02_synthetic.py`.

```
"""Synthetic numerical examples for M2; run from any working directory."""

import itertools
import json
import math
from example_helpers import write_csv, ols_slope, shares

r = {}
campaigns = [
    ("A", "new", 100, 9),
    ("A", "returning", 100, 80),
    ("B", "new", 200, 20),
    ("B", "returning", 100, 81),
]
write_csv(
    "M02_campaigns_synthetic.csv",
    ["campaign", "customer_type", "assigned_customers", "conversions"],
    campaigns,
)
r["M02"] = {
    "observed_A": 89 / 200,
    "observed_B": 101 / 300,
    "common_mix_A": (0.09 + 0.80) / 2,
    "common_mix_B": (0.10 + 0.81) / 2,
    "basket_jaccard_high": 3 / 5,
    "basket_jaccard_low": 1 / 7,
}
assert r["M02"]["observed_A"] > r["M02"]["observed_B"]
assert r["M02"]["common_mix_B"] > r["M02"]["common_mix_A"]
variety_shares = {"equal": [0.5, 0.5], "concentrated": [0.9, 0.1]}
r["M02"]["variety"] = {
    name: 1 - sum(s * s for s in vals) for name, vals in variety_shares.items()
}
write_csv(
```

```
    "M02_variety_synthetic.csv",
    ["basket", "category", "spending_share"],
    [
        (name, f"C{i+1}", share)
        for name, vals in variety_shares.items()
        for i, share in enumerate(vals)
    ],
)
assert math.isclose(r["M02"]["variety"]["equal"], 0.5)
assert math.isclose(r["M02"]["variety"]["concentrated"], 0.18)
print(json.dumps(r, indent=2))
```

2.13 mHealth: Recency and Frequency at a Fixed Date

A manager considering a reminder campaign needs to distinguish a customer who logs frequently from one who logged recently. Both customers may have the same total number of records. Recency measures elapsed time since the last observed activity, while frequency measures repeated activity within a specified window. Their interpretation depends on the date at which the manager must make the decision.

For every user with a recorded day by June 30, define recency as the number of days from the most recent recorded day to June 30. Define frequency as distinct recorded days during June 1–30. A customer who last logged on June 30 has recency zero. A customer with no June logs has frequency zero and remains eligible because an earlier record establishes membership. July activity is an outcome and enters neither feature.

The 358 eligible users are grouped using prespecified recency boundaries of 7 and 30 days and a frequency boundary of 15 days. These cutoffs define a descriptive comparison; they have not been optimized for a campaign. The empty high-frequency, high-recency cell follows from the definitions: a customer whose last log was more than 30 days ago has zero June logging days.

Recency on June 30	June Logging Days	Users	Any July Log	Mean July Logging Days
0–7 Days	0–14	68	92.6%	13.90
0–7 Days	15–30	220	96.8%	22.88
8–30 Days	0–14	34	67.6%	7.26
8–30 Days	15–30	6	83.3%	9.67
31+ Days	0–14	30	50.0%	5.70

Recent frequent loggers have a higher July logging rate in this cohort. The six-user cell warrants particular care because a single outcome changes its rate substantially. These rates describe future activity among known users; they do not identify which users would respond most to a reminder. Selecting recipients requires an incremental-effect comparison and a cost per contact.

Joao Correia's RFM example combines recency, frequency, and monetary value in transaction data. mHealth supports the first two dimensions. Calories, entry counts, and body weight have units different from spending and cannot supply monetary value. A commercial RFM table would need an actual monetary variable, a transaction definition, and a consistent observation window.

A coding assistant should receive the snapshot date and the eligible population explicitly. Ask it to verify that adding July rows leaves both June features unchanged, that repeated food entries on one day count only once toward frequency, and that zero-log members remain in the outcome denominator. Quantile bins require extra care when many customers share the same integer count; the fixed boundaries here avoid assigning tied counts to different groups.

Python source: recency.py. From its directory, run `python recency.py`.

```
"""Describe later logging using features available at a fixed snapshot."""

import json
import pandas as pd
from common import OUTPUT, load_food, make_daily, save_result


def snapshot(daily, cutoff):
    cutoff = pd.Timestamp(cutoff)
    history = daily.loc[daily.date <= cutoff]
    last = history.groupby("Userid").date.max()
    recent = history.loc[history.date > cutoff - pd.Timedelta(days=30)]
    counts = recent.groupby("Userid").date.nunique().reindex(last.index, fill_value=0)
    return pd.DataFrame(
        {"recency_days": (cutoff - last).dt.days, "frequency_30d": counts}
    )


if __name__ == "__main__":
    daily = make_daily(load_food())
    customers = snapshot(daily, "2016-06-30")
    customers["recency_group"] = pd.cut(
        customers.recency_days,
        [-1, 7, 30, float("inf")],
        labels=["0-7 days", "8-30 days", "31+ days"],
    )
    customers["frequency_group"] = customers.frequency_30d.ge(15).map(
        {True: "15-30 days", False: "0-14 days"}
    )
    july = daily.loc[daily.date.between("2016-07-01", "2016-07-31")]
    customers["july_days"] = (
        july.groupby("Userid").date.nunique().reindex(customers.index, fill_value=0)
    )
    customers["july_any_log"] = customers.july_days.gt(0)
    assert customers.frequency_30d.between(0, 30).all()
    summary = (
        customers.groupby(["recency_group", "frequency_group"], observed=True)
        .agg(
            users=("july_days", "size"),
            july_users=("july_any_log", "sum"),
            july_logging_probability=("july_any_log", "mean"),
            mean_july_days=("july_days", "mean"),
        )
```

```
    .reset_index()
)
result = {
    "snapshot": "2016-06-30",
    "users": len(customers),
    "groups": summary.to_dict(orient="records"),
}
save_result("recency", result)
summary.to_csv(OUTPUT / "recency.csv", index=False)
print(json.dumps(result, indent=2))
```

2.14 mHealth: Chart Choice, Denominators, and Customer Differences

A manager sees nearly unchanged activity on days when customers record food and considers reducing attention to retention. That comparison conditions on having a recorded day. A customer who stops logging disappears from the conditional average. A useful business review therefore distinguishes the frequency of recorded days from the amount of activity on those days, with the eligible population fixed before either reporting month.

The comparison retains the 340 users with at least one observed day before May 1, 2016. Each user contributes all 31 May days and all 30 June days to the logging denominator. Repeated food entries on one day count once toward logging coverage. A calendar day without a food entry receives zero for the logging indicator. Its food consumption remains unobserved. Food-entry counts retain the recorded rows and describe recording intensity.

Measure	May	June
Eligible Users	340	340
Users With Logs	317	313
Eligible Calendar User-Days	10,540	10,200
Days With Logs	7,082	6,331
Days With Logs / Eligible Days	67.19%	62.07%
Days With Logs / Logger Calendar Days	72.07%	67.42%
Entries per Recorded Day	11.80	11.69

The conditional logging rate divides by calendar days for users who logged at least once in that month. Its denominator changes with the outcome: May contains 317 such users and June contains 313. Both conditional rates exceed the corresponding fixed-population rates. Logging coverage in the fixed population falls by 5.12 percentage points, while entries per recorded day change little. Stable recording intensity among observed days can coexist with declining coverage of the eligible population. These patterns support examining disengagement and recording behavior; they do not identify a change in dietary intake or an effect of a reminder.

2.14.1 Match the Display to the Decision

The Financial Times Visual Vocabulary organizes displays by the relationship the reader needs to assess. The first choice is the comparison. A list of available plotting functions provides little

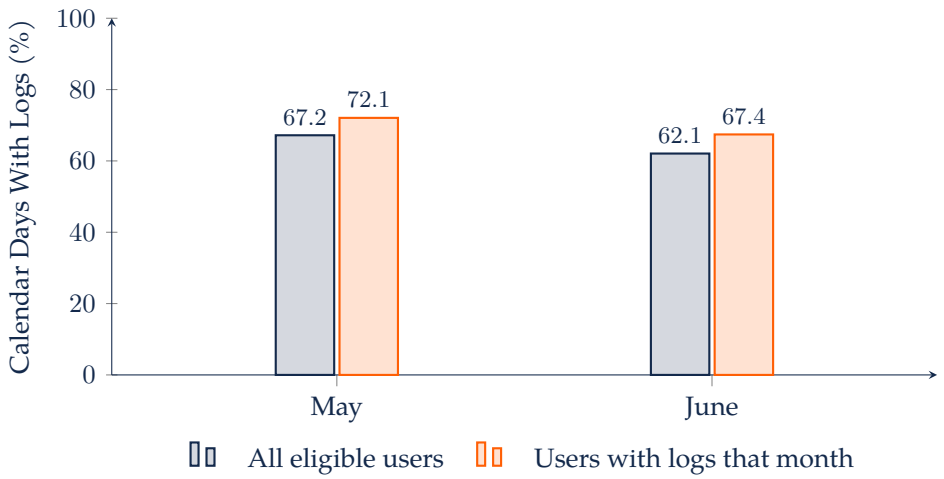


Figure 2.5. The same logged days produce different rates when the denominator excludes users with no logs.

guidance about whether the manager needs a ranking, a distribution, a trend, or an estimated contrast.

Analytical Question	Useful Display	Essential Check
Which segment is larger?	Sorted bars or dots	Common units and base
Where is the distribution concentrated?	Histogram or ECDF	Bins, ties, and weights
How does a metric change over time?	Line or paired dots	Calendar and membership
How do two variables move together?	Scatterplot or hexagons	Support and overlap
What share belongs to each category?	Aligned shares or bars	Common denominator
How uncertain is a contrast?	Point and interval	Estimand and sampling unit

A bar encodes its magnitude through length from a baseline, so the logging-rate bars start at zero. Two percentage axes with different limits would change the visual comparison even when the data are identical. A dot-and-interval plot can use a narrower, explicitly labeled scale because the position represents the estimate and the interval represents its uncertainty. For dense scatterplots, transparency, hexagonal bins, or a documented sample can reveal concentrations that overlapping opaque points hide. Bin boundaries and color limits should stay fixed when comparing customer groups.

Use Illinois Blue and Illini Orange to distinguish the principal series, with labels and line styles carrying the same information. A heatmap of a nonnegative count needs an ordered light-to-dark scale. A signed deviation needs a meaningful center, usually zero, with comparable positive and negative distances. Unobserved cells need a separate label or pattern so that a reader can distinguish them from observed zeros. Small multiples should retain a common scale when the decision depends on cross-panel magnitudes.

2.14.2 An Average Decline Can Coexist With No Median Change

The user-level outcome is the fraction of a month's calendar days with a log. For a value x between zero and one, its empirical cumulative distribution is

$$\widehat{F}_m(x) = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\{p_{im} \le x\},$$

where p_{im} is user i's logging fraction in month m and N is the fixed eligible-user count. The height at x is the fraction of users whose logging frequency is at or below that value. Each user has equal weight. Ties produce jumps, including the jump at zero for users without a recorded day. The curve avoids a histogram's bin-width choice, while its denominator and observation unit still require an explicit definition.

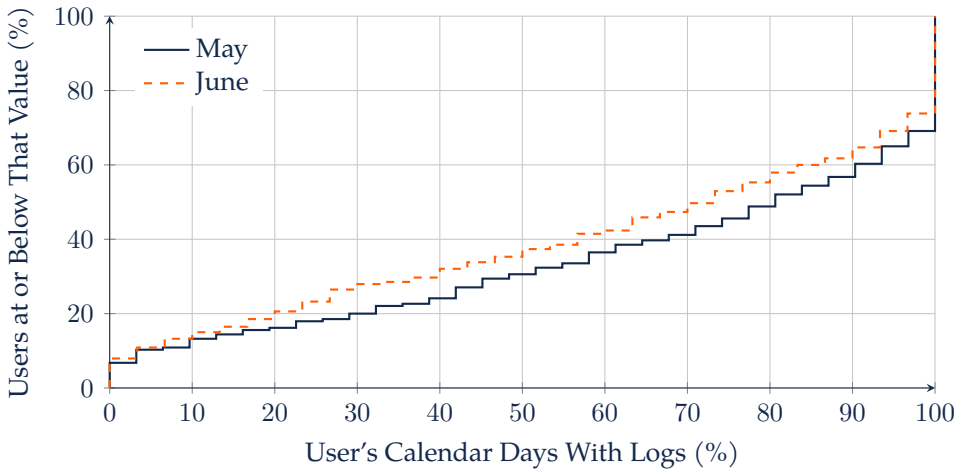


Figure 2.6. The logging-frequency distributions retain users at zero and reveal the concentration near daily recording.

The zero-logging share is 6.76% in May and 7.94% in June. At the other endpoint, 105 users log on every May day and 89 log on every June day. The median user's June-minus-May change is zero, while the mean change is negative. The middle half of individual changes spans approximately -17.58 to $+3.36$ percentage points. A mean decline therefore describes an aggregate change that is distributed unevenly across customers. It does not imply that every user reduced logging by five percentage points.

Histograms remain useful when the shape of a distribution matters. Compare several defensible bin widths and align bins across groups. A smooth density estimate requires extra care for bounded fractions with substantial mass at zero and one; smoothing can suggest observations outside the feasible range or conceal endpoint masses. The seaborn distribution tutorial discusses these choices. A distribution's spread and an estimate's uncertainty answer different questions, as the error-bar tutorial makes explicit. M3, Section 3.6, calculates uncertainty for the average paired change.

2.14.3 Reproduce the Comparison

The program selects membership before May, counts distinct logged days, creates complete user-month pairs, and exports only aggregate tables. It also calculates the paired-user interval used in M3. Missing-month users remain in the logging calculation. The 5,000 bootstrap draws resample users with both monthly outcomes together. The deliberately incorrect independent-day interval is retained with its assumption stated in its name.

Python source: visualization.py. From its directory, run `python visualization.py`.

```
"""Prepare aggregate chart data and paired uncertainty for May and June."""

import json
import numpy as np
import pandas as pd
from common import OUTPUT, ROOT, SEED, load_food, save_result

MONTHS = ["2016-05", "2016-06"]


def prepare(food):
    # Membership is fixed before May; an absent log is zero logging only.
    members = sorted(food.loc[food.date < "2016-05-01", "Userid"].unique())
    if not members:
        raise ValueError("No users observed before May")
    rates = pd.DataFrame(index=members)
    rows = []
    for month in MONTHS:
        period = pd.Period(month, freq="M")
        part = food.loc[
            food.Userid.isin(members) & food.date.dt.to_period("M").eq(period)
        ]
        counts = (
            part[["Userid", "date"]].drop_duplicates().groupby("Userid").size()
            .reindex(members, fill_value=0)
        )
        rates[month] = counts / period.days_in_month
        observed = int(counts.sum())
        active = int(counts.gt(0).sum())
        assert counts.between(0, period.days_in_month).all()
        rows.append({
            "period": month, "eligible_users": len(members),
            "users_with_logs": active, "observed_user_days": observed,
            "calendar_user_days": len(members) * period.days_in_month,
            "logger_calendar_days": active * period.days_in_month,
            "food_entries": len(part),
            "logging_percent": 100 * rates[month].mean(),
            "conditional_logging_percent": (
                100 * observed / (active * period.days_in_month) if active else None
            ),
            "entries_per_observed_day": len(part) / observed if observed else None,
```

```
        })
    return rates, pd.DataFrame(rows)


def ecdf(values):
    """Exact right-continuous CDF; tied values contribute their full mass."""
    values = np.asarray(values, dtype=float)
    if len(values) == 0 or not np.isfinite(values).all():
        raise ValueError("A nonempty finite sample is required")
    x, count = np.unique(values, return_counts=True)
    return pd.DataFrame({
        "logging_percent": 100 * x, "users_at_value": count,
        "cumulative_percent": 100 * count.cumsum() / len(values),
        "eligible_users": len(values),
    })


def paired_summary(rates, draws=5000):
    # Each sampled index carries both months for the same user.
    difference = rates[MONTHS[1]].to_numpy() - rates[MONTHS[0]].to_numpy()
    if len(difference) < 2 or not np.isfinite(difference).all():
        raise ValueError("At least two complete user pairs are required")
    rng = np.random.default_rng(SEED)
    indices = rng.integers(0, len(difference), size=(draws, len(difference)))
    bootstrap = difference[indices].mean(axis=1)
    low, high = np.quantile(bootstrap, [0.025, 0.975])
    return {
        "users": len(difference), "bootstrap_draws": draws, "seed": SEED,
        "change_pp": float(100 * difference.mean()),
        "ci_low_pp": float(100 * low), "ci_high_pp": float(100 * high),
        "paired_se_pp": float(100 * difference.std(ddof=1) / np.sqrt(len(difference))),
        "individual_change_q25_pp": float(100 * np.quantile(difference, 0.25)),
        "individual_change_median_pp": float(100 * np.median(difference)),
        "individual_change_q75_pp": float(100 * np.quantile(difference, 0.75)),
    }


def main():
    rates, monthly = prepare(load_food())
    comparison = paired_summary(rates)
    # Deliberately wrong independence assumption, retained as a teaching contrast.
    p = monthly.logging_percent.to_numpy() / 100
    n = monthly.calendar_user_days.to_numpy()
    naive_se = float(100 * np.sqrt(np.sum(p * (1 - p) / n)))
    comparison.update({
        "naive_independent_days_se_pp": naive_se,
        "naive_ci_low_pp": comparison["change_pp"] - 1.96 * naive_se,
        "naive_ci_high_pp": comparison["change_pp"] + 1.96 * naive_se,
    })
    cdf = pd.concat([ecdf(rates[m]).assign(period=m) for m in MONTHS], ignore_index=True)
    assert np.allclose(monthly.logging_percent,
```

```
                    100 * monthly.observed_user_days / monthly.calendar_user_days)
    assert cdf.groupby("period").users_at_value.sum().eq(len(rates)).all()
    OUTPUT.mkdir(exist_ok=True)
    monthly.to_csv(OUTPUT / "visualization_monthly.csv", index=False)
    cdf.to_csv(OUTPUT / "visualization_ecdf.csv", index=False)
    manifest = json.loads((ROOT.parent / "manifest.json").read_text())
    result = {
        "source_sha256": manifest["files"]["food_entries.csv.gz"]["sha256"],
        "membership": "At least one observed day before 2016-05-01",
        "observation": "user-month; fraction of calendar days with a food entry",
        "interpretation": "descriptive within the selected cohort",
        "months": monthly.to_dict(orient="records"), "paired_change": comparison,
    }
    save_result("visualization", result)
    print(json.dumps(result, indent=2))


if __name__ == "__main__":
    main()
```

2.15 Interactive Charts Must Preserve the Comparison

An interactive display can help a manager inspect alternative denominators and distribution shapes. Each control also changes the information being presented. A month selector should update the numerator, denominator, labels, and distribution together. Retaining a stale count after filtering would make a correct plotted height describe the wrong population. Holding axes at 0–100% makes changes across selections comparable.

The mHealth chart compares the all-user and observed-logger denominators for a selected month. Its second panel displays the selected month's distribution across the full eligible cohort. The data table remains visible below the chart, with the observation counts and denominators for both months. Hover labels supplement the table. A reader can verify each rate without inferring a value from a bar's height.

Altair's tutorial provides a declarative approach in which the chart specifies data fields, visual encodings, and interactions. The older tutorial's selection syntax differs from the version used here. The program uses Altair 5.5.0, a bound parameter, and explicitly fixed scales. The widget documentation and HTML-export documentation describe these operations. The exported HTML contains aggregate data and runs without a Python server. Its JavaScript libraries load from a public CDN, so chart rendering requires internet access.

Static figures and interactive views should agree on every plotted value. Run `visualization.py` before `visualization_charts.py`. The chart program checks the monthly table against the saved numerical results and uses the same aggregate data for both forms of display.

Python source: visualization_charts.py. From its directory, run `python visualization_charts.py`.

```
"""Build TikZ figures and a browser chart from verified aggregate tables."""

import json
import altair as alt
import pandas as pd
from common import OUTPUT, ROOT
from visualization import MONTHS

FIGURES = ROOT.parents[2] / "06_Lecture_Notes" / "figures"
BLUE, ORANGE = "#13294B", "#FF5F05"


def coordinates(x, y):
    return " ".join(f"({a:.8f},{b:.8f})" for a, b in zip(x, y))


def tikz_figures(monthly, cdf, result):
    FIGURES.mkdir(exist_ok=True)
    bars = r"""\begin{tikzpicture}
\begin{axis}[ybar,bar width=22pt,width=12.5cm,height=6.3cm,
 ymin=0,ymax=100,ylabel={Calendar Days With Logs (\%)},
 xtick={0,1},xticklabels={May,June},enlarge x limits=0.6,
 legend style={at={(0.5,-0.17)},anchor=north,legend columns=2,column sep=10pt},
 nodes near coords,point meta=y,
 every node near coord/.append style={font=\footnotesize,/pgf/number format/fixed,/pgf/number
 ↪  format/precision=1}]
\addplot[draw=IlliniBlue,fill=IlliniBlue!18] coordinates {ALL};
\addplot[draw=IlliniOrange,fill=IlliniOrange!18] coordinates {CONDITIONAL};
\legend{All eligible users,Users with logs that month}
\end{axis}
\end{tikzpicture}
"""
    bars = bars.replace("ALL", coordinates([0, 1], monthly.logging_percent))
    bars = bars.replace("CONDITIONAL", coordinates([0, 1], monthly.conditional_logging_percent))
    (FIGURES / "M02_logging_denominators.tex").write_text(bars)
    distribution = r"""\begin{tikzpicture}
\begin{axis}[width=12.5cm,height=6.5cm,xmin=0,xmax=100,ymin=0,ymax=100,
 xlabel={User's Calendar Days With Logs (\%)},ylabel={Users at or Below That Value (\%)},
 legend style={at={(0.03,0.97)},anchor=north west},grid=major]
"""
    for month, style in zip(MONTHS, ["IlliniBlue,solid", "IlliniOrange,dashed"]):
        part = cdf.loc[cdf.period.eq(month)]
        points = "(0,0) " + coordinates(part.logging_percent, part.cumulative_percent)
        distribution += rf"\addplot[{style},const plot] coordinates {{{points}}};" + "\n"
    distribution += "\\legend{May,June}\n\\end{axis}\n\\end{tikzpicture}\n"
    (FIGURES / "M02_logging_distribution.tex").write_text(distribution)
    comparison = result["paired_change"]
    intervals = r"""\begin{tikzpicture}
\begin{axis}[width=11cm,height=5.5cm,xmin=-10,xmax=1,ymin=-0.6,ymax=1.6,
```

```
 xlabel={June Minus May (Percentage Points)},ytick={0,1},
 yticklabels={{Independent days (incorrect)},{Paired user bootstrap}},
 grid=major]
\addplot[Storm,dashed] coordinates {(0,-0.5) (0,1.5)};
"""
    for y, low, high, color in [
        (1, comparison["ci_low_pp"], comparison["ci_high_pp"], "IlliniBlue"),
        (0, comparison["naive_ci_low_pp"], comparison["naive_ci_high_pp"], "Storm"),
    ]:
        points = coordinates([low, high], [y, y])
        intervals += rf"\addplot[{color},very thick,mark=|,mark size=5pt] coordinates {{{points}}};" +
        ↪  "\n"
        intervals += rf"\addplot[{color},only marks,mark=*] coordinates
        ↪  {{({comparison['change_pp']:.8f},{y})}};" + "\n"
    intervals += "\\end{axis}\n\\end{tikzpicture}\n"
    (FIGURES / "M03_logging_uncertainty.tex").write_text(intervals)


def interactive_chart(monthly, cdf):
    records = []
    for row in monthly.to_dict(orient="records"):
        for population, users, denominator, value in [
            ("All eligible users", row["eligible_users"], row["calendar_user_days"],
            ↪  row["logging_percent"]),
            ("Users with logs", row["users_with_logs"], row["logger_calendar_days"],
            ↪  row["conditional_logging_percent"]),
        ]:
            records.append({"period": row["period"], "population": population,
                            "users": users, "calendar_days": denominator,
                            "observed_days": row["observed_user_days"], "percent": value})
    table = pd.DataFrame(records)
    month = alt.param(name="month", value=MONTHS[0],
                      bind=alt.binding_select(options=MONTHS, name="Month: "))
    bars = alt.Chart(table).transform_filter(alt.datum.period == month).mark_bar().encode(
        x=alt.X("population:N", title=None, axis=alt.Axis(labelAngle=0)),
        y=alt.Y("percent:Q", title="Calendar Days With Logs (%)", scale=alt.Scale(domain=[0, 100])),
        color=alt.Color("population:N", scale=alt.Scale(range=[BLUE, ORANGE]), legend=None),
        tooltip=[alt.Tooltip("population:N"), alt.Tooltip("percent:Q", format=".2f"),
                 "users:Q", "observed_days:Q", "calendar_days:Q"],
    ).properties(width=320, height=300, title="The Denominator Changes the Rate")
    curve = alt.Chart(cdf).transform_filter(alt.datum.period == month).mark_line(
        interpolate="step-after", color=BLUE
    ).encode(
        x=alt.X("logging_percent:Q", title="User's Days With Logs (%)", scale=alt.Scale(domain=[0,
        ↪  100])),
        y=alt.Y("cumulative_percent:Q", title="Users at or Below (%)", scale=alt.Scale(domain=[0, 100])),
        order="logging_percent:Q",
        tooltip=[alt.Tooltip("logging_percent:Q", format=".2f"),
                 alt.Tooltip("cumulative_percent:Q", format=".2f"), "users_at_value:Q",
                 ↪  "eligible_users:Q"],
    ).properties(width=320, height=300, title="Distribution Across All Eligible Users")
```

```
    chart = alt.hconcat(bars, curve).add_params(month).configure_axis(
        labelColor=BLUE, titleColor=BLUE, labelFontSize=12, titleFontSize=12
    ).configure_title(color=BLUE, fontSize=15).configure_view(stroke=None)
    chart.save(OUTPUT / "visualization_interactive.vl.json")
    html = chart.to_html(embed_options={"renderer": "svg", "actions": False})
    table_text = table.rename(columns={
        "period": "Month", "population": "Population", "users": "Users",
        "calendar_days": "Calendar User-Days", "observed_days": "Days With Logs",
        "percent": "Days With Logs (%)",
    }).to_html(index=False, float_format=lambda x: f"{x:.2f}", border=0)
    n = int(monthly.eligible_users.iloc[0])
    introduction = (
        "<main><h1>mHealth: Logging Coverage</h1><p>Marketing Analytics · By Nathan Yang</p>"
        f"<p>{n} users observed before May 2016. The left chart compares two denominators. "
        "The right chart retains every eligible user, including users with no recorded days. "
        "Change the month to inspect both views. Both axes retain the same scales.</p>"
    )
    description = (
        "<h2>Chart Data</h2><p>Calendar user-days equal users multiplied by days in the month. "
        "No logged day means no observed food entry, without implying zero consumption. "
        "These descriptive comparisons do not estimate a treatment effect.</p>"
        + table_text
        + "<p>Internet access is required to display the charts. "
        "The data table remains available offline.</p></main>"
    )
    css = """<style>body{font:16px system-ui;color:#13294B;max-width:1000px;margin:32px auto;padding:0
    ↪  24px}
    h1{border-bottom:3px solid #FF5F05;padding-bottom:12px}p{line-height:1.55;max-width:850px}
    table{border-collapse:collapse;font-size:14px}td,th{padding:10px;border-bottom:1px solid
    ↪  #C6C7C6;text-align:right}
    #vis{max-width:100%;overflow-x:auto}th{background:#EEF1F5}select{font-size:16px;padding:6px}.vega-bi
    ↪  ndings{margin:15px
    ↪  0}</style>"""
    html = html.replace("</head>", css + "</head>").replace("<body>", "<body>" + introduction)
    html = html.replace("</body>", description + "</body>")
    (OUTPUT / "visualization_interactive.html").write_text(html)


if __name__ == "__main__":
    monthly = pd.read_csv(OUTPUT / "visualization_monthly.csv")
    cdf = pd.read_csv(OUTPUT / "visualization_ecdf.csv")
    result = json.loads((OUTPUT / "visualization.json").read_text())
    pd.testing.assert_frame_equal(monthly, pd.DataFrame(result["months"]), check_dtype=False)
    assert cdf.groupby("period").users_at_value.sum().eq(result["paired_change"]["users"]).all()
    tikz_figures(monthly, cdf, result)
    interactive_chart(monthly, cdf)
    print("Built three TikZ figures, the Vega-Lite specification, and interactive HTML.")
```

2.16 McDonald's: Measuring Ice-Cream Availability

A restaurant manager deciding where to investigate service interruptions needs to know which customers could not obtain the advertised product. A digital ordering system supplies one observation of availability. McBroken records an indicator named `is_broken` from the ordering channel. An unavailable item can reflect equipment failure, cleaning, a closed restaurant, depleted inventory, or a problem in the ordering system. A verification call can distinguish these mechanisms before a manager sends a technician or changes a promotion. The analytical outcome is therefore **recorded order unavailability**. A repair log would supply a different outcome.

The supplied McBroken collection contains US records from October 25, 2025, through October 24, 2026, plus a longer weekly country series. Rashiq Zahid's McBroken is the underlying source; MrFlynn's archive preserves successive snapshots. The archive's inspected workflow selects records with a reported check age below 61 minutes. Its older README describes a different freshness window, so the retrieval rule and version matter. An archive commit time records when a file was saved. The estimated check time also depends on the reported check age.

2.16.1 Coverage Determines the Population Behind a Percentage

Let R_{it} indicate that address key i appears in the selected snapshot on date t, and let U_{it} equal one when that record reports unavailability. The observed daily percentage is

$$100\widehat{p}_t = 100\frac{\sum_i R_{it}U_{it}}{\sum_i R_{it}}.$$

The denominator counts records observed in that snapshot. An absent record has an unknown status. It contributes to neither the numerator nor the denominator. Counting absence as availability would lower the reported percentage whenever collection coverage falls. The fraction also differs from the share of customer visits that encounter an unavailable item: customer arrivals and purchases supply the weights for that managerial outcome.

The audit reconciles the store panel with the supplied national totals and verifies unique store-date keys. The 365-date calendar contains US records on 356 dates, including three dates with fewer than 2,000 records. Nine dates have no US observations. The July 24, 2026, snapshot has only two US records, both reporting availability. Its zero percentage describes those two records. The missing dates remain missing in the course table.

An equal-day average gives each observed date the same weight. A pooled fraction gives greater weight to dates with more records. For these data, the equal-day average is 13.18% and the pooled record fraction is 13.37%. Restricting the calculation to dates with at least 2,000 records gives 13.22% and 13.37%, respectively. Requiring 5,000 records leaves 279 dates and gives 13.48% and 13.47%. These comparisons measure sensitivity to coverage restrictions; a count threshold alone does not make the retained sample representative. The 2,000-record rule is an explicit teaching choice.

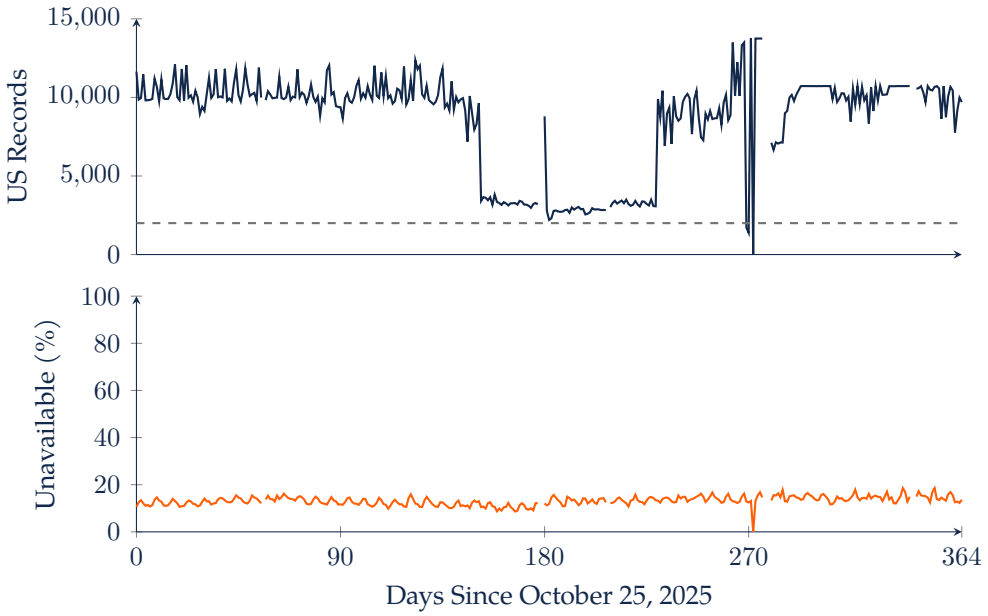


Figure 2.7. US record counts and recorded unavailability follow different series. The dashed count threshold is an imposed sensitivity screen, and gaps remain missing.

2.16.2 Address Keys and Restaurant Identity

The supplied dimension table contains 29,123 address-based keys and 13,821 exact coordinate pairs. Changes in street spelling can create a new key at the same coordinates. The discrepancy is a useful linkage diagnostic, although a coordinate match by itself does not establish that two records describe the same restaurant. Nearby outlets, rounded coordinates, and corrected locations require additional review. The calculations retain the supplied keys and record the coordinate count without merging outlets automatically.

The median key appears on 112 dates. Among 3,024,184 successive within-key record pairs, 929,451 span more than one calendar day. A row shift would silently describe many irregular intervals as one-day transitions. An exact join on key and next calendar date preserves the intended horizon. A key that changes spelling between those dates then lacks a linked outcome. This linkage choice narrows the forecast population and makes the missing-label count economically relevant.

2.16.3 Verify the Parser Before Treating a Status as Observed

JSON booleans, missing fields, and text strings require separate treatment. In Python, `bool(None)` returns `False`, whereas `bool("false")` returns `True`. Either conversion can produce a plausible but incorrect status if the input schema changes. An unrecognized check-age string also leaves the check time unknown; substituting zero minutes makes the observation appear newly collected. The parser below uses constructed edge cases to preserve these distinctions. The supplied CSV status column passes a binary-value check, but that check cannot recover information already lost during its original conversion.

Python source: api_checks.py. From its directory, run `python api_checks.py`.

```
"""Preserve unknown API values instead of inventing a working machine or check time."""
import re

import pandas as pd


def status(value):
    if value is True:
        return 1
    if value is False:
        return 0
    if value is None:
        return pd.NA
    raise ValueError(f"Expected a JSON boolean or null, received {value!r}")


def check_time(snapshot_utc, text):
    match = re.fullmatch(r"Checked (\d+) minutes? ago", text or "")
    if match is None:
        return pd.NaT
    return pd.Timestamp(snapshot_utc) - pd.Timedelta(minutes=int(match.group(1)))


if __name__ == "__main__":
    # Constructed edge cases for an AI-generated parser, not observed store data.
    values = pd.Series([status(v) for v in [True, False, None]], dtype="Int64")
    assert values.iloc[0] == 1 and values.iloc[1] == 0 and pd.isna(values.iloc[2])
    assert pd.isna(check_time("2026-10-24T18:00:00Z", None))
    try:
        status("false")
    except ValueError:
        print("A string status requires schema review; an unknown status remains missing.")
    else:
        raise AssertionError("String status was silently accepted")
```

2.16.4 Audit and Reproduce the Availability Comparisons

The preparation program checks unique keys, join coverage, status values, check ages, national totals, and the dimension table's observed-date counts. It creates a complete calendar before calculating rates. It also constructs exact next-day matches for Section 7.18, retaining the number of eligible origins when their next-day outcome is missing. The source-file hashes and selected archive commit identifiers keep the calculations traceable. The source bundle is required to rebuild these aggregates; the released aggregate tables support the subsequent programs directly.

Python source: prepare.py. From its directory, run `python prepare.py`. The default input is the supplied resource bundle; `--source PATH` selects another copy of its data directory.

```
"""Audit supplied McBroken tables and build aggregate, offline teaching inputs."""
import argparse
import hashlib
import json
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parent
DEFAULT_SOURCE = (ROOT.parents[1] / "01_Resources" /
                  "02_M2_Customer_Data_and_Visualization/mcbroken_api_lab/data")
PREFIX = "mcbroken_daily_2025-10_2026-10_"


def validate_panel(panel, dimension):
    if panel.duplicated(["date", "store_key"]).any():
        raise ValueError("Duplicate store-date records")
    if dimension.store_key.duplicated().any():
        raise ValueError("Duplicate dimension keys")
    if not panel.is_broken.isin([0, 1]).all():
        raise ValueError("Missing or nonbinary status")
    if not panel.store_key.isin(dimension.store_key).all():
        raise ValueError("Unmatched store key")
    if not panel.last_checked_min.between(0, 60).all():
        raise ValueError("Check age outside the supplied archive's range")


def make_transitions(panel, daily):
    """Match exact next calendar dates; retain the count of unlabeled origins."""
    good = daily.n_checked.ge(2000) & daily.offset_hours.abs().le(3)
    origin = panel[panel.date.isin(daily.loc[good, "date"])].copy()
    origin["date"] += pd.Timedelta(days=1)
    origin = origin[origin.date.le(daily.date.max())]
    origin = origin.rename(columns={"is_broken": "previous_unavailable"})
    target = panel[panel.date.isin(daily.loc[good, "date"])][
        ["date", "store_key", "is_broken"]]
    pairs = origin.merge(target, on=["date", "store_key"], how="left",
                         validate="one_to_one")
    return (pairs.groupby(["date", "previous_unavailable"]).is_broken
            .agg(n_origins="size", n_labeled="count", n_unavailable="sum")
            .reset_index())


def main(source):
    source = Path(source)
    used = []
    def read(name, **kwargs):
        path = source / name
```

```
    used.append(path)
    return pd.read_csv(path, **kwargs)

panel = read(PREFIX + "stores_us_panel.csv.gz", parse_dates=["date"])
dim = read(PREFIX + "stores_us_dim.csv")
country = read(PREFIX + "country.csv", parse_dates=["date"])
snaps = read(PREFIX + "snapshots_used.csv")
weekly = read("mcbroken_weekly_2020_2026_country.csv", parse_dates=["date"])
weekly_snaps = read("mcbroken_weekly_2020_2026_snapshots_used.csv")
validate_panel(panel, dim)
us = country[country.country.eq("USA")].copy()
if us.date.duplicated().any():
    raise ValueError("Duplicate US snapshot dates")
actual = panel.groupby("date").is_broken.agg(n_checked="size", n_broken="sum")
pd.testing.assert_frame_equal(actual.sort_index(),
    us.set_index("date")[["n_checked", "n_broken"]].sort_index(),
    check_dtype=False, check_freq=False)
days = panel.groupby("store_key").date.nunique()
pd.testing.assert_series_equal(days.sort_index(),
    dim.set_index("store_key").n_days.sort_index(), check_names=False)
snaps["snapshot_utc"] = pd.to_datetime(snaps.ts, utc=True)
snaps["date"] = snaps.snapshot_utc.dt.tz_localize(None).dt.normalize()
if snaps.date.duplicated().any():
    raise ValueError("Multiple selected snapshots per calendar date")
daily = pd.DataFrame({"date": pd.date_range(snaps.date.min(), snaps.date.max())})
daily = daily.merge(snaps[["date", "snapshot_utc", "sha"]], on="date",
                    how="left", validate="one_to_one")
daily = daily.merge(us[["date", "n_checked", "n_broken"]], on="date",
                    how="left", validate="one_to_one")
daily["share_unavailable"] = daily.n_broken / daily.n_checked
target = daily.date.dt.tz_localize("UTC") + pd.Timedelta(hours=18)
daily["offset_hours"] = (daily.snapshot_utc - target).dt.total_seconds() / 3600
transitions = make_transitions(panel, daily)
for col in ["n_origins", "n_labeled", "n_unavailable"]:
    transitions[col] = transitions[col].astype(int)
assert (transitions.n_unavailable <= transitions.n_labeled).all()
assert (transitions.n_labeled <= transitions.n_origins).all()

ordered = panel.sort_values(["store_key", "date"])
gaps = ordered.groupby("store_key").date.diff().dt.days
event = pd.Timestamp("2024-10-28")
weekly["event_week"] = (weekly.date - event).dt.days // 7
event_data = weekly[weekly.event_week.between(-12, 12)
                    & weekly.country.isin(["USA", "UK", "CA"])].copy()
event_data = event_data.merge(weekly_snaps[["period", "sha"]], on="period",
                              validate="many_to_one")
assert event_data.commit.eq(event_data.sha.str[:12]).all()
event_data["share_unavailable"] = event_data.n_broken / event_data.n_checked
out = ROOT / "data"
out.mkdir(exist_ok=True)
daily.to_csv(out / "daily_us.csv", index=False)
```

```
    transitions.to_csv(out / "transitions.csv", index=False)
    event_data.drop(columns="share_broken").to_csv(out / "policy_window.csv", index=False)
    audit = {
        "source_rows": len(panel), "address_keys": len(dim),
        "exact_coordinate_pairs": len(dim[["lat", "lon"]].drop_duplicates()),
        "calendar_days": len(daily), "us_observed_days": len(us),
        "missing_us_days": int(daily.n_checked.isna().sum()),
        "low_coverage_days": int(daily.n_checked.lt(2000).sum()),
        "successive_key_records": int(gaps.notna().sum()),
        "consecutive_day_pairs": int(gaps.eq(1).sum()),
        "long_gap_pairs": int(gaps.gt(1).sum()),
        "median_days_per_key": float(dim.n_days.median()),
        "snapshot_abs_offset_hours_max": float(daily.offset_hours.abs().max()),
        "duplicate_store_dates": 0, "unmatched_keys": 0,
        "country_counts_reconciled": True, "dimension_days_reconciled": True,
        "matching": "Original address keys; coordinates are an audit, not a merge rule.",
        "transition_screen": "Both dates: >=2000 US records and within 3 hours of 18:00 UTC.",
        "status_scope": "The supplied binary field was validated; raw API missingness cannot be
        ↪  recovered from these CSVs.",
    }
    (out / "audit.json").write_text(json.dumps(audit, indent=2) + "\n")
    manifest = {
        "source": "Rashiq Zahid, McBroken; snapshots via MrFlynn/mcbroken-archive",
        "urls": ["https://mcbroken.com", "https://github.com/MrFlynn/mcbroken-archive"],
        "files": {p.name: {"sha256": hashlib.sha256(p.read_bytes()).hexdigest(),
                           "bytes": p.stat().st_size} for p in used},
        "rights": "Upstream archive software uses Unlicense; no separate McBroken data license was
        ↪  established. Public teaching outputs are attributed aggregate calculations.",
    }
    (ROOT / "manifest.json").write_text(json.dumps(manifest, indent=2) + "\n")
    print(json.dumps(audit, indent=2))


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--source", type=Path, default=DEFAULT_SOURCE)
    main(parser.parse_args().source)
```

The coverage program reads the released calendar table, reports the sensitivity calculations, and generates the two-panel TikZ figure. Both panels use the same calendar. The percentage axis starts at zero and retains its full 0–100 range, so a change in the number observed remains visually distinct from a change in the observed fraction.

Python source: coverage.py. From its directory, run `python coverage.py`.

```
"""Compare availability denominators and draw the observed US time series."""
import json
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parent
FIGURES = ROOT.parents[1] / "06_Lecture_Notes/figures"


def main():
    daily = pd.read_csv(ROOT / "data/daily_us.csv", parse_dates=["date"])
    results = ROOT / "results"
    results.mkdir(exist_ok=True)
    rows = []
    for threshold in [0, 1000, 2000, 5000]:
        sample = daily[daily.n_checked.notna() & daily.n_checked.ge(threshold)]
        rows.append({"minimum_checked": threshold, "days": len(sample),
                     "checked_records": int(sample.n_checked.sum()),
                     "unavailable_records": int(sample.n_broken.sum()),
                     "equal_day_percent": 100 * (sample.n_broken / sample.n_checked).mean(),
                     "pooled_record_percent": 100 * sample.n_broken.sum() / sample.n_checked.sum()})
    pd.DataFrame(rows).to_csv(results / "coverage_sensitivity.csv", index=False)
    (results / "coverage.json").write_text(json.dumps({
        "missing_us_dates": daily.loc[daily.n_checked.isna(), "date"].dt.strftime("%Y-%m-%d").tolist(),
        "low_coverage_dates": daily.loc[daily.n_checked.lt(2000),
        ↪  "date"].dt.strftime("%Y-%m-%d").tolist(),
        "sensitivity": rows}, indent=2) + "\n")
    daily["day"] = (daily.date - daily.date.min()).dt.days
    def points(column, scale):
        return " ".join(f"({r.day},{getattr(r, column) * scale:.6f})"
                        if pd.notna(getattr(r, column)) else f"({r.day},nan)"
                        for r in daily.itertuples())
    figure = r"""\begin{tikzpicture}
\begin{axis}[name=counts,width=12.5cm,height=4.7cm,xmin=0,xmax=364,
 ymin=0,ymax=15000,ylabel={US Records},xtick=\empty,scaled y ticks=false,
 ytick={0,5000,10000,15000},unbounded coords=jump]
\addplot[IlliniBlue] coordinates {COUNTS};
\addplot[Storm,dashed] coordinates {(0,2000) (364,2000)};
\end{axis}
\begin{axis}[at={(counts.south west)},anchor=north west,yshift=-0.55cm,
 width=12.5cm,height=4.7cm,xmin=0,xmax=364,ymin=0,ymax=100,
 ylabel={Unavailable (\%)},xlabel={Days Since October 25, 2025},
 xtick={0,90,180,270,364},unbounded coords=jump]
\addplot[IlliniOrange] coordinates {RATES};
\end{axis}
\end{tikzpicture}
"""
    (FIGURES / "M02_mcbroken_coverage.tex").write_text(
```

```
        figure.replace("COUNTS", points("n_checked", 1))
              .replace("RATES", points("share_unavailable", 100)))
    print(pd.DataFrame(rows).to_string(index=False))


if __name__ == "__main__":
    main()
```

2.17 Public Signals of Demand for Generative AI

A subscription provider deciding where to invest in acquisition needs to distinguish public attention, product use, and payment. A high position in an app chart establishes an ordering within that chart. It supplies neither the number of downloads nor the fraction of consumers who pay. A pricing page describes the offers available at a particular date, while transactions reveal which offers customers purchase. Combining these sources is useful when each measure retains its observation unit and population.

Source	Recorded Object	Relevant Interpretation
App Store top-free chart	App rank in one storefront and capture	Ordinal position among listed apps; absence means outside the returned list or otherwise absent from the feed
Wikipedia pageviews	View events for a title, project, access method, and agent category	Attention to selected pages; repeated views and unpaid users can contribute
OpenRouter model list	Listed model, listing timestamp, and posted token prices	Available API offers at capture; neither realized token demand nor verified public launch dates
Ramp AI Index	Paid-AI adoption among businesses in its stated transaction sample	Business spending, with a population different from consumers and app users
Consumer subscription panel	Consumer-provider-month payment status	Adoption, simultaneous subscriptions, and transitions under the panel's coverage rules

The supplied generative-AI resource bundle contains dated public captures and a separate simulated subscription panel. The captures are historical snapshots. The simulated panel's counts describe its imposed population and behavioral rules.

2.17.1 Title Changes and Apparent Demand Declines

A measurement system can record a substantial decline when attention moves between page titles. The supplied English Wikipedia capture contains two Claude titles, `Claude_(language_model)` and `Claude_(AI)`. The Wikimedia definition distinguishes view events from people and explains that redirects are not automatically added to the target page's views. A title-specific series therefore requires a stable definition of the pages being counted.

Restrict the comparison to June through September 2026 and the two named titles. Both title counts are present on all 122 calendar days. Within each month, sum the recorded daily events for each title exactly once. June and September each contain 30 days, so the endpoint comparison uses equally long windows.

Month	Older Title	Newer Title	Sum of Selected Titles
June 2026	173,190	52,103	225,293
July 2026	25,873	148,027	173,900
August 2026	19,768	132,501	152,269
September 2026	25,872	161,884	187,756

The older title falls 85.1% between June and September. Summing the two selected titles yields a 16.7% decline. The difference changes the managerial interpretation of the apparent loss of attention. The combined series remains a count of events on these pages. It supplies no conversion rate from readers to subscribers, and a person can generate multiple events across titles.

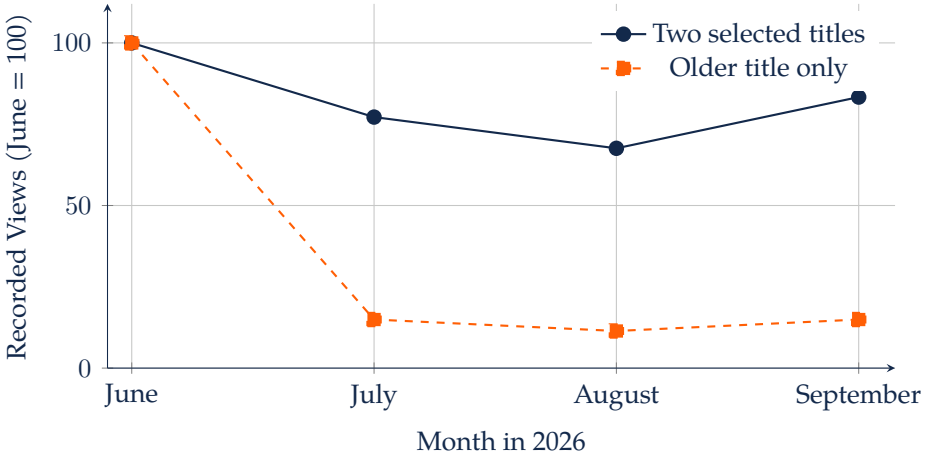


Figure 2.8. Recorded Wikipedia views indexed to June 2026. The older-title index falls to 14.9 in September, while the sum of the two selected titles falls to 83.3. Monthly totals use the supplied capture; both titles have complete daily records in the selected window.

2.17.2 Completeness, Identity, and AI-Assisted Extraction

An API response and a successful parser provide separate pieces of evidence. Repeated requests through different aliases can return the same title-date observation. Deduplicate by the returned title and date, verify that repeated values agree, and count the observation once. Preserve the retrieval status separately from the value: a failed request or an absent record does not by itself establish zero views. The code below builds the full calendar and returns an unknown combined count whenever either required title-day is missing.

For a public-price table, retain the model or plan identifier, currency, billing period, region, and capture date. Separate a monthly subscription amount from USD per million input or output tokens. A missing price remains unknown, and a routing service's variable price needs a sepa-

rate status. A listing timestamp identifies inclusion in that marketplace; an event study of public releases needs independently checked announcement dates and access conditions.

An AI-generated collector should be tested with three small inputs: a repeated title-date, conflicting values for that key, and a day on which one title is absent. The expected behavior follows from the observation unit. It should deduplicate the first case, reject the conflict, and preserve the missing value in the third. A plausible-looking chart is insufficient evidence that these rules were implemented.

Python source: measurement.py. From the script's folder, run `python3 measurement.py`. Inputs are read from the supplied local bundle; output tables are written beside the course scripts.

```
"""Audit the supplied Wikipedia capture without making network requests."""
from pathlib import Path
import json

import numpy as np
import pandas as pd

HERE = Path(__file__).resolve().parent
SOURCE = HERE.parents[1] / "01_Resources/08_M8_Current_Topics_in_Marketing/genai_demand_lab/data"
OUTPUT = HERE / "results"
TITLES = ["Claude_(language_model)", "Claude_(AI)"]


def unique_views(frame):
    """Count each title-date once; reject conflicting repeated responses."""
    keys = ["date", "article"]
    if frame[keys + ["provider", "views"]].isna().any().any():
        raise ValueError("Missing title, date, provider, or view count")
    if not np.isfinite(frame.views).all() or (frame.views < 0).any():
        raise ValueError("View counts must be finite and nonnegative")
    if (frame.views % 1 != 0).any():
        raise ValueError("View counts must be integers")
    conflicts = frame.groupby(keys)[["provider", "views"]].nunique()
    if (conflicts > 1).any().any():
        raise ValueError("Conflicting repeated title-date responses")
    return frame.drop_duplicates(keys).copy()


def title_calendar(frame, titles, start, end):
    """A missing title-day remains unknown; zero must be explicitly recorded."""
    frame = unique_views(frame)
    dates = pd.date_range(start, end, freq="D")
    selected = frame.loc[frame.article.isin(titles)]
    wide = selected.pivot(index="date", columns="article", values="views")
    wide = wide.reindex(index=dates, columns=titles)
    wide["observed_titles"] = wide[titles].notna().sum(axis=1)
    wide["selected_title_views"] = wide[titles].sum(axis=1, min_count=len(titles))
    wide.index.name = "date"
    return wide
```

```
def main():
    raw = pd.read_csv(SOURCE / "wikipedia_pageviews_daily_2026-10-26.csv", parse_dates=["date"])
    clean = unique_views(raw)
    daily = title_calendar(clean, TITLES, "2026-06-01", "2026-09-30")
    rows = []
    for month, group in daily.groupby(daily.index.to_period("M")):
        row = {"month": str(month), "calendar_days": len(group),
               "complete_days": int(group.selected_title_views.notna().sum())}
        for column in TITLES + ["selected_title_views"]:
            # Full-month totals are missing if even one title-day is unknown.
            row[column] = group[column].sum(min_count=len(group))
        rows.append(row)
    monthly = pd.DataFrame(rows)
    assert monthly.complete_days.eq(monthly.calendar_days).all()
    june, september = monthly.iloc[0], monthly.iloc[-1]
    result = {
        "input_rows": len(raw), "unique_title_days": len(clean),
        "exact_duplicates_removed": len(raw) - len(clean),
        "selected_titles": TITLES, "complete_days": int(daily.selected_title_views.notna().sum()),
        "old_title_change_pct": 100 * (september[TITLES[0]] / june[TITLES[0]] - 1),
        "selected_titles_change_pct": 100 * (september.selected_title_views / june.selected_title_views
        ↪  - 1),
        "measurement": "Recorded pageview events for two selected English Wikipedia titles; not unique
        ↪  people or payments",
        "capture_status": "Supplied dated CSV, not a new retrieval; API responses and rename history
        ↪  were not re-collected",
    }
    # Independent aggregate check using the long input rather than the pivot.
    selected = clean.loc[clean.article.isin(TITLES) & clean.date.between(daily.index.min(),
    ↪  daily.index.max())]
    direct = selected.groupby(selected.date.dt.to_period("M")).views.sum().to_numpy()
    np.testing.assert_array_equal(monthly.selected_title_views, direct)
    OUTPUT.mkdir(exist_ok=True)
    daily.to_csv(OUTPUT / "wikipedia_title_days.csv", float_format="%.8f")
    monthly.to_csv(OUTPUT / "wikipedia_months.csv", index=False, float_format="%.8f")
    (OUTPUT / "measurement.json").write_text(json.dumps(result, indent=2) + "\n")
    print(json.dumps(result, indent=2))


if __name__ == "__main__":
    main()
```

Chapter 3

Regression, Interpretation, and Marketing Decisions

A retailer observes that stores with larger advertising budgets sell more. The manager wants to know whether another dollar of advertising would be profitable. The observed relationship combines the response to advertising with differences in store size, local demand, seasonality, and the firm's reasons for choosing the budget. Regression provides a way to summarize conditional relationships. Its value depends on which variation identifies the comparison and whether that comparison answers the manager's question.

3.1 A Conditional Mean and a Regression Line

Let Y_i be weekly store sales measured in thousands of dollars and X_i be weekly advertising expenditure in thousands of dollars. A simple linear model writes

$$Y_i = \beta_0 + \beta_1 X_i + \varepsilon_i.$$

The intercept β_0 is the fitted sales level at zero advertising; its practical interpretation is weak when zero lies far outside the observed range. The slope β_1 is the change in fitted sales, in thousands of dollars, associated with an additional thousand dollars of advertising. The residual captures the difference between observed sales and the fitted line. Linearity is an approximation to the relationship over the relevant range, not a claim that every customer's behavior is linear.

Ordinary least squares chooses coefficients to minimize the sum of squared residuals. In a simple regression with an intercept, its slope is

$$\hat{\beta}_1 = \frac{\sum_i (X_i - \bar{X})(Y_i - \bar{Y})}{\sum_i (X_i - \bar{X})^2}.$$

The numerator measures whether high-advertising observations tend to have high sales. The denominator measures how much advertising varies. When advertising hardly varies, the data provide little information about a slope, even if the sample contains many rows. A large sample of nearly identical budgets does not substitute for informative variation.

For three observations $(X, Y) = (1, 3), (2, 5), (3, 7)$, the fitted line is $\hat{Y} = 1 + 2X$. At $X = 2.5$, it predicts sales of \$6,000. If the sales contribution margin is 30%, an additional \$1,000 of advertising is associated with \$600 of contribution from \$2,000 of extra sales. Treating that association as a return to intervention requires an identification argument. Even under a causal interpretation, the incremental contribution is less than the additional advertising expense in this numerical example.

3.2 Multiple Regression and the Comparison It Uses

A multiple regression can include observed store characteristics:

$$Y_i = \beta_0 + \beta_1 X_i + \gamma Z_i + \varepsilon_i,$$

where Z_i might be floor area measured in thousands of square feet. The advertising coefficient summarizes variation in sales associated with advertising after removing the linear association with floor area. A useful way to understand this is to regress advertising on the other controls, regress sales on the same controls, and regress the resulting sales residuals on the advertising residuals. The Frisch-Waugh-Lovell result gives the same advertising coefficient as the full regression when the residualization includes the same intercept and controls.

"Controlling for" a variable therefore changes the comparison. It does not automatically remove all confounding. If managers observe local demand information that the analyst lacks, their advertising choices may remain related to unobserved sales determinants. Controls measured after the intervention can also remove part of the effect or introduce bias. Select variables using the decision timing and a substantive account of how the data were generated.

When an omitted variable affects both the predictor and the outcome, the simple slope can be misleading. In the linear population model above, suppose the regression disturbance is mean-independent of X and Z. Omitting Z yields a simple-regression slope equal to

$$\beta_1 + \gamma \frac{\text{Cov}(X, Z)}{\text{Var}(X)}.$$

The extra term has a substantive interpretation. If larger stores sell more, so $\gamma > 0$, and larger stores spend more on advertising, so the covariance is positive, the simple advertising slope exceeds the conditional slope. The equation is a model-based decomposition; its interpretation depends on the stated linear specification and exogeneity condition.

3.3 Case Study: Attractive Markets and Apparent Competitive Benefits

Igami and Yang (2016) study entry and competition among hamburger chains in Canada. Their setting motivates a general identification problem: attractive markets can support both more competitors and better outcomes for a focal firm. A positive relationship between rival presence and performance can therefore coexist with a negative competitive effect.

Within low-demand markets, suppose stores facing one, two, and three rivals have sales of 9, 8, and 7 units. Within high-demand markets, stores facing four, five, and six rivals have sales of 16, 15, and 14 units. Within each demand group, one additional rival is associated with one unit less sales. Across the six observations, the pooled slope is approximately $+1.57$: high-demand markets have more rivals and higher sales.

The within-group slope is -1, while baseline demand differs between groups. The positive pooled slope reflects that demand composition. In an actual retail application, market demand may be unobserved, entry may be forward-looking, and firms may respond strategically.

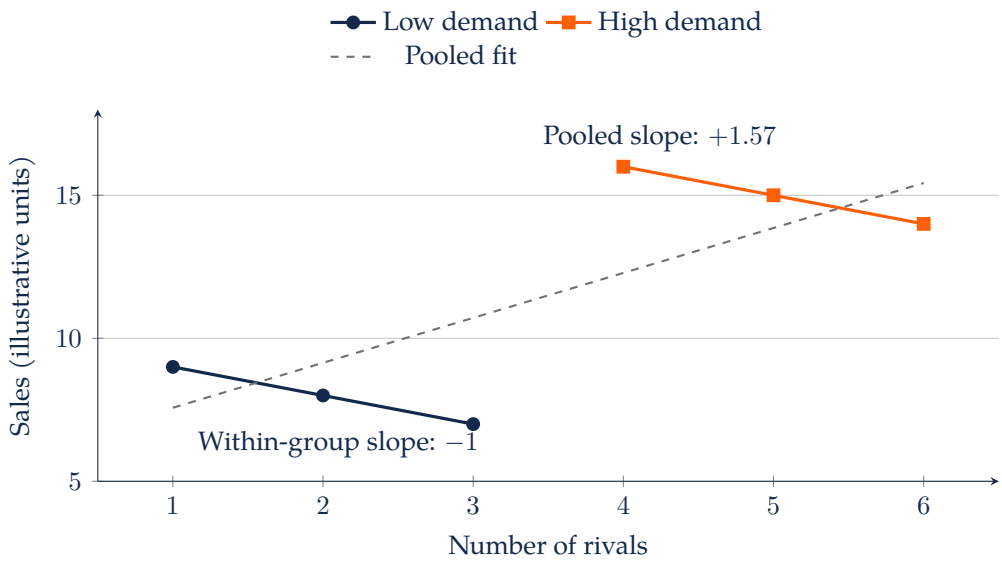


Figure 3.1. A positive pooled relationship can conceal a negative within-market-type relationship.

A manager considering entry should ask which outcomes would change if an additional competitor entered the same market. Comparing different markets does not automatically provide that counterfactual. Market fixed effects, measured demand controls, an instrument, or a credible event design may improve the comparison, but each uses different variation and requires its own conditions. A highly predictive regression can remain unsuitable for interpreting the coefficient as competition's causal effect.

3.4 Transformations and Economically Meaningful Units

A level-level specification gives an additive interpretation: one more unit of X changes fitted Y by β_1. A log-level specification, $\log Y = \beta_0 + \beta_1 X + \varepsilon$, describes proportional differences in a positive outcome. A one-unit change in X changes the fitted log outcome by β_1; the corresponding multiplicative change in the log-scale prediction is e^{β_1}, or $100(e^{\beta_1} - 1)$ percent. The approximation $100\beta_1$ percent is useful when the coefficient is small.

A log-log demand model, $\log Q = \alpha + \beta \log P + \varepsilon$, gives β an elasticity interpretation over the range where the specification applies. If $\beta = -1.5$, a 1% higher price is associated with approximately 1.5% lower quantity. Sales revenue is PQ, so its local price elasticity is $1 + \beta = -0.5$. Profit also depends on unit cost. A price that raises revenue can still lower profit, and observed prices can be endogenous because firms set them in response to demand.

Zero and negative outcomes need deliberate treatment. Taking a logarithm of zero is undefined. Adding one changes the scale and interpretation, especially when values are small or the unit of measurement changes. A two-part model can separate purchase incidence from positive expenditure. A suitable generalized linear model may model the conditional mean directly. The choice should reflect the outcome and intended estimand, with diagnostics for the selected specification.

Back-transforming a predicted log outcome also needs care. Exponentiating the predicted conditional mean of $\log Y$ generally does not produce the conditional mean of Y, because exponenti-

ation is nonlinear. Retransformation methods or a model for the mean on the original scale may be needed when total sales is the target. Evaluate predictions in the units used for the managerial decision.

3.5 Interactions Describe How a Relationship Changes

Suppose an analyst expects a promotional message to work differently for customers with long tenure. A linear interaction specification is

$$Y_i = \beta_0 + \beta_1 D_i + \beta_2 T_i + \beta_3 D_i T_i + \varepsilon_i,$$

where D_i is a message indicator and T_i is tenure in years. The fitted message difference at tenure t is $\beta_1 + \beta_3 t$. The coefficient β_1 is the difference at zero tenure, so centering tenure at a meaningful value can make that coefficient easier to interpret. The interaction coefficient tells how the fitted difference changes per year of tenure.

With $\beta_1 = 4$ dollars and $\beta_3 = -1$ dollar per year, the fitted difference is \$4 at tenure zero, \$2 at two years, and zero at four years. An average coefficient can conceal that variation. Predictions beyond the observed tenure range should be treated as extrapolation. In a nonlinear probability model, an interaction coefficient on the model's index does not directly equal the interaction in probability; compute and plot the predicted probabilities or marginal contrasts.

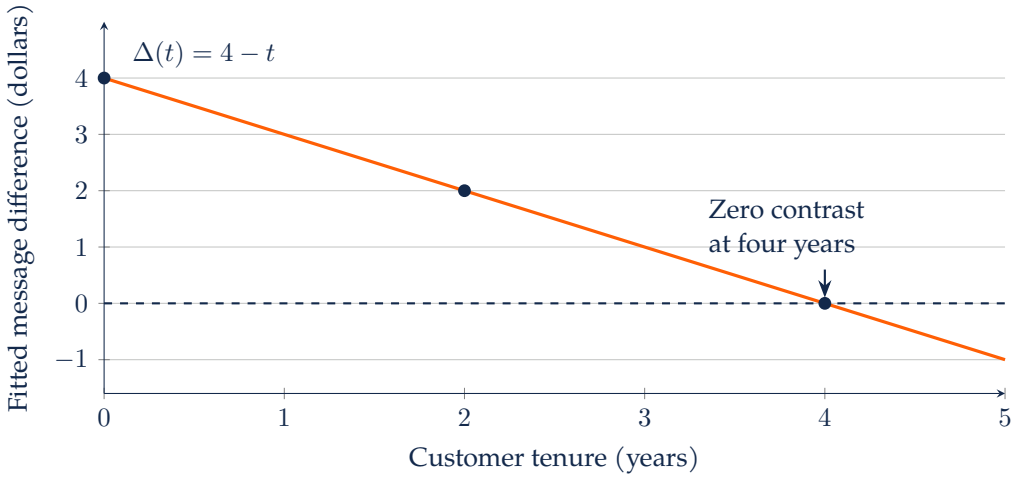


Figure 3.2. An interaction changes the relevant comparison across customer tenure.

For tenure between zero and five years, the assumed contrast $4 - t$ crosses zero at four years. In an estimated application, uncertainty around the contrast and support at each tenure would determine whether such a crossing is informative.

3.6 Uncertainty, Dependence, and Practical Importance

A standard error describes sampling uncertainty under an estimator and its assumptions. A confidence interval provides a range generated by a procedure with a stated repeated-sampling cov-

erage property. A p-value assesses compatibility with a particular null under its reference model. None of these quantities measures the managerial value of an effect by itself. A precisely estimated increase of one cent may be unimportant; a wide interval around a large potential benefit may justify another experiment.

For an approximately normal coefficient estimate, a rough 95% interval is the estimate plus or minus 1.96 standard errors, with appropriate adjustments when a finite-sample reference distribution is needed. If a sales slope is 2.0 with a standard error of 0.5, that interval is approximately [1.02, 2.98] in the slope's units. Report the units and the comparison alongside the interval.

Heteroskedasticity means that residual variance differs across observations. Heteroskedasticity-robust standard errors can address that feature for inference under suitable sampling conditions; they do not repair omitted-variable bias. Repeated store-week observations may share persistent store shocks. Clustered standard errors allow dependence within a specified cluster under their asymptotic conditions. The relevant number of independent clusters can be much smaller than the number of rows. With few clusters, conventional approximations may be unreliable and design-specific methods are needed.

3.6.1 A Paired Change Requires Paired Uncertainty

A manager comparing May and June logging has two observations for each eligible mHealth user. The target is the average within-user change in the fraction of calendar days with a recorded entry. Let p_{iM} and p_{iJ} denote the May and June fractions and define $d_i = p_{iJ} - p_{iM}$. With users treated as independent sampling units, an estimated standard error for the average change is

$$\widehat{\mathrm{SE}}(\bar{d}) = \frac{s_d}{\sqrt{N}}, \qquad s_d^2 = \frac{1}{N-1}\sum_i (d_i - \bar{d})^2.$$

This calculation preserves the dependence between a user's two outcomes. The variance identity $\mathrm{Var}(p_{iJ} - p_{iM}) = \mathrm{Var}(p_{iJ}) + \mathrm{Var}(p_{iM}) - 2\,\mathrm{Cov}(p_{iJ}, p_{iM})$ explains why separately resampling the two months changes the uncertainty calculation. Persistent user differences can cancel in a paired contrast. Dependence among days within a user also means that the number of calendar rows overstates the number of independent sampling units.

For the fixed population defined in M2, Section 2.14, the average change is -5.12 percentage points. Resampling complete users with both months together gives a 95% percentile-bootstrap interval of $[-7.93, -2.43]$ percentage points. The standard error calculated from the dispersion of user-level changes is 1.38 percentage points. An independent-Bernoulli-day calculation gives 0.66 percentage points and the narrower interval $[-6.42, -3.82]$. That calculation discards the observed dependence structure. A plotting library's default error bar can conceal this difference if the analyst supplies calendar rows without specifying the resampling unit.

The bootstrap treats observed user histories as draws from a common user distribution, with independence across users and both months fixed. It does not incorporate shocks shared across the entire platform or establish representativeness beyond this selected cohort. Both intervals concern a descriptive temporal difference. A causal claim about a product change would require an assignment design and uncertainty appropriate to that design. The full `visualization.py` program in M2, Section 2.14.3, calculates the monthly table, paired bootstrap, and independent-day comparison; `visualization_charts.py` generates the interval figure from those results.

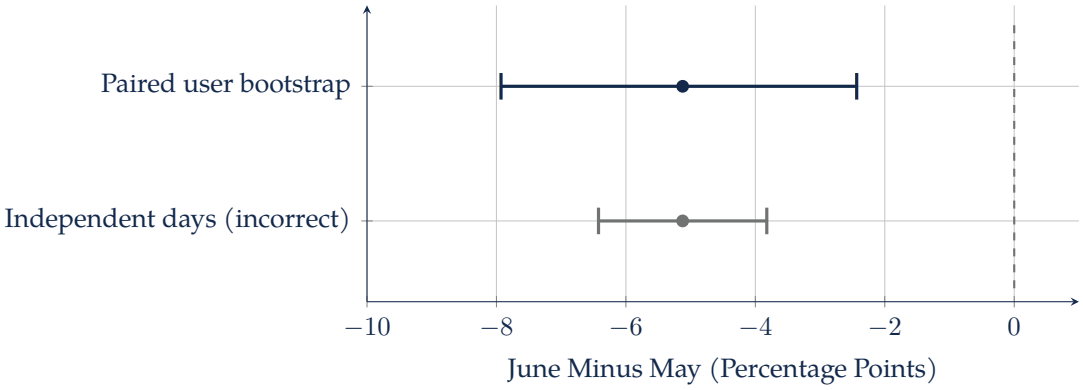


Figure 3.3. The same descriptive mean change has different intervals when the sampling unit is incorrectly changed from users to independent days.

3.7 Diagnostics and Panel Data

Inspect residuals against fitted values and important predictors for nonlinear patterns and changing variance. Investigate influential observations, errors in units, and unusual leverage. Multicollinearity can make individual coefficients unstable even when predictions are reasonably stable, because several predictors explain nearly the same variation. Removing a variable solely to obtain a smaller p-value changes the question and can introduce omitted-variable bias.

Store fixed effects absorb time-invariant differences between stores. Calendar fixed effects absorb common period-specific shocks. In a store-week panel, a model with both uses changes within stores relative to common time movements. It still requires a valid account of time-varying confounders. If a store raises advertising precisely when an unobserved local event increases demand, the advertising coefficient can remain biased.

A fitted model should be evaluated on the task for which it is used. For prediction, compare held-out errors with a useful baseline. For a descriptive coefficient, inspect support and sensitivity to substantively justified specifications. For a causal effect, assess the design and identifying conditions. A large R^2 indicates in-sample fit relative to an intercept-only benchmark; it does not establish a credible intervention comparison.

For repeated food logs, a customer fixed effect can absorb persistent differences in eating habits while relating changes in variety to changes in recorded consumption. It leaves time-varying motivation, activity, and reporting behavior as possible influences. The variety analysis in Nielsen and coauthors (2022) provides a concrete application. Percentage-change outcomes also require attention to zero denominators: excluding days with zero prior vegetable or snack consumption changes the sample relative to an analysis of total calories. Interpret each coefficient for its own observed population and outcome definition.

3.8 AI-Assisted Implementation and Practice

Ask an assistant to fit the pooled and demand-group-adjusted regressions in the six-observation example. Require a table with variable definitions, units, coefficients, and the exact sample. Verify

that the adjusted rival coefficient is -1 and the pooled coefficient is $11/7 \approx 1.57$. Ask it to explain why adding demand-group indicators changes the comparison. A numerical match without that explanation is an incomplete interpretation.

For a larger dataset, specify the formula and desired covariance estimator before requesting code. Ask the assistant to list dropped rows and explain how categorical variables were encoded. Have it calculate a predicted contrast at two values of a moderator and compare the result with the interaction formula. Treat diagnostic recommendations as hypotheses to inspect, rather than automatic permission to remove observations or alter the model.

For practice, consider an advertising coefficient measured per \$1,000 of spending. Re-express the predictor in dollars and explain how the numerical coefficient changes while fitted sales remain the same. Next explain why a robust standard error cannot eliminate the bias in the market-demand example. Finally, distinguish a confidence interval for the conditional mean from a prediction interval for a new store's realized sales: the latter also reflects individual outcome variation.

3.9 Readings

An Introduction to Statistical Learning provides a foundation for regression, model assessment, and nonlinear extensions. The statsmodels regression documentation supports implementation.

3.10 mHealth: Breadth and Next-Day Recorded Calories

The manager observes that users with broader daily food logs often report more calories and considers using breadth to identify customers who may want different suggestions. The relevant first comparison concerns timing: does today's recorded breadth predict the next calendar day's reported total? Taking the next available row would turn a one-day comparison into an interval of variable length when customers skip days.

Let K_{it} be today's recorded category count and $E_{i,t+1}$ the next calendar day's reported calories. Among positive consecutive-day pairs, estimate

$$\log E_{i,t+1} = \alpha_i + \beta K_{it} + \gamma_{m(t)} + \delta_{w(t)} + u_{it}.$$

The customer effect α_i absorbs time-invariant differences in typical logging and intake. Month and weekday effects describe calendar variation. The coefficient compares days with different recorded breadth for the same user, after these additive controls. It remains an association: changing appetite, omitted meals, or recording effort can affect both breadth and later totals. The NCI discussion of weekday effects motivates attention to calendar composition.

The sample requires positive reported totals on both dates and at least two eligible pairs per user. It contains 69,648 pairs from 370 users. Standard errors are clustered by user to allow dependence among overlapping pairs. The pooled model uses the same month and weekday controls but omits customer effects. No prior-outcome control or calorie winsorization is included in either baseline specification.

Specification	Pairs	Breadth Coefficient	Clustered 95% Interval
Pooled	69,648	0.02771	[0.01934, 0.03609]
Customer Fixed Effects	69,648	0.01284	[0.00997, 0.01572]
Fixed Effects, 100–5,000 on Both Days	69,216	0.01136	[0.00893, 0.01380]

In the fixed-effects model, one additional category corresponds to a 1.29% higher conditional geometric mean of next-day reported calories, calculated as $100(e^{\beta} - 1)$. This percentage does not automatically describe the arithmetic mean. The pooled estimate is larger, consistent with persistent differences across users contributing to the pooled association. Its reduction after adding fixed effects does not identify a causal mechanism.

The third row applies the explicitly imposed review range from M1 to both dates. It removes 432 pairs and produces a 1.14% geometric-mean association. This is a sensitivity comparison on a changed sample, not a validated nutrition screen. The baseline outcome ranges from 0.64 to 9,190, so the restriction directly addresses observations influential on the log scale. The adult and percentile screens in M2 answer different sample questions and should be named separately when used.

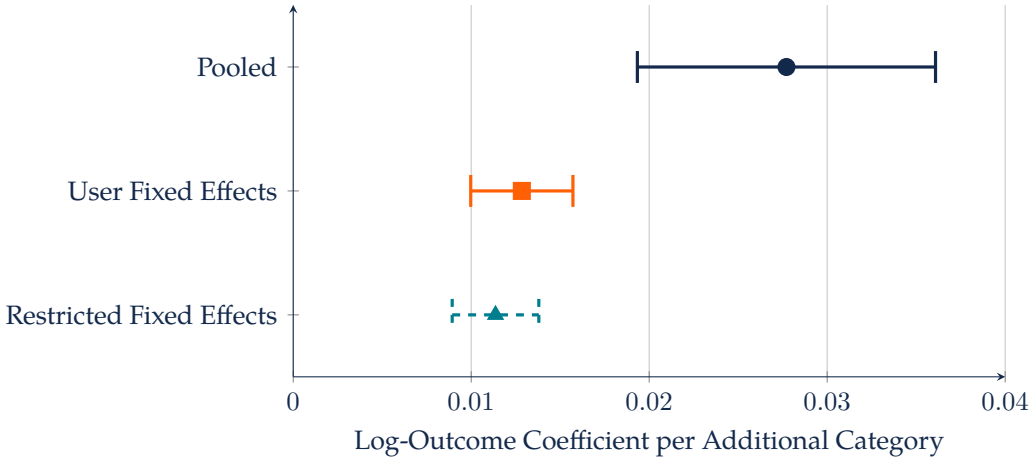


Figure 3.4. Customer controls and a calorie-range sensitivity restriction change the descriptive coefficient.

The intervals quantify sampling uncertainty under the regression and clustering specification. They do not account for all reporting error or make breadth an assigned treatment. A recommendation to induce greater breadth needs evidence about how a particular intervention changes behavior and outcomes.

The date shift below moves future outcomes back exactly one day before the merge. The program then applies each sample rule explicitly and reports pair and user counts. An AI-generated `groupby(...).shift(-1)` without a date-gap check would answer a different question.

Python source: m03.py. From its directory, run `python m03.py`.

```
"""Estimate descriptive next-calendar-day associations with clustered uncertainty."""

import json
import numpy as np
import pandas as pd
import statsmodels.formula.api as smf
from common import load_inputs, save_result

food, daily, baskets = load_inputs()
future = daily[["Userid", "date", "food_calories"]].copy()
future["date"] -= pd.Timedelta(days=1)
future = future.rename(columns={"food_calories": "next_calories"})
panel = daily.merge(future, on=["Userid", "date"], validate="one_to_one")
panel = panel.loc[panel.food_calories.gt(0) & panel.next_calories.gt(0)].copy()
counts = panel.groupby("Userid").size()
panel = panel.loc[panel.Userid.isin(counts[counts >= 2].index)].copy()
panel["log_next"] = np.log(panel.next_calories)
panel["month"] = panel.date.dt.month
panel["weekday"] = panel.date.dt.dayofweek
assert (panel.next_calories > 0).all()
restricted = panel.loc[
    panel.food_calories.between(100, 5000) & panel.next_calories.between(100, 5000)
].copy()
counts = restricted.groupby("Userid").size()
restricted = restricted.loc[restricted.Userid.isin(counts[counts >= 2].index)]
fits = {}
base = "log_next ~ category_count + C(month) + C(weekday)"
specifications = [
    ("pooled", base, panel),
    ("user_fixed_effects", base + " + C(Userid)", panel),
    ("restricted_fixed_effects", base + " + C(Userid)", restricted),
]
for name, formula, sample in specifications:
    fit = smf.ols(formula, data=sample).fit(
        cov_type="cluster", cov_kwds={"groups": sample.Userid}, use_t=True
    )
    coefficient = float(fit.params["category_count"])
    low, high = map(float, fit.conf_int().loc["category_count"])
    fits[name] = {
        "pairs": len(sample),
        "users": int(sample.Userid.nunique()),
        "coefficient": coefficient,
        "se": float(fit.bse["category_count"]),
        "ci_low": low,
        "ci_high": high,
        "geometric_mean_percent": 100 * float(np.expm1(coefficient)),
    }
result = {
    "pairs": len(panel),
```

```
    "users": int(panel.Userid.nunique()),
    "min_next_calories": float(panel.next_calories.min()),
    "max_next_calories": float(panel.next_calories.max()),
    "models": fits,
}
save_result("M03", result)
print(json.dumps(result, indent=2))
```

3.11 Python Calculations for the Synthetic Cases

The inputs below are imposed teaching values. Running the program creates the labeled synthetic CSV files and prints the corresponding calculations. They are separate from the mHealth estimates. Keep the script beside `example_helpers.py` in `03_Data_and_Code/Lecture_Examples`.

Python source: m03_synthetic.py. From its directory, run `python m03_synthetic.py`.

```
"""Synthetic numerical examples for M3; run from any working directory."""

import itertools
import json
import math
from example_helpers import write_csv, ols_slope, shares

r = {}
low = [(1, 9), (2, 8), (3, 7)]
high = [(4, 16), (5, 15), (6, 14)]
write_csv(
    "M03_rivals_synthetic.csv",
    ["demand_group", "rival_count", "sales_illustrative_units"],
    [("low", x, y) for x, y in low] + [("high", x, y) for x, y in high],
)
r["M03"] = {
    "pooled": ols_slope(low + high),
    "low": ols_slope(low),
    "high": ols_slope(high),
    "simple_sales_fit": ols_slope([(1, 3), (2, 5), (3, 7)]),
    "coefficient_ci": [2 - 1.96 * 0.5, 2 + 1.96 * 0.5],
}
assert math.isclose(r["M03"]["pooled"]["slope"], 11 / 7)
assert r["M03"]["low"]["slope"] == r["M03"]["high"]["slope"] == -1
print(json.dumps(r, indent=2))
```

3.12 Price Elasticity When Prices Respond to Demand

A retailer considers raising a price and wants the resulting percentage change in quantity. Historical prices can be high precisely when managers expect strong demand. A regression of quantity on price then combines the response to price with information used to choose it. Good predictive fit does not separate these two sources of variation.

Consider a synthetic population of independent markets. Let p_i be log price in dollars per unit and q_i log quantity sold. A standardized cost shifter z_i, an unobserved demand shock u_i, and independent shocks v_i, e_i are mutually independent standard normal variables. The imposed price rule and demand equation are

$$p_i = 2 + 0.4z_i + 0.6u_i + 0.2v_i, \qquad q_i = 5 - 1.5p_i + u_i + 0.2e_i.$$

The demand elasticity is -1.5: a small 1% increase in price changes quantity by approximately -1.5% with the demand shock held fixed. The coefficient 0.6 makes price respond positively to unobserved demand. The cost shifter changes price and has no direct effect on quantity in this benchmark. These equations are imposed behavioral relationships, without an estimated supply model or a solved pricing equilibrium.

For the general linear equation $q_i = \alpha + \beta p_i + \varepsilon_i$, an instrumental-variable interpretation rests on the following conditions.

Assumption 3.1 (Price Instrument)**.** *Markets are independent draws with finite second moments. The demand equation has a common coefficient β. The cost shifter satisfies* $\mathrm{Cov}(z_i, \varepsilon_i) = 0$ *and* $\mathrm{Cov}(z_i, p_i) \neq 0$.

Under Assumption 3.1, the ratio $\mathrm{Cov}(z_i, q_i)/\mathrm{Cov}(z_i, p_i)$ equals β. The numerator measures quantity variation associated with the cost shifter, and the denominator measures the induced price variation. Ordinary least squares instead has probability limit $\beta + \mathrm{Cov}(p_i, \varepsilon_i)/\mathrm{Var}(p_i)$, which is less negative in the benchmark because favorable demand shocks raise price.

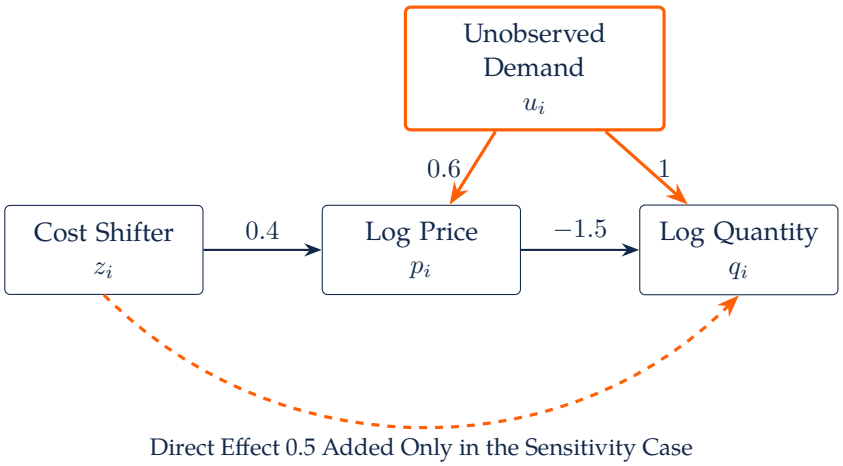


Figure 3.5. A cost shifter changes price; unobserved demand changes both price and quantity. A direct cost-to-demand path violates exclusion.

The sample contains 2,000 simulated markets with seed 534. OLS and two-stage least squares use an intercept and the same log-price regressor. The reported intervals use heteroskedasticity-robust HC1 covariance estimates and a normal approximation.

Estimator and Data Construction	Elasticity	95% Interval
OLS, Benchmark	-0.447	[-0.483, -0.410]
IV, Benchmark	-1.483	[-1.589, -1.378]
IV, Direct Cost Effect Added	-0.239	[-0.283, -0.195]

The third row adds $0.5z_i$ directly to log quantity while preserving the price rule and the structural price elasticity. Its first-stage F statistic remains 815.85, identical to the benchmark. A strong first stage therefore coexists with a severely misleading elasticity estimate when exclusion fails. For example, a proposed cost instrument that also changes product quality could affect demand through an additional path.

The PyBLP logit tutorial treats prices as endogenous in differentiated-product demand. Its market-share models require additional structure beyond this scalar simulation. The calculation below exposes the instrument logic directly. With X containing the intercept and price and Z the intercept and cost shifter, it forms $\widehat{X} = Z(Z'Z)^{-1}Z'X$ and computes $(\widehat{X}'X)^{-1}\widehat{X}'q$. Its covariance uses the structural residual $q - X\widehat{\beta}$, which differs from a second-stage residual formed with fitted price.

Python source: price_iv.py. From its directory, run `python price_iv.py`.

```
"""Synthetic price endogeneity and an exclusion-restriction sensitivity."""

from pathlib import Path
import json
import numpy as np


def fit_iv(y, x, z):
    # Instruments include the intercept. Avoid constructing an n-by-n matrix.
    xhat = z @ np.linalg.lstsq(z, x, rcond=None)[0]
    bread = np.linalg.inv(xhat.T @ x)
    beta = bread @ xhat.T @ y
    residual = y - x @ beta
    scores = xhat * residual[:, None]
    covariance = bread @ (scores.T @ scores) @ bread.T
    covariance *= len(y) / (len(y) - x.shape[1])  # HC1 correction
    se = np.sqrt(np.diag(covariance))
    return {
        "elasticity": float(beta[1]),
        "robust_se": float(se[1]),
        "ci95": [float(beta[1] - 1.96 * se[1]), float(beta[1] + 1.96 * se[1])],
    }


def calculate():
    rng = np.random.default_rng(534)
    n = 2000
    cost, demand, pricing_noise, sales_noise = rng.normal(size=(4, n))
    log_price = 2 + 0.4 * cost + 0.6 * demand + 0.2 * pricing_noise
    log_quantity = 5 - 1.5 * log_price + demand + 0.2 * sales_noise
    x = np.column_stack([np.ones(n), log_price])
    z = np.column_stack([np.ones(n), cost])
    first = z @ np.linalg.lstsq(z, log_price, rcond=None)[0]
    residual_ss = np.square(log_price - first).sum()
    total_ss = np.square(log_price - log_price.mean()).sum()
    first_stage_f = (total_ss - residual_ss) / (residual_ss / (n - 2))
```

```
    result = {
        "data": "synthetic independent markets",
        "markets": n,
        "seed": 534,
        "true_elasticity": -1.5,
        "ols": fit_iv(log_quantity, x, x),
        "valid_iv": fit_iv(log_quantity, x, z),
        "invalid_iv_direct_cost_effect_0_5": fit_iv(log_quantity + 0.5 * cost, x, z),
        "first_stage_f": float(first_stage_f),
    }
    return result


if __name__ == "__main__":
    result = calculate()
    output = Path(__file__).resolve().parent / "price_iv_results.json"
    output.write_text(json.dumps(result, indent=2, allow_nan=False) + "\n")
    print(json.dumps(result, indent=2))
```

Chapter 4

Product Preferences, Customer Segments, and Lifetime Value

A subscription retailer must choose a product bundle, decide whom to serve, and determine how much to spend acquiring or retaining customers. These decisions are connected. A feature that attracts one segment may be costly to deliver, and a segment with high initial spending may have low retention. Conjoint analysis estimates tradeoffs among product attributes, clustering summarizes patterns of similarity, and customer lifetime value translates future behavior into expected contribution.

4.1 Product Design Requires Tradeoffs

Asking consumers whether they like fast delivery often produces a favorable response, but it does not reveal how much they will pay for it or which other feature they will give up. Conjoint tasks present product profiles whose attributes vary. Respondents may rate profiles, rank them, or choose among alternatives. Choice-based tasks resemble an assortment decision, while ratings can provide more observations per respondent under a different response process.

Attributes should represent decisions the firm can implement and descriptions respondents can understand. Levels should span a plausible range without creating impossible combinations. If every premium-quality product also has the highest price, the design provides little independent information about the quality-price tradeoff. Randomization and a suitable experimental design help separate attribute effects, but realistic constraints may limit which combinations can be shown.

Include the decision's relevant alternatives. A forced choice between two products estimates preference conditional on choosing one of them. An outside option such as "buy neither" helps represent category participation. A simulation that omits the outside option can overstate category demand because every respondent must buy something. Survey design, task burden, and hypothetical behavior also affect how well stated choices transfer to market behavior.

4.2 Utility, Choice Probabilities, and Willingness to Pay

A simple model assigns respondent i a systematic utility for alternative j:

$$V_{ij} = \beta_q q_j + \beta_f f_j + \beta_p p_j,$$

where q_j indicates premium quality, f_j indicates a delivery feature, and p_j is price in dollars. Utility has an arbitrary scale. Coefficients describe tradeoffs on that scale, and price is expected to enter negatively in a conventional demand interpretation. Alternative-specific constants can capture average preference for named options relative to a reference alternative.

Under a multinomial-logit random-utility specification, the probability of choosing j from the available set $\mathcal{J}_i$ is

$$P_{ij} = \frac{\exp(V_{ij})}{\sum_{k \in \mathcal{J}_i} \exp(V_{ik})}.$$

Multinomial logit imposes proportional substitution across alternatives. Whether this pattern fits the product category requires empirical assessment. Repeated choices by one respondent are dependent; estimation and validation should respect that respondent structure. Heterogeneous preferences may be represented with interactions, latent classes, or random coefficients when the data support the additional complexity.

For a binary feature whose coefficient is β_f, a local willingness-to-pay tradeoff under linear price utility is $-\beta_f/\beta_p$. If $\beta_f = 1.2$ and $\beta_p = -0.1$ per dollar, the model equates the feature with \$12 of price. This ratio is conditional on the specification and utility scale cancellation. A price coefficient close to zero makes the ratio unstable, and preferences outside the experimental price range are extrapolations.

Attribute importance is often calculated from the range of an attribute's estimated part-worths relative to the sum of ranges. The result depends on the levels included in the study. Expanding the price range can increase measured price importance even when underlying preferences are unchanged. Report the design range when discussing importance.

4.3 Case Study: A New Bundle Can Cannibalize Existing Sales

Consider two subscription bundles and an outside option. Bundle A has premium quality and costs \$10; bundle B has standard quality and costs \$6. Let $\beta_q = 1.2$, $\beta_p = -0.1$, and normalize outside-option utility to zero. The systematic utilities are 0.2 for A and -0.6 for B. The logit probabilities are approximately 44.1% for A, 19.8% for B, and 36.1% for the outside option.

Now introduce bundle C with the same modeled attributes as A. The logit model assigns A and C approximately 30.6% each, B 13.7%, and the outside option 25.1%. C's share is 30.6 percentage points, but the total purchasing share rises only from 63.9% to 74.9%, an increase of about 11.0 points. The remaining share comes from the existing products. Forecasting a new product's sales without accounting for substitution would overstate incremental demand.

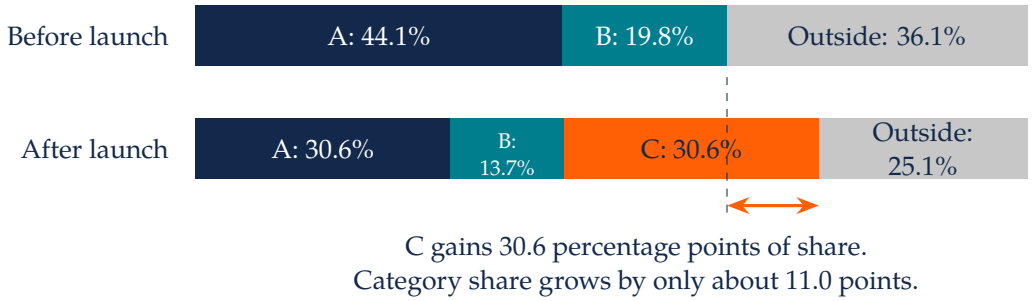


Figure 4.1. A new product's sales include substitution from existing products.

With utility coefficients and the potential-customer population fixed, adding C changes only the available choice set. Because C has the same modeled attributes as A, it is a close substitute. The logit model's independence-of-irrelevant-alternatives property makes C draw share proportionally from all existing alternatives. Whether that pattern is plausible is an empirical question. Richer substitution models may be needed when close substitutes compete especially strongly with one another.

A profit calculation must attach contribution margins and fixed costs to the simulated quantities. If A and C each contribute $4 per purchase and B contributes $3, expected contribution per potential customer rises from about $2.36 to $2.86 before product-development and operating costs. The contribution comparison is conditional on the specified utilities, margins, and fixed customer population. A pilot or holdout choice tasks can help evaluate the proposed product and model before a larger decision.

4.4 Estimate and Validate Preferences

A conjoint workflow begins with the managerial choice and feasible attributes, then constructs the task design and recruits a relevant respondent sample. Inspect completion time, straight-line responses, inconsistent attention, and whether participants understood the attributes. Exclusion rules should be set and reported transparently. Removing respondents because their preferences are inconvenient changes the target population.

Estimate a transparent baseline before adopting a more elaborate model. Check coefficient coding, reference levels, price units, and whether probabilities sum to one for every choice set. Evaluate prediction on held-out tasks or respondents according to the intended use. A model that fits familiar respondents' repeated choices may generalize differently to a new customer population.

A useful simulation changes one decision at a time and records what is fixed: attributes, price, competitor options, market size, and margins. Share of preference from a survey is not automatically market share. Distribution, awareness, availability, purchase frequency, and sampling can create differences. State the link between the conjoint output and the demand quantity used in the business calculation.

4.5 Segmentation Begins With a Definition of Similarity

A retailer may want segments for differentiated service, product positioning, or campaign design. Those objectives should guide the variables used to define similarity. Grouping customers by recent spending answers a different question from grouping them by category preferences or service needs. A segment should be interpretable, sufficiently stable, and connected to an action the firm can take.

Recency, frequency, and monetary value provide a useful descriptive starting point. Define recency as time since the last eligible purchase at a common observation date, frequency as the number of eligible purchases during a fixed window, and monetary value as sales or contribution during that window. Customers with different observation histories require adjustment or a deliberate eligibility rule. A new customer observed for one month should not be compared mechanically with one observed for a year.

K-means selects K centers to minimize within-cluster squared Euclidean distance:

$$\sum_{i=1}^{n} \left\| z_i - \mu_{c(i)} \right\|^2 .$$

Here z_i is the vector of transformed customer features, $c(i)$ is customer i's assigned cluster, and $\mu_{c(i)}$ is its cluster center. The feature transformations determine the distance. Standardization often helps prevent a large numerical scale from dominating, but it does not establish that all features should have equal managerial importance.

For three customers with spending-frequency pairs A=(100,1), B=(110,9), and C=(200,1), raw Euclidean distance makes B much closer to A than C is. If spending is expressed in hundreds of dollars while frequency remains in purchases, C becomes closer to A. The customer records have not changed; the implicit weights have. This is why units and scaling belong in the substantive definition of a segment.

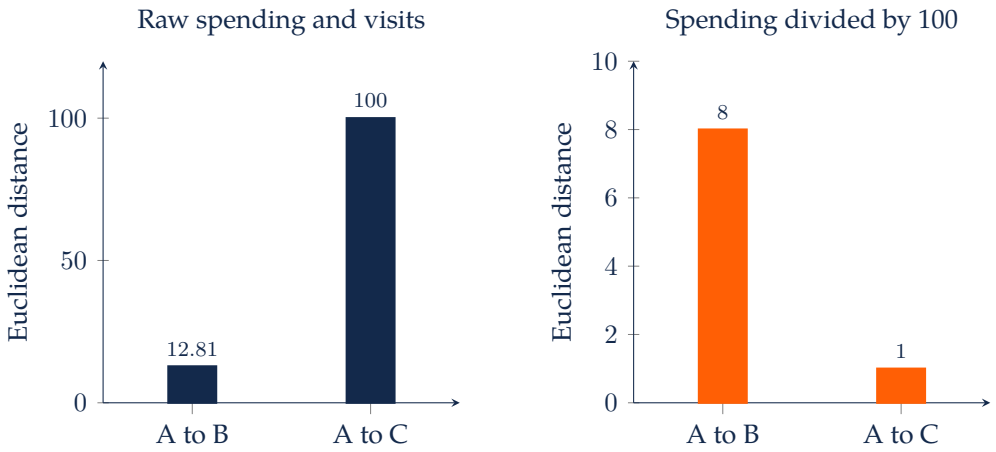


Figure 4.2. Changing feature scales changes which customers appear similar.

Changing spending units reverses the nearest neighbor because it changes the relative weight placed on spending and visits. A segmentation report should explain that metric and examine whether the proposed actions remain sensible under reasonable transformations.

4.6 Choosing and Evaluating Segments

Use more than one criterion to choose K. Within-cluster dispersion falls mechanically as more clusters are added. A silhouette measure compares cohesion with separation under the chosen distance, but a high silhouette does not establish business value. Examine segment sizes, profiles, stability across resamples or time, and whether the segments imply different feasible actions.

Hierarchical clustering provides a nested set of groupings whose interpretation depends on the linkage and distance. Density-based methods can identify irregular groups and mark some observations as noise, with sensitivity to neighborhood parameters. Mixture models provide probabilistic membership under distributional assumptions. These alternatives address different patterns;

no algorithm discovers a uniquely correct segmentation independent of representation and purpose.

For customer activation, avoid using future purchase outcomes to define a segment that supposedly exists before the campaign. For evaluating segmentation, examine behavior not used mechanically to construct it. A high-spending segment will have high spending by design if spending is a clustering input. A more useful question is whether the grouping helps predict future needs or improves the result of a tested service policy.

Clustering shopping trips on basket size, category composition, and spending can help separate trip types before comparing channels. Fit the representation without allowing the desired channel comparison to determine the grouping. Examine profiles and sensitivity, because a trip-classification algorithm can also create misleading comparisons if its features or thresholds differ across channels.

4.7 Customer Lifetime Value Depends on Timing and Margin

Customer lifetime value is the discounted expected contribution from a relationship, under a stated time horizon and behavioral model. For a newly acquired customer, a simple finite-horizon expression is

$$\text{CLV} = \sum_{t=1}^{T} \frac{\Pr(\text{active at } t)\, m_t}{(1+d)^t} - a.$$

Here t indexes periods after acquisition, m_t is expected contribution conditional on being active in period t, d is the discount rate per period, and a is acquisition cost paid at time zero. Contribution should deduct relevant variable service and fulfillment costs. The expression assumes the conditional margin and activity probability have been defined consistently; more general models can use the expected product of activity and customer-specific margin.

Assume a stationary subscription process: a currently active customer survives each period with probability r, earns a constant contribution m at the end of a period only if still active, and has an infinite horizon. With $r/(1+d) < 1$, the geometric series gives

$$\text{CLV} = \frac{mr}{1+d-r} - a.$$

This convention excludes an immediate current-period contribution. Including one would add that contribution and change the formula. Monthly retention requires monthly margins and a monthly discount rate. Combining monthly retention with an annual discount rate produces an inconsistent calculation.

With monthly contribution $m = 60$, retention $r = 0.80$, monthly discount rate $d = 0.01$, and acquisition cost $a = 50$, CLV is approximately \$178.57. Increasing retention to 0.90 raises it to approximately \$440.91. The gain is nonlinear because retention affects many future payments. These values are conditional on constant retention and margin; changes in retention or margin over time would change the valuation.

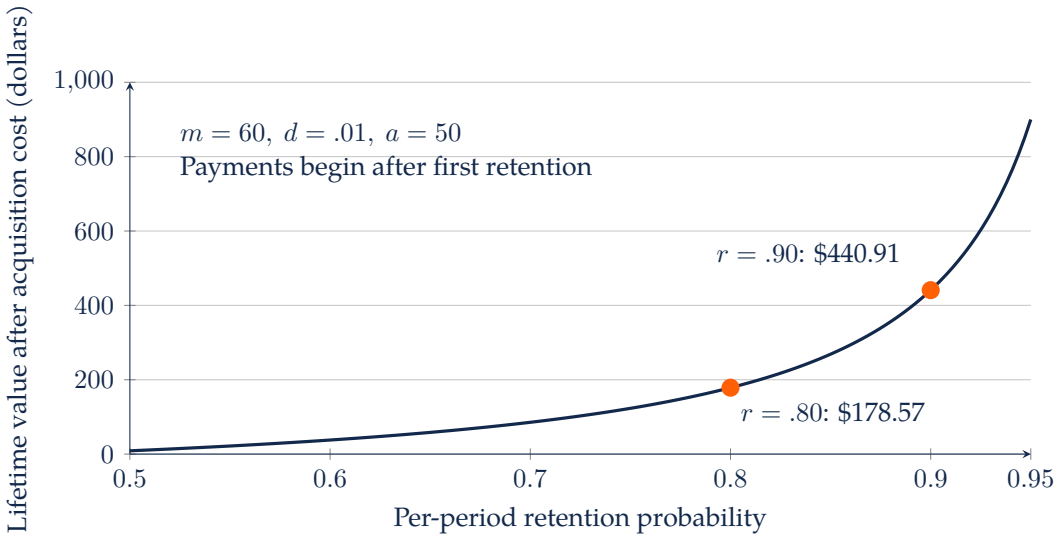


Figure 4.3. The value of retention is nonlinear under a stationary subscription model.

With monthly contribution, discounting, and acquisition cost fixed, a one-point retention increase has greater value at a higher baseline retention rate. Uncertainty about long-run retention can therefore dominate a CLV forecast.

4.8 Subscription and Noncontractual Relationships

In a subscription setting, cancellation is often observed. Survival analysis can model time to cancellation and account for right censoring when observation ends before cancellation. A hazard describes the probability of exit during a period conditional on survival to that period. Cohort effects, tenure dependence, and changes in service quality can make constant retention implausible.

In a noncontractual retail setting, the last purchase does not reveal whether a customer has permanently left. Models such as BG/NBD represent purchase frequency and latent dropout under specific stochastic assumptions. A separate spending model may estimate value per transaction. Those assumptions require checking, including the treatment of returns and any assumed relationship between purchase frequency and monetary value. A customer-level count alone does not identify permanent churn without a model.

Evaluate value forecasts on a holdout period with the same horizon used in the decision. Compare aggregate and customer-level calibration, rank ordering, and errors for new or infrequent customers. A model can rank customers well while overpredicting total value. For an acquisition budget, the aggregate monetary error may be more consequential than a small improvement in ranking.

4.9 Integrated Case: Which Segment Should Receive Retention Support?

Suppose segment H has high predicted CLV but little response to service outreach, while segment M has moderate predicted CLV and a larger response. Targeting only H can be inferior even if its value forecast is accurate. The relevant quantity is the change in expected future contribution caused by outreach, net of the outreach cost. CLV describes a baseline relationship; a retention intervention changes that relationship under a separate causal model.

A useful analysis would estimate or specify segment-specific preferences, margins, and retention, then test the proposed offer. Product design might attract a segment with expensive service needs; a seemingly attractive acquisition price might therefore have low contribution. Segment labels should help organize the decision without replacing these calculations. A final recommendation should show which assumption changes the preferred segment or product.

For practice, recompute the subscription value with $r = 0.85$ and verify the result before acquisition cost is subtracted. Add an immediate \$60 contribution and explain the change in timing. In the conjoint example, remove the outside option and observe that simulated purchase incidence becomes one by construction. In the clustering example, compare similarity using category shares instead of spending levels and explain which managerial question the new representation serves.

4.10 AI-Assisted Implementation and Readings

Ask a coding assistant to calculate choice probabilities from a supplied profile table, check that they sum to one, and separate a new product's share from incremental category demand. For clustering, require documented transformations, segment profiles, and a sensitivity comparison. For CLV, request an explicit payment timeline and verify the closed-form calculation against a long finite sum. These checks catch common mistakes that can survive apparently reasonable charts.

Sawtooth's academic materials provide conjoint examples. Dolnicar, Grün, and Leisch's segmentation book develops the connection between segmentation design and use. PyMC-Marketing's CLV documentation offers implementation examples whose model assumptions and software versions should be checked before adaptation.

4.11 mHealth: Behavioral Segments and Product Economics

A product manager may want different interfaces for occasional loggers, frequent loggers with narrow recorded diets, and frequent loggers with broad recorded diets. Segmenting on January through June behavior creates groups whose later logging can be described without using later observations to define the groups. A segment is useful when it corresponds to a decision the manager can implement, such as simplifying entry or offering a broader set of suggestions.

Eligibility requires at least 30 observed days before July 1 and nonmissing average variety. Each user contributes three features: observed days divided by the 182 calendar days in the first half of 2016, mean recorded category count, and mean calorie-share variety. Standardization gives each

feature unit variance before three-cluster k-means. Three clusters are imposed for a manageable descriptive comparison. Cluster labels are ordered by their mean prior logging fraction.

Segment	Users	Prior Logging Fraction	Mean Breadth	Later Logging Fraction
1	126	0.426	10.84	0.338
2	90	0.759	6.99	0.594
3	115	0.870	12.80	0.654

Later logging uses the 184 calendar days from July through December, with zero observed days for eligible users absent from that period. The differences describe persistence in recorded use. They are not treatment responses, contract renewal rates, or purchase probabilities. In particular, Segment 2 combines frequent logging with relatively narrow breadth. A single ranking by activity would conceal that distinction.

Repeating k-means with a second seed produces an adjusted Rand index of 1.0 for these inputs. The partition is stable to this initialization check. That result does not establish three natural customer types or prove stability to other features, calendar windows, or measurement screens. The calorie-based variety and breadth features are related, so including both places additional emphasis on recorded dietary diversity. Nutrition screening can also alter the segmentation inputs; compare its consequences before naming a segment around a health interpretation.

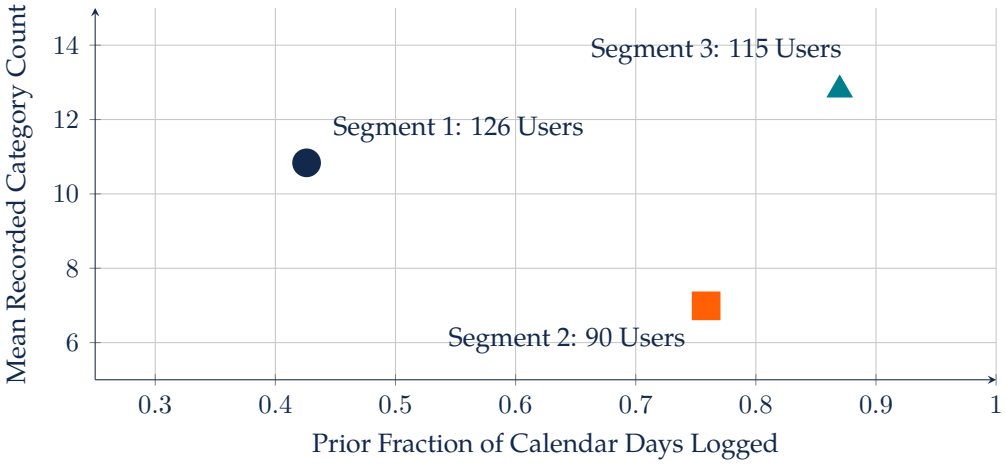


Figure 4.4. Prior logging frequency and recorded breadth distinguish groups that a single activity ranking would combine. Both axes use January through June; later behavior does not define the partition.

The food logs supply no observed subscription price, product-choice experiment, margin, acquisition cost, or payment-based retention. A hypothetical paid-plan calculation can nevertheless clarify the extra inputs needed. Set the outside-option utility to zero, paid-plan benefit to 0.8, price coefficient to -0.1 per dollar, and price to \$5. The imposed logit model yields a paid-plan share of 0.5744. These inputs are assumptions, not conjoint estimates from the logs.

For a separate hypothetical subscription, suppose monthly contribution conditional on retention is \$6, the monthly discount rate is 1%, acquisition cost is \$10, and the first contribution occurs one month after acquisition. With constant monthly retention r, value is $6r/(1.01 - r) - 10$. Assumed retention of 0.80, 0.85, and 0.90 gives values of \$12.86, \$21.87, and \$39.09. These scenarios are not

assigned to the three behavioral segments. Doing so would equate logging persistence with paid retention without evidence.

The code fits the scaler and clusters using only the earlier period, then joins later logging for description. An AI assistant should explain each feature's units, how missing later records enter the denominator, and which economic inputs remain hypothetical before proposing a targeting rule.

Python source: m04.py. From its directory, run `python m04.py`.

```
"""Segment January-June behavior and describe later logging."""

import json
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import adjusted_rand_score
from common import OUTPUT, SEED, load_inputs, save_result

food, daily, baskets = load_inputs()
prior = daily.loc[daily.date.lt("2016-07-01")]
features = prior.groupby("Userid").agg(
    logged_days=("date", "size"),
    breadth=("category_count", "mean"),
    variety=("variety", "mean"),
)
features = features.loc[features.logged_days.ge(30)].dropna()
features["logging_fraction"] = features.logged_days / 182
columns = ["logging_fraction", "breadth", "variety"]
scaled = StandardScaler().fit_transform(features[columns])
model = KMeans(n_clusters=3, n_init=20, random_state=SEED).fit(scaled)
alternative = KMeans(n_clusters=3, n_init=20, random_state=SEED + 1).fit(scaled)
# Number clusters by their mean prior logging fraction for readable output.
order = (
    features.assign(raw=model.labels_)
    .groupby("raw")
    .logging_fraction.mean()
    .sort_values()
    .index
)
labels = {raw: rank + 1 for rank, raw in enumerate(order)}
features["segment"] = [labels[value] for value in model.labels_]
later = daily.loc[daily.date.ge("2016-07-01")].groupby("Userid").size()
features["later_logging_fraction"] = later.reindex(features.index, fill_value=0) / 184
table = features.groupby("segment").agg(
    users=("segment", "size"),
    prior_logging=("logging_fraction", "mean"),
    breadth=("breadth", "mean"),
    variety=("variety", "mean"),
    later_logging=("later_logging_fraction", "mean"),
)
```

```
OUTPUT.mkdir(exist_ok=True)
table.to_csv(OUTPUT / "M04_segments.csv")
# Imposed monthly financial assumptions, separate from the log estimates.
retention = np.array([0.80, 0.85, 0.90])
clv = 6 * retention / (1.01 - retention) - 10
utility = np.array([0.0, 0.8 - 0.1 * 5])  # Outside option, hypothetical paid plan.
choice = np.exp(utility - utility.max())
choice /= choice.sum()
result = {
    "users": len(features),
    "features": columns,
    "seed_agreement_ari": float(
        adjusted_rand_score(model.labels_, alternative.labels_)
    ),
    "segments": table.reset_index().to_dict(orient="records"),
    "hypothetical_retention": retention.tolist(),
    "hypothetical_clv": clv.tolist(),
    "hypothetical_paid_plan_share": float(choice[1]),
}
save_result("M04", result)
print(json.dumps(result, indent=2))
```

4.12 Python Calculations for the Synthetic Cases

The inputs below are imposed teaching values. Running the program creates the labeled synthetic CSV files and prints the corresponding calculations. They are separate from the mHealth estimates. Keep the script beside `example_helpers.py` in `03_Data_and_Code/Lecture_Examples`.

Python source: m04_synthetic.py. From its directory, run `python m04_synthetic.py`.

```
"""Synthetic numerical examples for M4; run from any working directory."""

import itertools
import json
import math
from example_helpers import write_csv, ols_slope, shares

r = {}
before = shares({"A": 0.2, "B": -0.6, "outside": 0})
after = shares({"A": 0.2, "B": -0.6, "C": 0.2, "outside": 0})
r["M04"] = {
    "shares_before": before,
    "shares_after": after,
    "category_share_increase": before["outside"] - after["outside"],
    "contribution_before": 4 * before["A"] + 3 * before["B"],
    "contribution_after": 4 * (after["A"] + after["C"]) + 3 * after["B"],
    "wtp_dollars": 1.2 / 0.1,
    "distance_raw_AB": math.hypot(10, 8),
    "distance_raw_AC": 100,
    "distance_rescaled_AB": math.hypot(0.1, 8),
```

```
    "distance_rescaled_AC": 1,
    "clv_after_acquisition": {
        str(retention): 60 * retention / (1.01 - retention) - 50
        for retention in [0.8, 0.85, 0.9]
    },
}
write_csv(
    "M04_segmentation_synthetic.csv",
    ["customer", "spending_dollars", "visits"],
    [("A", 100, 1), ("B", 110, 9), ("C", 200, 1)],
)
assert math.isclose(sum(before.values()), 1) and math.isclose(sum(after.values()), 1)
print(json.dumps(r, indent=2))
```

4.13 mHealth: Time Until a Logging Gap

A retention manager may want to intervene before a long interruption in activity. The analytical object must specify when the interruption has actually occurred. A user who is absent for two days and returns on the third day has a different history from one who remains absent for several weeks. The end of the data window can arrive before either pattern is fully observed.

Define eligibility using at least 30 recorded days before July 1. Among eligible users who return during July, set time zero to that user's first July logging day. The event occurs on the 14th consecutive day without a subsequent log, and observation ends on August 31. The event date is the day the gap is confirmed. A later return does not erase a previously confirmed gap. Customers without a confirmed gap by August 31 are administratively censored at their elapsed follow-up time.

This definition concerns an interruption in observed logging. Permanent customer departure, failure to eat, and cancellation of a paid subscription are separate events. July return is also a selection condition: 331 users satisfy prior eligibility, and 297 have a July time origin. The curve describes these returners, including a mixture of early and late July start dates.

To allow different amounts of follow-up, define n_t as users still under observation and gap-free immediately before elapsed day t, and d_t as confirmed first gaps on that day. The Kaplan–Meier estimate is

$$\widehat{S}(t) = \prod_{s \le t} \left(1 - \frac{d_s}{n_s}\right).$$

A censored user contributes to earlier risk sets and leaves subsequent ones. Censoring itself causes no downward step. When an event and administrative censoring share a day, the event is counted before censoring. A population interpretation of the pooled curve requires censoring to be independent of gap timing, possibly conditional on relevant start-date characteristics. Calendar trends can undermine that condition because later July starters have shorter follow-up.

Confirmed Gap	Users	Events by August 31	Censored	Gap-Free at Day 30	Gap-Free at Day 60
14 Days	297	93	204	0.8182	0.6867
28 Days	297	33	264	0.9630	0.8877

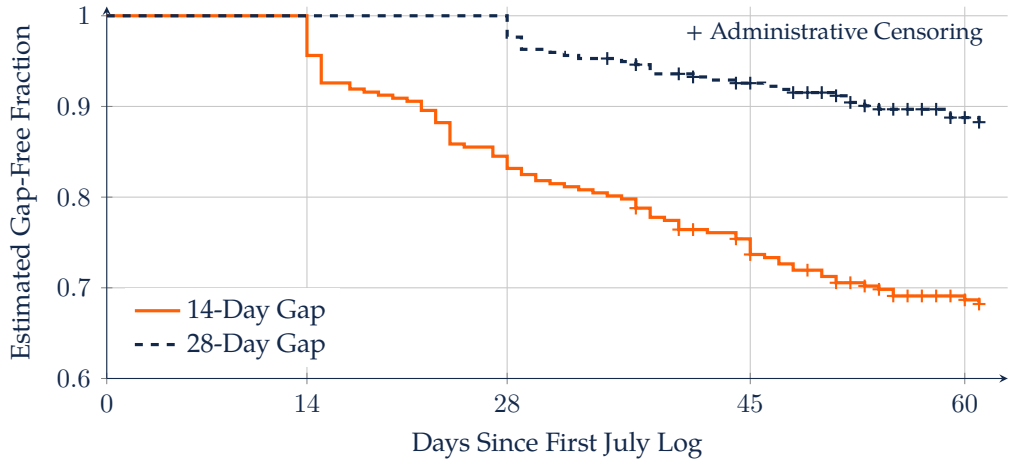


Figure 4.5. Gap-free estimates depend on the interruption threshold; users censored at the data boundary leave the risk set without becoming events.

The 28-day definition produces fewer confirmed events because an interruption must persist longer. It also confirms an event later even when the same interruption satisfies both thresholds. At elapsed day 60, the risk sets contain 160 and 187 users for the 14- and 28-day definitions, respectively. The table therefore requires an event definition and a time origin to support a retention comparison. Converting either curve into customer lifetime value would additionally require a commercial relationship, margins, discounting, and a model linking logging interruptions to that relationship.

The lifelines customer-churn example uses explicit event and duration fields. The Kaplan–Meier documentation supplies the corresponding risk-set estimator. The code below computes each risk set explicitly so a small hand-constructed event/censoring table can verify the calculation.

Python source: logging_gaps.py. From its directory, run `python logging_gaps.py`.

```
"""Kaplan-Meier estimates for a defined logging-gap event, with censoring."""

import json
import pandas as pd
from common import OUTPUT, load_food, make_daily, save_result


def first_gap(dates, start, end, gap_days):
    dates = set(dates)
    run = 0
    for day in pd.date_range(start + pd.Timedelta(days=1), end):
        run = 0 if day in dates else run + 1
        if run == gap_days:
```

```
            return (day - start).days, True
    return (end - start).days, False


def kaplan_meier(records):
    # At a tied time, events precede administrative censoring.
    survival = 1.0
    rows = []
    for time in sorted(records.duration.unique()):
        risk = int(records.duration.ge(time).sum())
        events = int((records.duration.eq(time) & records.event).sum())
        censored = int((records.duration.eq(time) & ~records.event).sum())
        survival *= 1 - events / risk
        rows.append(
            {
                "day": int(time),
                "at_risk": risk,
                "events": events,
                "censored": censored,
                "gap_free": survival,
            }
        )
    return pd.DataFrame(rows)


if __name__ == "__main__":
    daily = make_daily(load_food())
    prior = daily.loc[daily.date < "2016-07-01"].groupby("Userid").size()
    eligible = prior.index[prior >= 30]
    july = daily.loc[
        daily.Userid.isin(eligible) & daily.date.between("2016-07-01", "2016-07-31")
    ]
    starts = july.groupby("Userid").date.min()
    observed = daily.groupby("Userid").date.agg(set)
    end = pd.Timestamp("2016-08-31")
    result = {
        "eligible_before_july": len(eligible),
        "july_returners": len(starts),
        "administrative_end": str(end.date()),
        "thresholds": [],
    }
    for gap in [14, 28]:
        records = pd.DataFrame(
            [
                first_gap(observed[user], start, end, gap)
                for user, start in starts.items()
            ],
            columns=["duration", "event"],
        )
        curve = kaplan_meier(records)
        summary = {
```

```
            "gap_days": gap,
            "users": len(records),
            "events": int(records.event.sum()),
            "censored": int((~records.event).sum()),
        }
        for horizon in [30, 45, 60]:
            before = curve.loc[curve.day <= horizon]
            summary[f"gap_free_day_{horizon}"] = (
                float(before.gap_free.iloc[-1]) if len(before) else 1.0
            )
            summary[f"at_risk_day_{horizon}"] = int(records.duration.ge(horizon).sum())
        result["thresholds"].append(summary)
        OUTPUT.mkdir(exist_ok=True)
        curve.to_csv(OUTPUT / f"logging_gap_{gap}.csv", index=False)
    save_result("logging_gaps", result)
    print(json.dumps(result, indent=2))
```

4.14 Subscription Retention and Category Participation

A provider considering a retention offer needs to know whether departing subscribers continue buying from competitors. A customer who retains the category may respond to a different price, product, or bundle. A customer who stops paying for the category raises a different acquisition question. These outcomes require linked customer histories and a common starting cohort.

4.14.1 Adoption and Multihoming Denominators

The supplied synthetic panel contains a fixed roster of 60,000 consumers and 40 calendar months from January 2023 through April 2026. Subscription rows record held provider plans; the roster also includes consumers who never subscribe. Within this complete simulation, an absent consumer-provider-month row means no paid subscription. That interpretation relies on the generator's recording rule and would require separate validation in a transaction dataset.

Let D_{ijt} indicate that consumer i holds a paid subscription to provider j in month t. To measure category adoption and multihoming among payers, calculate

$$A_t = \frac{\sum_{i=1}^{N} \mathbf{1}\{\sum_j D_{ijt} > 0\}}{N}, \qquad H_t = \frac{\sum_{i=1}^{N} \mathbf{1}\{\sum_j D_{ijt} > 1\}}{\sum_{i=1}^{N} \mathbf{1}\{\sum_j D_{ijt} > 0\}}.$$

Here N is the full consumer roster. Adoption uses all consumers, while multihoming uses the consumers with at least one subscription. In April 2026, the simulation has 1,741 payers holding 1,813 subscriptions, and 71 payers hold multiple providers. Adoption is therefore 2.902%, and multihoming among payers is 4.078%. The extra subscriptions and the number of multihomers differ because a consumer can hold more than two providers.

The generator adjusts a common utility term each month to approach a chosen aggregate adoption path. The final adoption rate is consequently a calibration outcome. It supplies no independent estimate of actual market penetration.

4.14.2 Following the Same Starting Customers

Define C_j as the set of consumers paying provider j in January 2025. At a later month, partition this fixed set into consumers paying j, consumers paying only other providers, and consumers paying no provider. Consumers who hold j and a rival belong in the first group. These three groups exhaust the original cohort, so their shares sum to one.

The table follows each January 2025 provider cohort to January 2026. All values are synthetic. Provider cohorts overlap when a starting customer holds multiple subscriptions; their denominators should not be added as a unique-consumer count.

January 2025 Provider	Starting Customers	Same Provider After 12 Months	Other Providers Only	No Paid Provider
Anthropic	317	32.8%	26.5%	40.7%
Google	157	18.5%	38.2%	43.3%
Microsoft	84	1.2%	53.6%	45.2%
OpenAI	430	37.7%	20.9%	41.4%
Perplexity	63	3.2%	58.7%	38.1%

For the 430 starting OpenAI subscribers, 162 pay OpenAI after twelve months and another 90 pay other providers exclusively. Category participation within this same cohort is $252/430 = 58.6\%$. The 20.9-percentage-point difference isolates continued category participation among customers who no longer hold the original provider. Comparing a provider cohort with the pooled cohort of all initial AI payers would also change the mix of customers in the denominator.

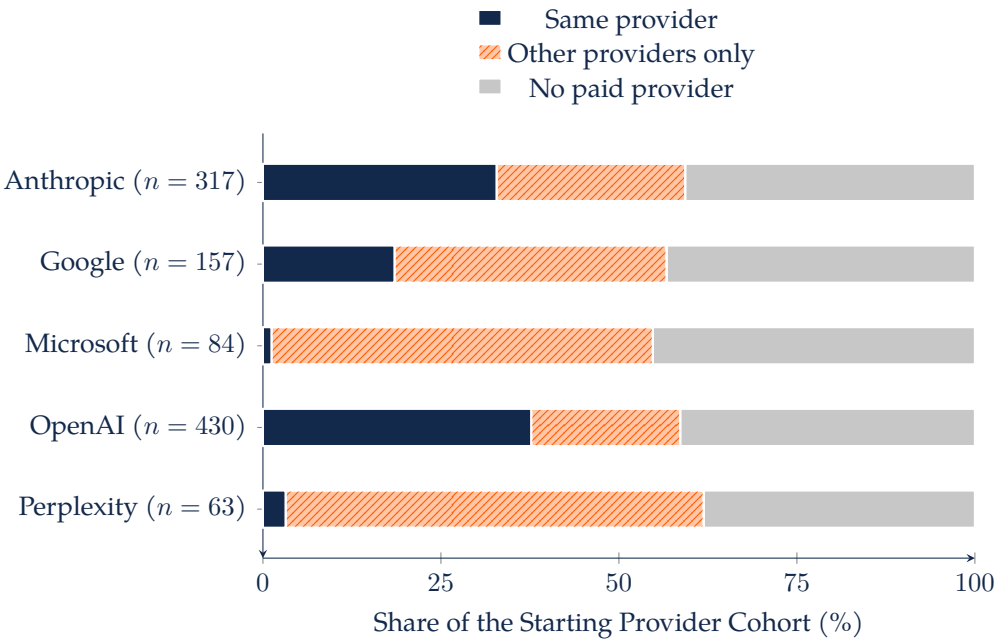


Figure 4.6. Synthetic January 2025 provider cohorts partitioned by January 2026 payment status. Each horizontal bar uses its own fixed starting cohort and sums to 100 percent: same provider, other providers only, and no paid provider. The five provider cohorts can overlap.

4.14.3 Endpoint Retention, Continuous Retention, and Acquisition

Endpoint retention permits cancellation followed by reentry. Continuous retention requires the same provider to be held in every intervening month. Of the 430 starting OpenAI subscribers, 129 remain continuously subscribed through January 2026, a 30.0% continuous-retention rate. The 162 endpoint subscribers include 33 who returned after at least one observed gap. A cohort of existing subscribers also differs from a cohort of newly acquired customers because subscription tenure and prior survival affect its composition. Applying an acquisition-cohort CLV formula to this twelve-month endpoint rate would obscure both distinctions.

Likewise, the supplied `new` flag records a provider absent from the previous month's holdings. It can describe first paid entry, reentry to the category, a complete replacement of previous providers, or an additional provider while retaining at least one incumbent. In April 2026, the 401 new-provider events comprise 197 first paid entries, 157 category returns, 32 complete provider switches, and 15 provider additions. There are 400 distinct consumers behind these events because one consumer starts two providers. The event denominator is therefore explicit.

A release-month increase in new-provider events can reflect several of these mechanisms. A causal claim about a release also requires a credible comparison for what would have happened without it. Timing, publicity, contemporaneous prices, overlapping releases, and previous subscription states affect that comparison. The descriptive partition supports the acquisition decision without treating every new-provider event as switching.

Python source: retention.py. From the script's folder, run `python3 retention.py`. Inputs are read from the supplied local bundle; output tables are written beside the course scripts.

```
"""Measure synthetic subscription stocks, cohorts, and acquisition events."""
from pathlib import Path
import json

import numpy as np
import pandas as pd

HERE = Path(__file__).resolve().parent
SOURCE = HERE.parents[1] / "01_Resources/08_M8_Current_Topics_in_Marketing/genai_demand_lab/data"
OUTPUT = HERE / "results"


def acquisition_type(previous, current, ever_paid):
    """Classify a newly held provider using adjacent monthly subscription sets."""
    if not previous:
        return "category_return" if ever_paid else "first_paid_entry"
    if previous.isdisjoint(current):
        return "complete_provider_switch"
    return "provider_addition"


def cohort_outcomes(subs, provider, start, horizon, observed_months):
    """Endpoint and continuous retention for one fixed provider cohort."""
    required = {str(pd.Period(start, freq="M") + k) for k in range(horizon + 1)}
    if not required.issubset(set(observed_months)):
```

```
        raise ValueError("Cohort horizon extends beyond the observed calendar")
    cohort = set(subs.loc[subs.month.eq(start) & subs.provider.eq(provider), "consumer_id"])
    if not cohort:
        raise ValueError("Empty starting cohort")
    continuous = cohort.copy()
    rows = []
    for k in range(horizon + 1):
        month = str(pd.Period(start, freq="M") + k)
        active = subs.loc[subs.month.eq(month)]
        own = set(active.loc[active.provider.eq(provider), "consumer_id"])
        any_provider = set(active.consumer_id)
        continuous &= own
        row = {"provider": provider, "start_month": start, "month": month,
               "horizon_months": k, "cohort_size": len(cohort),
               "same_provider": len(cohort & own),
               "other_provider_only": len((cohort & any_provider) - own),
               "no_paid_provider": len(cohort - any_provider),
               "continuous_same_provider": len(continuous)}
        assert row["same_provider"] + row["other_provider_only"] + row["no_paid_provider"] == len(cohort)
        rows.append(row)
    return pd.DataFrame(rows)


def main():
    consumers = pd.read_csv(SOURCE / "synthetic_consumers.csv")
    subs = pd.read_csv(SOURCE / "synthetic_subscriptions.csv.gz")
    dgp = json.loads((SOURCE / "synthetic_dgp.json").read_text())
    months = sorted(dgp["g_trend"])
    n = len(consumers)
    assert n == dgp["n_consumers"] and consumers.consumer_id.is_unique
    assert not subs.duplicated(["consumer_id", "month", "provider"]).any()
    assert subs.consumer_id.isin(consumers.consumer_id).all()
    assert subs.month.isin(months).all() and subs.price.gt(0).all()
    assert subs["new"].isin([True, False]).all()
    assert subs.tier.isin(["basic", "premium"]).all()
    states = {m: {int(i): set(g.provider) for i, g in s.groupby("consumer_id")}
              for m, s in subs.groupby("month")}
    events, monthly, history = [], [], set()
    previous = {}
    for month in months:
        current = states.get(month, {})
        for consumer, providers in current.items():
            old = previous.get(consumer, set())
            for provider in sorted(providers - old):
                events.append({"month": month, "consumer_id": consumer, "provider": provider,
                               "event": acquisition_type(old, providers, consumer in history)})
        group = subs.loc[subs.month.eq(month)]
        monthly.append({"month": month, "population": n, "payers": len(current),
                        "subscriptions": len(group), "adoption": len(current) / n,
                        "multihomers": sum(len(s) > 1 for s in current.values()),
                        "basic_subscriptions": int(group.tier.eq("basic").sum())})
```

```
        history.update(current)
        previous = current
    events = pd.DataFrame(events)
    keys = ["month", "consumer_id", "provider"]
    recorded = set(subs.loc[subs["new"], keys].itertuples(index=False, name=None))
    calculated = set(events[keys].itertuples(index=False, name=None))
    assert recorded == calculated
    monthly = pd.DataFrame(monthly)
    monthly["multihoming_given_payment"] = monthly.multihomers / monthly.payers.replace(0, np.nan)
    monthly["basic_subscription_share"] = monthly.basic_subscriptions / monthly.subscriptions.replace(0,
    ↪  np.nan)
    cohort_start, horizon = "2025-01", 12
    assert str(pd.Period(cohort_start, freq="M") + horizon) <= months[-1]
    providers = sorted(subs.loc[subs.month.eq(cohort_start), "provider"].unique())
    cohorts = pd.concat([cohort_outcomes(subs, p, cohort_start, horizon, months)
                         for p in providers], ignore_index=True)
    last_events = events.loc[events.month.eq(months[-1])]
    counts = last_events.event.value_counts().sort_index()
    last = monthly.iloc[-1]
    result = {"data_type": "synthetic", "population": n, "subscription_rows": len(subs),
              "calendar_months": len(months), "last_month": months[-1],
              "last_payers": int(last.payers), "last_subscriptions": int(last.subscriptions),
              "last_adoption": float(last.adoption),
              "last_multihomers": int(last.multihomers),
              "last_multihoming_given_payment": float(last.multihoming_given_payment),
              "last_acquisition_events": int(len(last_events)),
              "last_acquisition_consumers": int(last_events.consumer_id.nunique()),
              "last_acquisition_types": {k: int(v) for k, v in counts.items()},
              "new_flag_verified": True}
    OUTPUT.mkdir(exist_ok=True)
    monthly.to_csv(OUTPUT / "adoption.csv", index=False, float_format="%.8f")
    cohorts.to_csv(OUTPUT / "cohorts.csv", index=False)
    counts.rename_axis("event").rename("events").to_csv(OUTPUT / "acquisition_types.csv")
    (OUTPUT / "retention.json").write_text(json.dumps(result, indent=2) + "\n")
    print(json.dumps(result, indent=2))


if __name__ == "__main__":
    main()
```

Chapter 5

Text, Social Media, and Customer Networks

A retailer receives thousands of reviews and support messages each week. The service manager needs to distinguish delivery delays, product defects, and billing problems, then decide which issues deserve attention. A count of negative words is an incomplete representation: "the refund was fast, but the replacement failed" contains different evaluations of two aspects of the experience. The analytical task is to turn language into a defensible measurement of a concept relevant to action.

Text analysis also raises a sampling problem. Customers who post reviews can differ from those who remain silent, and the platform can affect what gets posted or displayed. Social-network analysis adds another layer: observed connections and similar behavior can reflect influence, common circumstances, or selection into relationships.

5.1 Define the Construct and the Document

A construct is the concept the analysis intends to measure, such as delivery dissatisfaction, perceived authenticity, or purchase intent. A label is the operational category assigned to an observation. A document might be a complete review, one sentence, a support ticket, or a conversation. Those units should follow the construct. A review-level sentiment label can obscure a positive service experience and a negative product experience in the same review.

Write an annotation codebook before producing labels at scale. It should define each category, inclusion and exclusion rules, treatment of ambiguity, and examples near category boundaries. If a message can contain several problems, a multi-label task may be appropriate. If the decision requires routing each ticket to one team, define a primary issue or a routing priority separately from the presence of every issue.

Metadata can be informative and potentially misleading. Ratings may help validate a sentiment measure but can also leak the intended label into a prediction exercise. Product category, language, channel, and date can reveal domain differences. Remove or protect personal information before sending text to an external service, and use approved data access. A scraped review is not automatically free of personal or contractual restrictions.

5.2 Representing Language With Counts and Weights

A bag-of-words representation counts tokens while largely ignoring their order. An n-gram representation retains local sequences, so "not good" can differ from "good." Tokenization, case normalization, punctuation, and stop-word rules change the resulting features. Removing negation words can destroy information needed for sentiment. Removing product names may help focus on attributes or may erase the very object whose complaints the manager needs to identify.

Term frequency-inverse document frequency gives greater weight to terms that are frequent in a document but less common across the corpus. One simple convention is

$$\text{tfidf}_{wd} = \text{tf}_{wd} \log\left(\frac{N}{\text{df}_w}\right),$$

where tf_{wd} is the frequency of word w in document d, N is the number of documents, and df_w is the number of documents containing the word. Implementations vary in smoothing, normalization, and term-frequency definitions. Record the convention rather than assuming a library's output matches this exact expression.

If "delivery" appears in two of three documents, its inverse-document-frequency term under this convention is $\log(3/2) \approx 0.405$. A word appearing in every document receives zero weight. That weighting can help discriminate documents, but a common term may still be substantively important. Statistical distinctiveness and managerial importance are different criteria.

Fit a learned vocabulary and IDF weights on training text when evaluating prediction. Learning them from the test corpus exposes information about the evaluation distribution. The leakage may appear minor compared with using labels, yet it still changes the task. A pipeline makes the training-only transformation explicit and reproducible.

5.3 Embeddings and Semantic Similarity

An embedding maps a document or token into a vector learned from language data. Similar vectors can capture related meanings even when documents use different words. Cosine similarity compares the angle between vectors:

$$\cos(u, v) = \frac{u^\top v}{\|u\| \, \|v\|}.$$

The normalization removes vector length from the comparison. This is useful when direction represents meaning in the selected model, but it does not guarantee that the similarity corresponds to the marketing construct. Two messages about refunds can be semantically close while expressing opposite evaluations. Retrieval quality should be checked with examples that matter for the downstream task.

A simple TF-IDF classifier can be a strong baseline for a well-defined domain. Embeddings may help with paraphrases, sparse labels, and semantic retrieval, but they introduce dependence on a pretrained model and its training domain. Compare alternatives on the same held-out data and operational criteria. Record model versions because an updated representation can change nearest neighbors and labels even when the input text is unchanged.

For interpretation, inspect representative successes and failures. A model may associate a brand name with complaints because one product had a recall during training. It can then misclassify neutral mentions after the issue is resolved. Temporal validation and product-level holdouts help reveal whether performance depends on such shortcuts.

5.4 Sentiment, Aspect Labels, and Topic Models

A sentiment lexicon assigns scores to words and combines them using rules. It offers a transparent baseline, but sarcasm, domain-specific meanings, and context can cause errors. A supervised classifier learns a mapping from labeled examples to outcomes. Its labels and training sample define what it learns; a generic "positive/negative" dataset may not represent customer-service severity.

Aspect-based analysis separates the object being evaluated from the evaluation itself. For the sentence "delivery was late, but the support agent solved the problem," a useful record might contain delivery-negative and service-positive labels. A single overall score discards that distinction. If the action is a logistics intervention, the aspect-specific measure is closer to the decision.

Topic models describe recurring patterns of word co-occurrence or document similarity. A topic is an estimated pattern, not automatically a stable consumer need. In probabilistic models, documents can combine topics; in clustering approaches, assignments may be discrete or probabilistic. Analysts supply topic names after inspecting high-weight words and representative documents. Validate those names with actual text and examine stability across preprocessing and model choices.

A manager should avoid interpreting an increase in a topic's share without checking its denominator. More delivery complaints can mean more delivery problems, more deliveries overall, a changed review prompt, or fewer reviews about other topics. Combining text measures with transaction or service-volume data helps distinguish these explanations.

5.5 Case Study: A Complaint Detector Under Changing Prevalence

Suppose a classifier identifies a rare but costly product defect. In a synthetic evaluation population of 1,000 tickets, 200 truly mention the defect. The classifier detects 160 of them and incorrectly flags 80 of the 800 other tickets. Sensitivity, or recall, is 80%; specificity is 90%; precision is 160/240, or 66.7%.

Population	True Defect Tickets	True Positives	False Positives	Precision
Evaluation sample	200	160	80	66.7%
Deployment sample	10	8	99	7.5%

The second row holds sensitivity at 80% and specificity at 90%, but reduces true prevalence to 1%. Among 990 nondefect tickets, 99 are flagged. Only eight of the 107 flagged tickets are true positives. A manager using the original precision to plan review staffing would be badly misled.

Bayes' rule makes the dependence explicit. If prevalence is ρ, sensitivity is s, and specificity is q, then

$$\Pr(\text{defect} \mid \text{flag}) = \frac{s\rho}{s\rho + (1-q)(1-\rho)}.$$

The denominator counts both correct and incorrect flags. For a rare issue, even a modest false-positive rate can dominate that total. This is why validation should represent the deployment population or explicitly adjust for a changed sampling design.

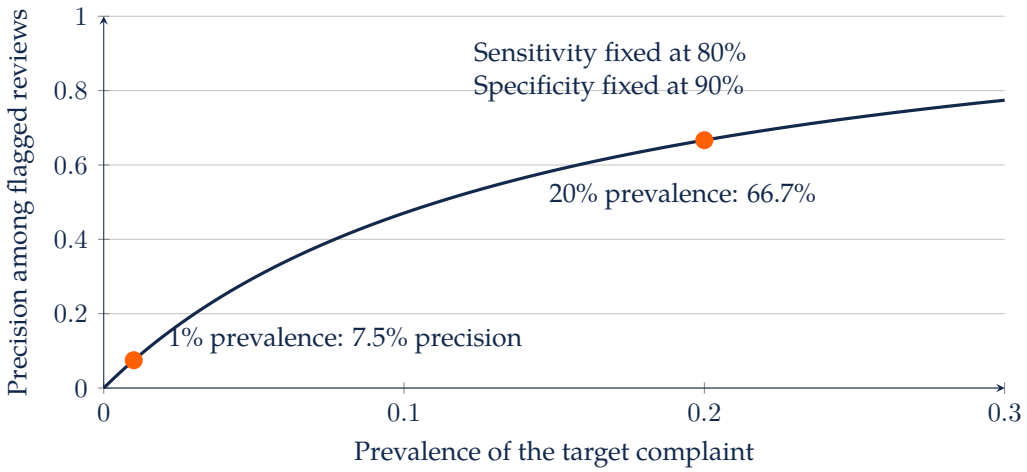


Figure 5.1. Precision falls as the target issue becomes rarer, even with unchanged sensitivity and specificity.

Sensitivity and specificity remain at 0.80 and 0.90 in both populations. The decline in precision comes entirely from the lower prevalence. Operational evaluation should also consider the cost of missed defects, the burden of false alarms, and whether uncertain cases can be routed to human review.

5.6 Human and LLM Annotation

LLMs can help draft a codebook, suggest candidate labels, or classify text using explicit instructions. For a defensible measurement exercise, preserve the original text, the codebook version, model identifier, prompt, output schema, and date. Require a bounded output format with an allowed "uncertain" category where appropriate. Validate the output against the schema and inspect cases that violate the instructions.

A prompt should identify the construct, unit, categories, boundary rules, and examples. For delivery dissatisfaction, it should distinguish a complaint about an actual delay from a question about the promised delivery date. Ask for a short supporting span from the supplied text when that improves auditability. A fluent explanation is not evidence that the label is correct; compare it with independent human judgments on a relevant sample.

Human agreement is informative because the construct may itself be ambiguous. Double-code a sample, examine disagreements, and refine definitions before scaling. Report category-specific errors, prevalence, and performance across languages or product groups. An average accuracy can conceal systematic failure on the minority category that motivates the analysis.

Treat text being classified as data. A review may contain language resembling an instruction to the model. It should not change the annotation task, request external actions, or supply new system rules. Separate the fixed task instructions from the document content, constrain outputs, and inspect unexpected behavior. Use synthetic or approved text for classroom exercises involving external services.

5.7 Networks: Connections, Reach, and Brokerage

A network represents entities as nodes and relationships as edges. Nodes might be customers, creators, brands, or products. An edge might indicate following, communication, co-purchase, or a similarity criterion. Directed and weighted networks preserve different information from undirected binary networks. The edge definition should follow the substantive relationship the analyst intends to study.

Degree counts a node's direct connections. Betweenness summarizes how often a node lies on shortest paths between other nodes. A person with few direct contacts can have high betweenness if those contacts connect otherwise separate groups. Eigenvector-based measures give more weight to connections with well-connected nodes. These metrics describe network structure under their definitions; they do not by themselves establish persuasion, credibility, or the effect of paying a creator.

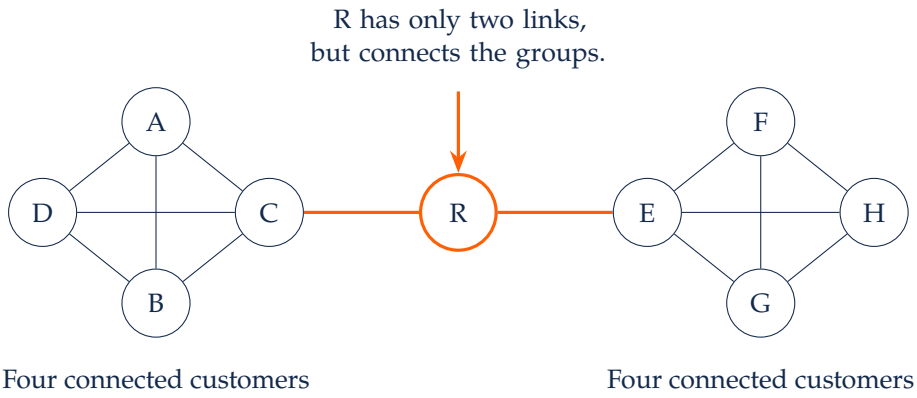


Figure 5.2. A broker with few direct links can connect two otherwise separate customer groups.

Broker R has two direct connections, while each within-group connector has four. Removing R disconnects the two groups. A customer with fewer direct contacts can therefore be essential for reaching a different audience. A marketing campaign may value either role, depending on message transmission, audience overlap, and the intended outcome.

Observed similarity among connected customers can arise from influence, homophily, or common exposure. Friends may adopt a product because they affect one another, because they already share tastes, or because they see the same promotion. Regressing adoption on friends' adoption does not distinguish those mechanisms without additional design. Randomized encouragement, timing restrictions, or credible natural variation can help, with attention to spillovers and the unit of assignment.

5.8 Social-Media Case: Engagement and Commercial Value

A brand compares two creators. One receives many comments, but those comments largely come from an audience that already follows the brand. The other reaches a smaller group with less overlap. A high engagement count does not establish more incremental sales. The comparison

requires a target population, exposure definition, outcome horizon, and a way to measure what would have happened without the collaboration.

Text can help describe comment content, and network overlap can describe audience duplication. Those measurements support campaign design, but they should be combined with an evaluation strategy. A test might randomize eligible regions or audience groups when feasible, or compare outcomes using a credible observational design. If communication crosses groups, spillovers can complicate the interpretation of an untreated group.

For classroom practice, construct two small audience sets and calculate their overlap. Then compare comments per exposed user with total comments. Explain why neither is a direct estimate of incremental profit.

5.9 AI-Assisted Workflow, Practice, and Readings

Start a text exercise with a small manually inspected sample and a codebook. Build a transparent baseline, reserve an evaluation sample, and compare a more complex method on the same outcomes. Ask an assistant to produce a confusion matrix and representative errors, then independently recompute precision and recall from the counts. Keep model selection and prompt refinement away from the final test sample.

For practice, calculate accuracy in the first complaint-detector sample: 88%. Explain why a classifier that always predicts "no defect" would achieve 99% accuracy in the rare-prevalence deployment sample while detecting no defects. Next identify how removing "not" changes the meaning of "not reliable." Finally, explain why two highly connected adopters may share preferences even if neither caused the other's adoption.

The scikit-learn text-feature documentation supports count and TF-IDF representations. VADER's author repository provides a rule-based sentiment baseline. Hagen and colleagues (2020) discuss ways machine learning can support behavioral marketing research.

5.10 mHealth: Classifying New Food Descriptions

The manager wants to reduce the effort required to categorize a food entry. A lookup table may work well for familiar descriptions, while a text classifier may help with a new description. These are distinct deployment conditions. Repeated descriptions in both training and test records can make a random entry split look successful even when the system will struggle with unseen language.

Normalize descriptions by lowercasing, collapsing whitespace, and trimming the ends. Retain nonempty descriptions and nonmissing source categories, then sample 200,000 eligible entries with seed 534. The sample contains 37,570 distinct normalized descriptions; 1,725 appear with more than one source category. Those inconsistencies create label ambiguity. The source category is the prediction target, not a validated nutritional interpretation of the food.

The first split assigns approximately 20% of entries to testing. The second uses a fixed SHA-256 rule to hold out entire normalized descriptions. Its test entries therefore share no exact normalized description with training. Both evaluations weight food entries equally, so frequent descriptions

receive more weight. A second evaluation weighting unique descriptions equally would answer a different product question.

A training-only lookup chooses the most frequent category for each description, breaking count ties alphabetically and falling back to the most frequent training category. The text alternative fits a TF-IDF vocabulary of unigrams and bigrams on distinct training descriptions, caps the vocabulary at 20,000 features, and requires a term to occur in at least two training descriptions. Multinomial naive Bayes uses smoothing of 0.1 and training description-category counts as weights.

Evaluation	Test Entries	Familiar Descriptions	Lookup Accuracy	Text Accuracy	Text Macro F1
Entry Split	40,273	88.65%	84.90%	84.94%	0.5531
Description Split	36,987	0%	3.82%	67.42%	0.3465

Macro F1 averages across the 312 labels present in the combined sampled data, assigning zero to undefined class scores. Rare labels therefore matter more than in accuracy. The gap between the two evaluations reflects both familiarity and the changed test composition; it is not a causal estimate of leakage's effect. For a service processing new descriptions, the description-level evaluation is the closer test of that requirement.

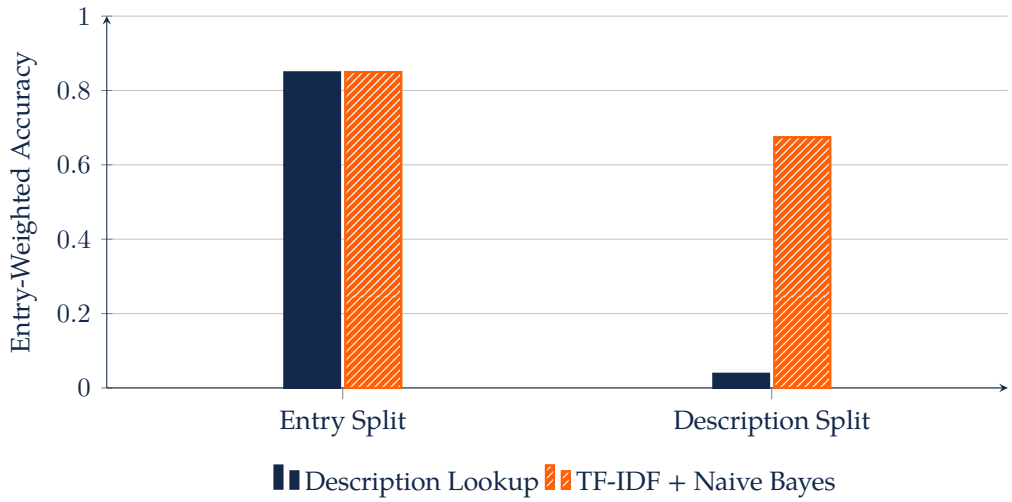


Figure 5.3. Exact-description familiarity changes the task measured by a text-classification test.

The lookup nearly matches the text classifier on the entry split because most test descriptions already appear in training. Its collapse on unseen descriptions reveals how much of that performance depends on memory. A product could use the lookup for verified familiar terms and route uncertain new terms for review. Nutrition-based food composition matching would additionally require portion units, preparation details, and validated food codes. A category prediction alone supplies none of those quantities.

An AI assistant can help inspect ambiguous examples and propose a labeling rubric. Use only permitted local processing, preserve the original category, and evaluate proposed relabel-

ing against independently reviewed examples. The code below makes no external model request and learns every lookup and text transformation from training rows only.

Python source: m05.py. From its directory, run `python m05.py`.

```
"""Contrast familiar-description and unseen-description classification."""

import json
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, f1_score
from common import SEED, description_holdout, load_food, save_result

food = load_food()
usable = food.loc[food.description.ne("") & food.category.ne("Unclassified")]
sample = usable.sample(n=min(200000, len(usable)), random_state=SEED).reset_index(
    drop=True
)
rng = np.random.default_rng(SEED)
masks = {
    "entry_split": rng.random(len(sample)) < 0.2,
    "description_split": sample.description.map(description_holdout).to_numpy(),
}
results = {}
for name, mask in masks.items():
    train, test = sample.loc[~mask], sample.loc[mask]
    counts = train.groupby(["description", "category"]).size().reset_index(name="count")
    # Break tied category counts alphabetically; learn the lookup only on training rows.
    lookup = counts.sort_values(["count", "category"], ascending=[False, True])
    lookup = lookup.drop_duplicates("description").set_index("description").category
    majority = train.category.value_counts().sort_index().idxmax()
    baseline = test.description.map(lookup).fillna(majority)
    vectorizer = TfidfVectorizer(ngram_range=(1, 2), max_features=20000, min_df=2)
    vectorizer.fit(train.description.drop_duplicates())
    classifier = MultinomialNB(alpha=0.1)
    classifier.fit(
        vectorizer.transform(counts.description),
        counts.category,
        sample_weight=counts["count"],
    )
    prediction = classifier.predict(vectorizer.transform(test.description))
    labels = sorted(set(train.category) | set(test.category))
    results[name] = {
        "train_rows": len(train),
        "test_rows": len(test),
        "familiar_description_fraction": float(
            test.description.isin(lookup.index).mean()
        ),
        "lookup_accuracy": float(accuracy_score(test.category, baseline)),
        "nb_accuracy": float(accuracy_score(test.category, prediction)),
        "nb_macro_f1": float(
```

```
            f1_score(
                test.category,
                prediction,
                labels=labels,
                average="macro",
                zero_division=0,
            )
        ),
        "test_label_unseen_in_train_fraction": float(
            (~test.category.isin(train.category.unique())).mean()
        ),
        "metric_classes": len(labels),
    }
ambiguity = sample.groupby("description").category.nunique()
result = {
    "sample_rows": len(sample),
    "descriptions": len(ambiguity),
    "descriptions_with_multiple_labels": int(ambiguity.gt(1).sum()),
    "splits": results,
}
save_result("M05", result)
print(json.dumps(result, indent=2))
```

5.11 Python Calculations for the Synthetic Cases

The inputs below are imposed teaching values. Running the program creates the labeled synthetic CSV files and prints the corresponding calculations. They are separate from the mHealth estimates. Keep the script beside `example_helpers.py` in `03_Data_and_Code/Lecture_Examples`.

Python source: m05_synthetic.py. From its directory, run `python m05_synthetic.py`.

```
"""Synthetic numerical examples for M5; run from any working directory."""

import itertools
import json
import math
from example_helpers import write_csv, ols_slope, shares

r = {}
r["M05"] = {}
for positive in [200, 10]:
    tp = 0.8 * positive
    fn = positive - tp
    fp = 0.1 * (1000 - positive)
    tn = 1000 - positive - fp
    r["M05"][str(positive / 1000)] = {
        "tp": tp,
        "fp": fp,
        "fn": fn,
        "tn": tn,
```

```
        "precision": tp / (tp + fp),
        "accuracy": (tp + tn) / 1000,
    }
write_csv(
    "M05_annotation_synthetic.csv",
    [
        "prevalence",
        "true_positive",
        "false_positive",
        "false_negative",
        "true_negative",
    ],
    [(p, v["tp"], v["fp"], v["fn"], v["tn"]) for p, v in r["M05"].items()],
)
print(json.dumps(r, indent=2))
```

5.12 mHealth: Text Topics and Their Denominators

A product manager wants to organize a large food-description vocabulary into interpretable groups. Repeated entries make a frequently logged description prominent, while a catalog of distinct descriptions emphasizes the range of language available. These populations answer different questions. A topic model fitted to distinct descriptions can summarize vocabulary, but its topic shares need a separate choice of observation unit.

Use all 50,860 distinct nonempty normalized descriptions recorded before July 1. Fit TF-IDF on these descriptions, retaining at most 3,000 unigram and bigram features, removing English stop words, and requiring a feature in at least three documents. The maximum document-frequency threshold is 0.95. Fix the number of nonnegative matrix factorization components at four as an illustrative specification. No July description is used to learn the vocabulary, inverse-document frequencies, or component vectors.

Let V contain nonnegative TF-IDF features, with one row per training description. NMF approximates V by WH, where the rows of H contain term patterns and each row of W gives the associated nonnegative loadings. The chosen specification minimizes squared reconstruction error subject to nonnegative factors. Multiplying a row of H by a constant and dividing its column of W by the same constant preserves the approximation. Before comparing loadings, the code therefore scales them to correspond to component term vectors with unit Euclidean norm.

July contains 17,042 distinct nonempty descriptions and 79,957 entries, including 5,208 descriptions absent from training. Apply the frozen vectorizer and factorization to July descriptions. Assign each description to its largest scaled loading when positive. A zero-vector description remains unassigned. The resulting groups have the following leading terms and shares.

Leading Terms	Share of Distinct July Descriptions	Share of July Entries
Chicken, Breast, Soup	14.83%	10.12%
Cheese, Cheddar, Pizza	32.52%	29.92%
Chocolate, Bar, Dark	32.22%	38.45%
Salad, Dressing, Greek	17.01%	19.15%
Unassigned	3.42%	2.36%

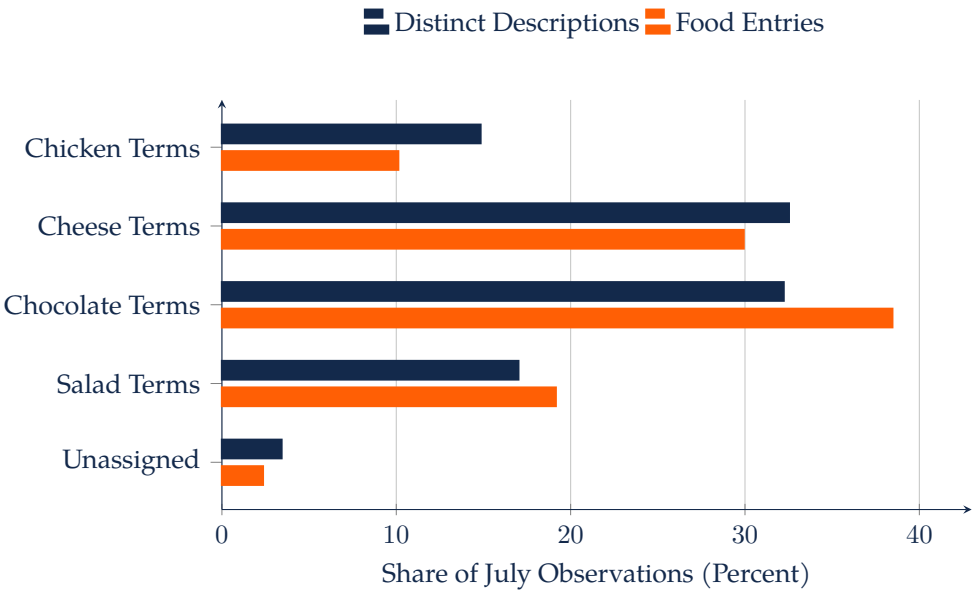


Figure 5.4. Weighting descriptions by their observed entry frequency changes the apparent prominence of the same fixed topics.

The chocolate-related component receives a larger share when repeated entries determine the weights. This result concerns the language and repetition of recorded entries. The component includes other terms and does not define a validated food group, calorie share, or measure of nutritional quality. Assigning a dominant component also discards information about mixtures. The 583 unassigned descriptions expose a vocabulary-coverage problem that would be hidden by forcing every text into a topic.

Blueprints for Text Analytics, Chapter 8 provides an NMF topic-model example. BERTopic's representation documentation offers separate methods for describing topics. An LLM-generated topic name should remain attached to the component's terms and inspected examples. A fluent name supplies no independent evidence that a component measures preference, healthfulness, or a commercial segment.

A useful AI-assisted extension changes one choice at a time: the number of topics, frequency weighting during fitting, or the vocabulary threshold. Compare representative descriptions and unmatched texts as well as reconstruction error. Improving the numerical objective alone does not establish that a proposed topic label answers the manager's question.

Python source: topics.py. From its directory, run `python topics.py`.

```
"""Local topic decomposition of food descriptions with a future-text check."""

import json
import numpy as np
from sklearn.decomposition import NMF
from sklearn.feature_extraction.text import TfidfVectorizer
from common import SEED, load_food, save_result


if __name__ == "__main__":
    food = load_food()
    prior = food.loc[(food.date < "2016-07-01") & food.description.ne("")]
    training_text = sorted(prior.description.unique())
    july = food.loc[
        food.date.between("2016-07-01", "2016-07-31") & food.description.ne("")
    ]
    counts = july.groupby("description").size().sort_index()
    vectorizer = TfidfVectorizer(
        min_df=3,
        max_df=0.95,
        max_features=3000,
        ngram_range=(1, 2),
        stop_words="english",
    )
    train_matrix = vectorizer.fit_transform(training_text)
    model = NMF(
        n_components=4, init="nndsvda", random_state=SEED, max_iter=500, tol=1e-4
    )
    model.fit(train_matrix)
    future_matrix = vectorizer.transform(counts.index)
    # Put topic term vectors on a common L2 scale before comparing loadings.
    weights = model.transform(future_matrix) * np.linalg.norm(model.components_, axis=1)
    assigned = weights.sum(axis=1) > 1e-12
    labels = np.where(assigned, weights.argmax(axis=1), -1)
    terms = vectorizer.get_feature_names_out()
    rows = []
    for topic in range(4):
        selected = labels == topic
        top = np.argsort(-model.components_[topic], kind="stable")[:6]
        rows.append(
            {
                "topic": topic + 1,
                "terms": terms[top].tolist(),
                "distinct_description_share": float(selected.mean()),
                "food_entry_share": float(
                    counts.to_numpy()[selected].sum() / counts.sum()
                ),
            }
        )
```

```
result = {
    "training_distinct_descriptions": len(training_text),
    "training_entries": len(prior),
    "vocabulary_terms": len(terms),
    "july_distinct_descriptions": len(counts),
    "july_entries": int(counts.sum()),
    "july_new_descriptions": int((~counts.index.isin(training_text)).sum()),
    "july_zero_vocabulary_descriptions": int(
        (future_matrix.getnnz(axis=1) == 0).sum()
    ),
    "july_unassigned_descriptions": int((~assigned).sum()),
    "july_unassigned_entries": int(counts.to_numpy()[~assigned].sum()),
    "iterations": int(model.n_iter_),
    "topics": rows,
}
assert model.n_iter_ < model.max_iter
assert np.isclose(
    sum(row["food_entry_share"] for row in rows)
    + result["july_unassigned_entries"] / counts.sum(),
    1,
)
save_result("topics", result)
print(json.dumps(result, indent=2))
```

Chapter 6

Causal Effects and Marketing Experiments

A retailer wants to know whether an email campaign increases contribution. Customers who open marketing emails often purchase more, but they may already be more engaged. Comparing openers with nonopeners can therefore reward a campaign for behavior it did not cause. The decision requires a comparison between outcomes under alternative actions for the same target population.

6.1 Define the Treatment, Outcome, and Estimand

Let $Y_i(1)$ be customer i's 30-day contribution if assigned an email and $Y_i(0)$ the contribution if not assigned. The individual effect is $Y_i(1) - Y_i(0)$. Only one of these outcomes is observed for a given customer under a single assignment. A causal design supplies a defensible way to learn an average comparison despite that missing counterfactual.

For a target population, the average treatment effect is

$$\text{ATE} = E[Y_i(1) - Y_i(0)].$$

The expectation refers to the stated population and outcome window. An average treatment effect on the treated uses the population that actually received treatment. An intention-to-treat effect uses assignment as the treatment, even if some assigned customers never open the email. These are different estimands. The managerial decision to send an email often corresponds closely to the assignment effect, because opening is outside the firm's direct control.

A well-defined treatment includes the offer, timing, channel, and implementation. "Digital marketing" is too broad to identify a single comparison. The outcome should include economically relevant costs and have enough follow-up to capture returns or delayed purchases. If customers can affect one another, an individual's outcome may depend on other customers' assignments. The unit and design then need to account for interference rather than silently assuming it away.

6.2 Why the Treated-Untreated Difference Can Be Biased

The observed difference between treated and untreated customers can be decomposed as

$$E[Y \mid D = 1] - E[Y \mid D = 0] = E[Y(1) - Y(0) \mid D = 1] + \{E[Y(0) \mid D = 1] - E[Y(0) \mid D = 0]\}.$$

Here D indicates treatment. The first term is the average treatment effect among treated customers. The bracketed term compares their untreated outcomes with untreated customers' outcomes. If the firm targets people who would purchase anyway, that selection term can be positive. If it targets customers in serious decline, the term can be negative. The direction follows the assignment process, not a universal rule about observational data.

An unobserved engagement variable may affect both email opening and purchasing. Adjusting for pre-treatment engagement measures can be useful if they capture the relevant confounding, but controlling for a variable does not guarantee this condition. A post-treatment variable such as clicking the email can mediate part of its effect. Conditioning on customers who click changes the population and can introduce selection.

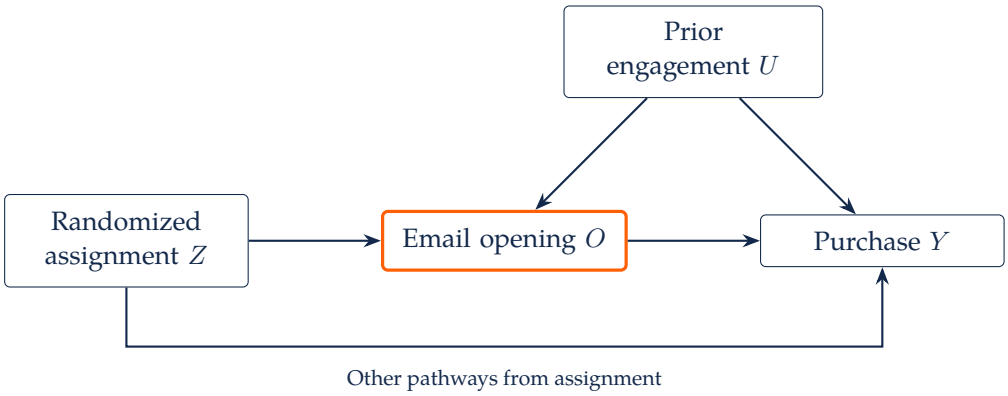


Figure 6.1. Assignment, engagement, and opening define different causal comparisons.

The firm randomizes email assignment, while customers choose whether to open. Assignment can affect opening, and engagement can affect both opening and purchases. Restricting attention to openers creates a selected group. An assignment-based analysis preserves the randomized comparison when its implementation and outcome measurement are valid.

6.3 Designing an Experiment Around the Decision

An experiment needs an eligible population, assignment unit, treatment implementation, primary outcome, horizon, and analysis plan. Randomize at a unit that corresponds to exposure and likely spillovers. A customer-level email experiment may be appropriate when accounts act independently. A store-level display intervention requires store assignment or a design that accounts for shared exposure. Household, geographic, or network spillovers can require broader clusters.

Balance treatment groups through randomization, with blocking or stratification when important pre-treatment characteristics are known. For example, randomize within customer-tenure groups to improve balance and precision. Record assignment before contact and retain every assigned unit in the primary analysis when estimating the intention-to-treat effect. Excluding nonopeners, customers who later unsubscribe, or stores with disappointing execution can undermine the randomized comparison.

Specify a primary outcome and guardrails before viewing results. Incremental contribution might be primary, with unsubscribe rates, complaints, and returns as guardrails. Multiple outcomes can be informative, but selecting the most favorable result after examining many tests exaggerates evidence. Decide how multiple testing, sequential monitoring, and stopping will be

handled. Ordinary fixed-horizon p-values do not generally retain their advertised interpretation under unrestricted repeated peeking and outcome selection.

6.4 Case Study: Estimate an Email Assignment Effect

In a synthetic experiment, 1,000 customers are assigned an email and 1,000 are assigned no email. Within 30 days, 130 assigned customers and 100 controls make at least one eligible purchase. The estimated assignment effect on purchase incidence is

$$\hat{\tau} = \frac{130}{1000} - \frac{100}{1000} = 0.03,$$

or three percentage points. Relative to the control rate of 10%, this is a 30% relative increase. The two descriptions use different denominators; reporting only "30% lift" can obscure the absolute change.

Under independent customer assignments and the usual large-sample approximation, an unpooled standard error is

$$\widehat{\mathrm{SE}}(\hat{\tau}) = \sqrt{\frac{0.13(0.87)}{1000} + \frac{0.10(0.90)}{1000}} \approx 0.0143.$$

An approximate 95% interval is $[0.0021, 0.0579]$, or about 0.21 to 5.79 percentage points. This interval describes sampling uncertainty under the stated design and approximation. It does not account for an unmeasured outcome-definition error or interference between assigned customers.

Suppose an additional purchase contributes \$40 and email delivery costs \$0.50 per assigned customer. If the offer has no additional discount cost and purchase incidence is the relevant incremental quantity, estimated incremental contribution is $\$40(0.03)$-\$0.50=\$0.70 per assigned customer. Applying the same transformation to the effect interval gives approximately \$-0.42 to \$1.82. A positive point estimate therefore coexists with economically consequential uncertainty. A decision can combine that uncertainty with rollout costs, the value of another test, and guardrail outcomes.

The margin assumption matters. If treatment also changes basket size, returns, or discount expense, purchase incidence alone is insufficient. An experiment measuring contribution directly may better match the objective.

6.5 Power, Precision, and the Minimum Useful Effect

Power planning begins with the smallest effect that could change the decision, not with a convenient number of rows. For two equally sized independent groups and a binary outcome with baseline rate near p, a rough per-group sample-size approximation is

$$n \approx \frac{2p(1-p)(z_{1-\alpha/2} + z_{1-\beta})^2}{\delta^2},$$

where δ is the absolute effect in probability units, α is the two-sided significance level, and $1-\beta$ is target power. This approximation uses a common baseline variance and normal critical values.

More precise planning can use the expected rates under the alternative and the actual assignment design.

With $p = 0.10$, $\delta = 0.02$, $\alpha = 0.05$, and 80% power, using 1.96 and 0.84 gives about 3,528 customers per group. Halving the target effect approximately quadruples the required sample under this formula. A two-percentage-point effect is 0.02, not 2; a unit error can produce a meaningless sample-size recommendation.

Clustering reduces the amount of independent information. A common approximation with equal cluster size m and intracluster correlation ρ multiplies individual-level variance by $1 + (m - 1)\rho$. For $m = 20$ and $\rho = 0.05$, that factor is 1.95. The number of clusters, unequal cluster sizes, assignment design, and small-sample inference still require explicit consideration. Thousands of customers across four stores do not provide thousands of independent store-level treatment assignments.

6.6 Implementation Checks That Protect the Comparison

Verify that the assignment counts match the intended allocation and investigate material sample-ratio discrepancies. Check whether tracking differs between treatment and control, whether treatment changes the probability an outcome is observed, and whether exclusions were applied consistently. Pre-treatment balance is a diagnostic, not proof that every causal assumption holds. In a correctly randomized experiment, chance imbalances can occur.

Noncompliance changes the estimand. If only 40% of assigned customers open the email, the intention-to-treat effect remains the effect of assignment under the observed implementation. Dividing it by an opening-rate difference identifies a complier effect only under additional instrumental-variable conditions, including an appropriate exclusion restriction and monotonicity. The firm may care about the assignment effect even when a different scientific question concerns actual exposure.

Use pre-treatment outcomes to improve precision when justified. A regression adjustment can compare outcomes while accounting for baseline spending or stratification variables. Avoid selecting an adjustment because it makes the treatment coefficient significant. Report the planned analysis and explain any departures. Analyze heterogeneity cautiously, especially when many subgroups are examined after seeing the main result.

6.7 Observational Adjustment and Its Limits

When randomization is unavailable, a common approach assumes that treatment is conditionally independent of potential outcomes given observed pre-treatment covariates, with overlap in treatment probabilities. Regression adjustment, matching, and inverse-probability weighting implement different ways to construct a comparison under that condition. The central question is whether the observed covariates make the treated and untreated outcomes comparable.

A propensity score is the treatment probability given those covariates. Extreme estimated scores indicate limited overlap and can create unstable weights. Trimming or restricting the sample changes the population for which the effect is estimated. Report that population explicitly. A flex-

ible prediction algorithm for treatment assignment does not establish that all confounders were measured.

A doubly robust estimator combines an outcome model and a treatment model and can be consistent when one nuisance model is correctly specified, under the underlying identification conditions and regularity assumptions. This robustness does not cover unmeasured confounding. For advanced implementations, sample splitting and cross-fitting help control overfitting of nuisance functions; the design remains the foundation of the causal interpretation.

6.7.1 Evaluating a Recommendation Policy From Logged Exposure

A retailer has records from an existing recommendation policy and wants to evaluate a different one. The existing policy shows products selectively: frequent buyers may usually see A, while occasional buyers usually see B. Comparing their observed click rates mixes product response with the customer populations receiving each recommendation. Schnabel and coauthors (2016) formulate recommendation learning and evaluation as problems of selected observation and use propensity weighting to address that selection under explicit conditions.

Consider a single recommendation opportunity per customer. Let X_i be information available before display, A_i the displayed item, and Y_i a click within the specified response window. The logging policy assigns item a with probability $p_0(a \mid X_i)$. A target policy assigns it with probability $\pi(a \mid X_i)$. The target is the expected clicks per opportunity if eligible customers receive recommendations from π. An inverse-propensity estimator is

$$\widehat{V}(\pi) = \frac{1}{n} \sum_{i=1}^{n} \frac{\pi(A_i \mid X_i)}{p_0(A_i \mid X_i)} Y_i.$$

The ratio changes how much each logged outcome represents the target policy. For a policy that always recommends A, outcomes following B receive zero weight. An A outcome observed under a 10% display probability receives weight ten. The denominator n counts all eligible logged opportunities, including those assigned zero weight. Dividing only by the number of matching displays estimates a different quantity.

Interpretation requires a well-defined action and outcome, conditional exchangeability of display given X_i, and positive logging probability wherever the target policy assigns positive probability. A known randomized logging policy can supply the assignment condition. Estimating probabilities from observational logs requires the additional substantive claim that the recorded context captures the relevant selection. The target population and response process must also match the logged environment. For a slate of products, position and other displayed items can affect the response; a single-item propensity generally does not describe assignment of the complete slate.

The weighting identity explains the adjustment. Conditional on $X = x$, averaging over logged actions gives $\sum_a p_0(a \mid x)\{\pi(a \mid x)/p_0(a \mid x)\}E[Y(a) \mid X = x]$. The logging probability cancels, leaving the response weighted by the target policy. Cancellation requires positive support. If an item was never eligible for a customer group, multiplying the observed records by larger weights cannot reveal that group's response to it.

6.7.2 Case Study: Exposure Adjustment Reverses the Preferred Item

Suppose 1,000 synthetic customers comprise 500 frequent buyers and 500 occasional buyers. The logging policy randomly assigns A within each group with probabilities 0.90 and 0.10, respectively. B receives the remaining probability. The constructed counts are:

Customer Group	Item	Display Probability	Displays	Clicks
Frequent	A	0.90	450	270
Frequent	B	0.10	50	25
Occasional	A	0.10	50	10
Occasional	B	0.90	450	180

A receives 280 clicks from 500 displays, or 56%. B receives 205 from 500, or 41%. Yet A's within-group rates are 60% and 20%, while B's are 50% and 40%. Applied to the same equally weighted customer population, A gives 40% and B gives 45%. A's displayed customers disproportionately come from the group with high response to either item.

The inverse-propensity calculations recover the common-population comparison:

$$\widehat{V}(A) = \frac{270/0.90 + 10/0.10}{1000} = 0.40, \qquad \widehat{V}(B) = \frac{25/0.10 + 180/0.90}{1000} = 0.45.$$

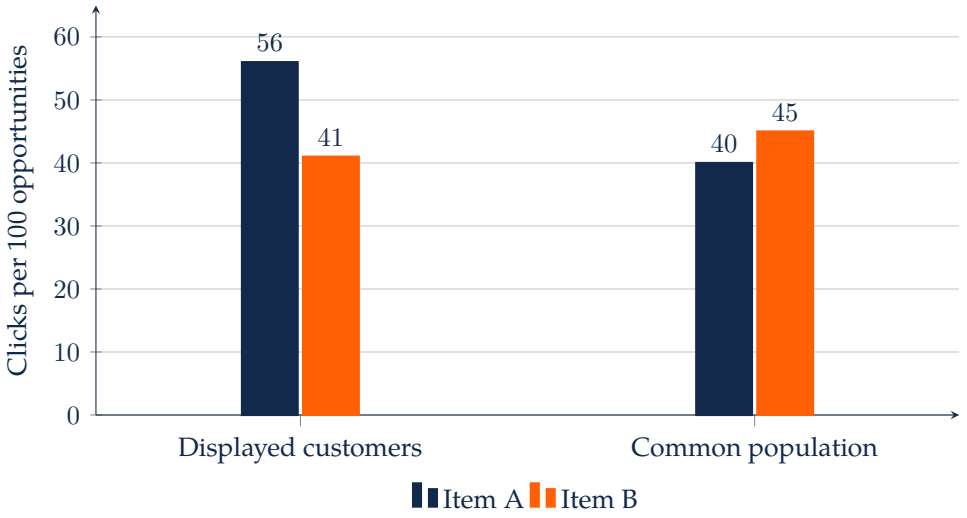


Figure 6.2. Selected display populations favor A, while comparison on the same population favors B.

The raw bars describe response among each item's displayed customers. The adjusted bars describe the same 50:50 customer mix for both policies. The five-percentage-point advantage for B is imposed by the synthetic cell counts and randomized logging design. It is not an estimate from the cited study.

Large weights reduce precision. For A, the 50 occasional buyers each represent ten eligible customers, so a small number of outcomes carries substantial influence. The diagnostic $(\sum_i w_i)^2 / \sum_i w_i^2$ equals 180 for either policy in this example, despite 1,000 logged opportunities. This effective sample size describes weight concentration; it does not replace a variance

calculation that respects assignment and dependence. Weight clipping trades variance for bias, and self-normalization changes the finite-sample estimator. Report these choices and retain an untouched evaluation sample when choosing between policies.

6.8 Difference-in-Differences and a Counterfactual Trend

A retailer introduces an app in treated markets while comparable markets remain untreated. A before-after difference in treated markets mixes the app's effect with common demand changes. Difference-in-differences subtracts the comparison group's change:

$$\hat{\tau}_{\text{DiD}} = (\bar{Y}_{T,post} - \bar{Y}_{T,pre}) - (\bar{Y}_{C,post} - \bar{Y}_{C,pre}).$$

In a synthetic example, treated-market sales rise from 120 to 150 and comparison-market sales rise from 100 to 110. The DiD estimate is 20 sales units. Under the identifying parallel-trends condition, the treated markets' untreated counterfactual would have risen to 130. Different starting levels are compatible with this design; the condition concerns untreated changes.

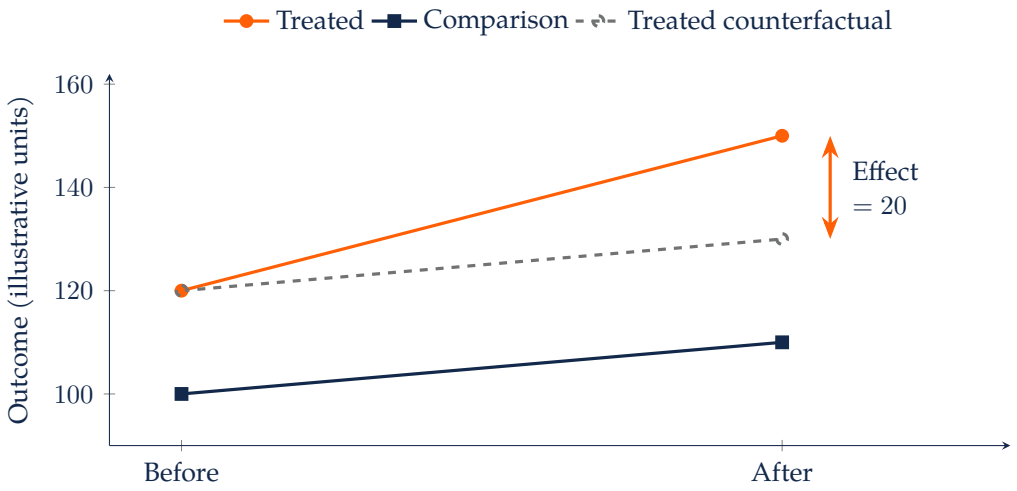


Figure 6.3. Difference-in-differences subtracts the comparison group's change to construct the counterfactual.

The dashed line is a counterfactual implied by parallel untreated trends, not an observed sales series. The vertical gap between observed treated sales and that line is the estimated effect. A treatment-specific local shock at the same time would challenge the interpretation even if the pre-treatment levels looked similar.

Inspect multiple pre-treatment periods, institutional timing, possible anticipation, spillovers, and changes in sample composition. Similar pre-trends can support the design's plausibility, but a nonsignificant pre-trend test does not establish parallel counterfactual trends. With staggered adoption and heterogeneous effects, conventional two-way fixed-effects coefficients can combine comparisons in problematic ways. Use estimators aligned with cohort and time-specific effects, with a clear comparison group, when extending beyond the simple two-period design.

6.9 Other Designs: Discontinuities and Instruments

A regression discontinuity design can use a treatment rule at a threshold, such as eligibility based on a pre-existing score. Customers just above and below the cutoff may be comparable if other determinants of the outcome vary smoothly there and the score is not precisely manipulated. The resulting effect is local to the cutoff and the relevant compliance behavior. Bandwidth, functional form, density patterns, and discontinuities in other predetermined variables deserve inspection.

An instrumental variable changes treatment while affecting the outcome only through the treatment channel, under the design's conditions. Relevance is an empirical first-stage requirement; exclusion is a substantive condition. A strong first stage cannot establish exclusion. In a binary-treatment setting with independence, exclusion, and monotonicity, the familiar local average treatment effect concerns compliers whose treatment changes with the instrument. It need not equal the population ATE.

Marketing instruments require particularly careful reasoning because pricing, distribution, and exposure decisions often respond to demand. A variable correlated with treatment is not sufficient. Explain who makes the relevant decision, when the instrument is determined, and which other channels could connect it to outcomes. If those channels remain credible, the causal interpretation needs revision.

6.10 Spillovers and Experiment Units

An advertising campaign can shift purchases between stores or products rather than increase total demand. A customer-level outcome may show a gain while chain-wide contribution changes little. Define the outcome at the level relevant to the managerial objective and examine displacement where it is plausible. The same issue arises when customers share offers or when a platform changes recommendations for multiple users.

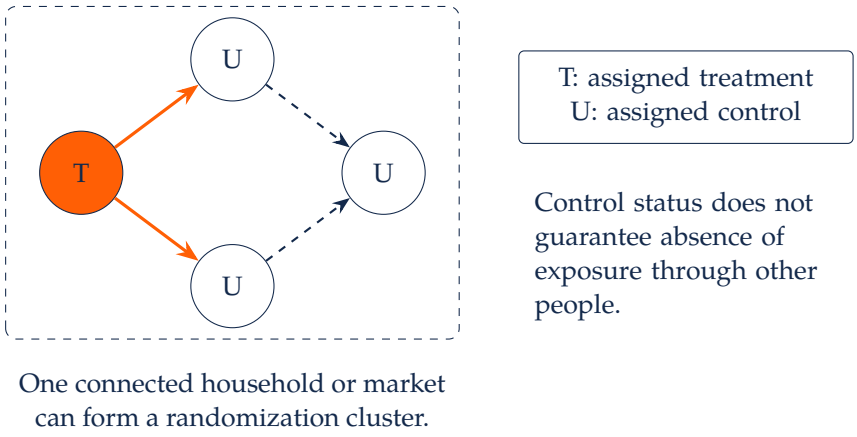


Figure 6.4. A treatment can affect both assigned units and nearby untreated units.

An assigned store can gain sales while diverting purchases from another store. If the comparison store loses customers, subtracting its decline can overstate the increase in total chain sales. Cluster

randomization, geographic separation, or designs that vary treatment saturation may help identify the relevant effects, subject to their own practical constraints.

6.10.1 Case Study: Recommendations Supply Information and Exposure

A recommendation can increase consumption by bringing a product to attention or by changing beliefs about its quality. Aridor, Gonçalves, Kluver, Kong, and Konstan's MovieLens experiment, in its November 2022 working-paper version, separates recommendation and exposure-only conditions and elicits users' beliefs. The results support an informational role: recommendations influence beliefs and consumption beyond simply exposing a movie. MovieLens provides recommendations without selling or hosting the movies, which helps define the incentives in this setting.

A retailer studying the same mechanism could distinguish ordinary display from an explicit personalized endorsement while keeping the eligible product set and available information controlled. The primary outcome would be purchases or contribution per assigned customer over a fixed horizon. Belief measures would address how the endorsement operates. The experiment must account for the possibility that asking about a product itself increases attention to it. Comparisons based only on customers who subsequently click would condition on a response that the endorsement can change.

The displayed information can also affect the labels collected afterward. Cosley and coauthors (2003) experimentally vary predicted ratings shown to MovieLens users and find that subsequent ratings tend to move toward the displayed prediction. A rating collected after a recommendation therefore reflects the evaluation context as well as the user's experience. A model trained to reproduce those ratings can partly reproduce its own previous influence. Collecting some evaluations before showing predicted ratings provides a comparison for that mechanism.

An experiment on an adaptive recommender also needs a rule for retraining. If both treatment and control data update a shared model, one customer's assignment may change another customer's recommendations. Freezing the model, maintaining separate models, or assigning sufficiently separated populations evaluates different policies. State which policy and learning process the experiment implements. A short experiment with a frozen model identifies a different intervention from deploying a model that learns from its own recommendations over several months.

6.11 AI-Assisted Workflow and Practice

Ask an assistant to state the estimand, assignment unit, outcome horizon, and comparison group before writing estimation code. For the email example, require the conversion effect in percentage points, the unpooled standard error, and the contribution calculation. Check the arithmetic independently. Ask it to explain why restricting the sample to email openers changes the analysis.

For observational data, draw or describe the assignment process before selecting controls. Request an overlap diagnostic and a report of which observations receive large weights. For DiD, ask the assistant to show observed and counterfactual series separately and identify the assumptions that connect them. A convincing plot should not substitute for those conditions.

If comparison-market sales rise by 25 rather than 10, the DiD estimate becomes five. If email cost doubles to \$1, the point estimate of incremental contribution falls to \$0.20. If treatment raises

returns, explain why a conversion effect can overstate the contribution effect. If two stores share most customers, identify why store assignment may create interference.

For the exposure example, ask an assistant to compute both the raw click rates and inverse-propensity policy values from the four aggregate rows, using the display counts when calculating denominators and weight concentration. Verify the weighted results independently by averaging the two group-specific response rates. Then set A's display probability among occasional buyers to zero in a proposed logging design and explain why the always-A policy becomes unidentified from those logs. The implementation should flag the unsupported action rather than silently drop that group.

6.12 Readings

Facure's causal-inference handbook supplies accessible implementations of several designs. Kohavi and colleagues' practical guide discusses controlled online experiments. Callaway and Sant'Anna (2021) develop difference-in-differences with multiple periods.

6.13 mHealth: Planning a Randomized Suggestion Test

The manager has a candidate suggestion policy and wants to know whether offering it changes customer behavior. The historical logs contain no documented randomized assignment or recommendation exposure. They can supply a distribution for planning precision and reveal how a proposed outcome is measured. An effect estimate requires the assignment and outcome records from a suitable new design.

Consider a user-randomized trial comparing the current interface with an offer of the new suggestions. Define the primary logging outcome as the fraction of the next 30 calendar days with at least one recorded food entry. Analyze all assigned eligible users under their original assignment, including users who never return. A suggestion may increase logging without changing consumption. A nutrition outcome therefore needs its own validated measurement and a plan for missingness. Post-assignment completeness screens could remove users differently across arms and alter the intention-to-treat comparison.

For historical planning, require at least 30 recorded days during January through May. Among the 327 eligible users, June's logging fraction averages 0.6408 with a standard deviation of 0.3542. Twenty-three users have no June entries and receive a zero logging fraction. That zero is meaningful for observed logging. It says nothing about zero consumption, nutritional adequacy, or contract cancellation.

With independent users, equal arm sizes, a two-sided 5% test, 80% power, and the historical standard deviation treated as the prospective outcome standard deviation, the normal approximation is

$$n_{\text{per arm}} \approx \frac{2\sigma^2(z_{0.975} + z_{0.80})^2}{\Delta^2}.$$

An imposed minimum detectable effect of 0.05 in the logging fraction, equivalent to 1.5 days out of 30, gives 788 users per arm after rounding upward. This is a planning scenario using his-

torical variance. Future eligibility, outcome distributions, unequal assignment, clustering, and the intended analysis can change the required size. The 327-user historical subset does not supply that many independent trial participants.

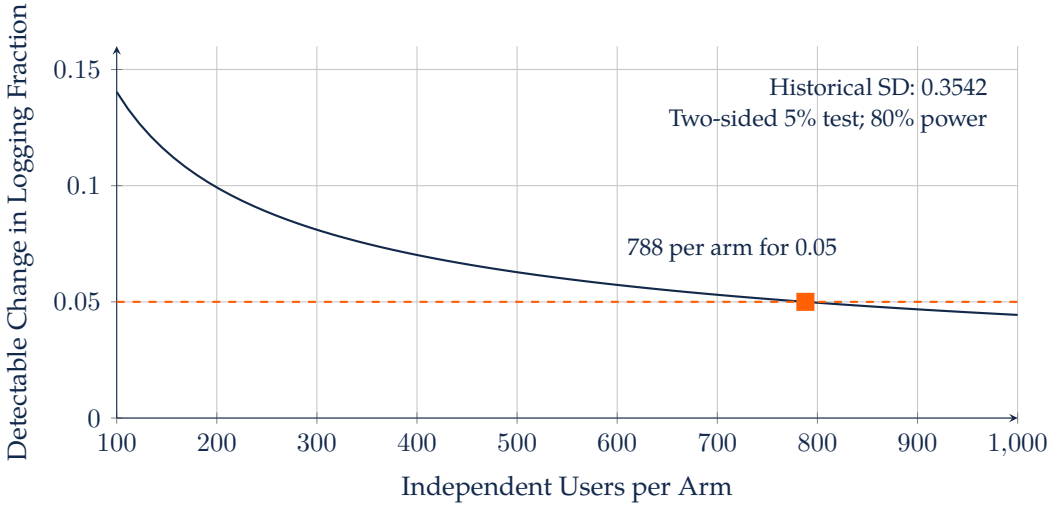


Figure 6.5. The curve holds the historical outcome variance and testing assumptions fixed. More independent users narrow the detectable effect; adding days within the same user does not supply the same information.

A randomization simulation assigns the same observed June outcomes to nearly equal arms 2,000 times while imposing a sharp zero effect. The standard deviation of the resulting mean difference is 0.0395, and the central 95% interval is approximately [-0.0798, 0.0740]. These differences arise from assignment variation alone. Repeatedly treating each user-day as an independent trial participant would conceal dependence within users and overstate the effective sample size.

The protocol should specify randomization before any policy exposure, record assignment and actual exposure separately, and keep the outcome window fixed. For nutrition assessments, collect reliable intake or appropriate validation measures independently of how often the app is used. Domain-based restrictions should be specified in advance, with their effect on the target population and missing-outcome analysis made explicit.

The program uses a local random generator with seed 534. It reports the assumed effect separately from the simulated differences and never labels an artificial assignment as an observed intervention.

Python source: m06.py. From its directory, run `python m06.py`.

```
"""Use historical logging variation to plan, not estimate, a randomized trial."""

import json
import numpy as np
from scipy.stats import norm
from common import SEED, load_inputs, save_result

food, daily, baskets = load_inputs()
prior = daily.loc[daily.date.between("2016-01-01", "2016-05-31")]
```

```
counts = prior.groupby("Userid").size()
eligible = counts[counts >= 30].index.sort_values()
june = daily.loc[daily.date.between("2016-06-01", "2016-06-30")]
# A zero means no recorded day in June, not zero consumption or poor health.
outcome = june.groupby("Userid").size().reindex(eligible, fill_value=0) / 30
sigma = float(outcome.std(ddof=1))
effect = 0.05  # Imposed minimum detectable increase in the fraction of logged days.
critical = norm.ppf(0.975) + norm.ppf(0.80)
per_arm = int(np.ceil(2 * sigma**2 * critical**2 / effect**2))
# Illustrative balanced assignment under a sharp zero-effect simulation.
rng = np.random.default_rng(SEED)
values = outcome.to_numpy()
differences = []
for repetition in range(2000):
    order = rng.permutation(len(values))
    treated, control = np.array_split(order, 2)
    differences.append(values[treated].mean() - values[control].mean())
result = {
    "eligible_users": len(eligible),
    "june_mean_logging_fraction": float(outcome.mean()),
    "june_sd": sigma,
    "zero_logged_days": int(outcome.eq(0).sum()),
    "assumed_effect": effect,
    "normal_approximation_users_per_arm": per_arm,
    "null_simulation_mean_difference": float(np.mean(differences)),
    "null_simulation_sd": float(np.std(differences, ddof=1)),
    "null_simulation_95_percent_interval": np.quantile(
        differences, [0.025, 0.975]
    ).tolist(),
}
save_result("M06", result)
print(json.dumps(result, indent=2))
```

6.14 Python Calculations for the Synthetic Cases

The inputs below are imposed teaching values. Running the program creates the labeled synthetic CSV files and prints the corresponding calculations. They are separate from the mHealth estimates. Keep the script beside `example_helpers.py` in `03_Data_and_Code/Lecture_Examples`.

Python source: m06_synthetic.py. From its directory, run `python m06_synthetic.py`.

```
"""Synthetic numerical examples for M6; run from any working directory."""

import itertools
import json
import math
from example_helpers import write_csv, ols_slope, shares

r = {}
se = math.sqrt(0.13 * 0.87 / 1000 + 0.1 * 0.9 / 1000)
```

```
ci = [0.03 - 1.96 * se, 0.03 + 1.96 * se]
r["M06"] = {
    "assignment_effect": 0.03,
    "standard_error": se,
    "effect_ci": ci,
    "contribution_per_assigned_customer": 40 * 0.03 - 0.5,
    "contribution_ci": [40 * x - 0.5 for x in ci],
    "sample_per_arm": 2 * 0.1 * 0.9 * (1.96 + 0.84) ** 2 / 0.02**2,
    "design_effect": 1 + (20 - 1) * 0.05,
    "difference_in_differences": (150 - 120) - (110 - 100),
}
write_csv(
    "M06_experiment_synthetic.csv",
    ["assignment", "assigned_customers", "purchasers"],
    [("email", 1000, 130), ("control", 1000, 100)],
)
write_csv(
    "M06_did_synthetic.csv",
    ["group", "before", "after"],
    [("treated", 120, 150), ("comparison", 100, 110)],
)

exposures = [
    ("frequent", "A", 0.9, 450, 270),
    ("frequent", "B", 0.1, 50, 25),
    ("occasional", "A", 0.1, 50, 10),
    ("occasional", "B", 0.9, 450, 180),
]
write_csv(
    "M06_exposure_synthetic.csv",
    ["customer_group", "item", "logging_probability", "displays", "clicks"],
    exposures,
)


def constant_policy_value(rows, item):
    groups = {row[0] for row in rows}
    selected = [row for row in rows if row[1] == item]
    if {row[0] for row in selected if row[2] > 0} != groups:
        raise ValueError(
            "Target action lacks positive logging support in a customer group"
        )
    n = sum(row[3] for row in rows)
    weighted_clicks = sum(clicks / prob for _, _, prob, count, clicks in selected)
    sum_weights = sum(count / prob for _, _, prob, count, clicks in selected)
    sum_squared_weights = sum(count / prob**2 for _, _, prob, count, clicks in selected)
    raw = sum(row[4] for row in selected) / sum(row[3] for row in selected)
    return {
        "raw_rate": raw,
        "ips_value": weighted_clicks / n,
        "effective_sample_size": sum_weights**2 / sum_squared_weights,
```

```
    }


r["M06"]["recommendation_policies"] = {
    item: constant_policy_value(exposures, item) for item in ["A", "B"]
}
for item, target in [("A", 0.4), ("B", 0.45)]:
    result = r["M06"]["recommendation_policies"][item]
    # Independent standardization uses the known 50:50 population mix.
    standardized = sum(
        0.5 * clicks / count for group, j, prob, count, clicks in exposures if j == item
    )
    assert math.isclose(result["ips_value"], target) and math.isclose(
        result["ips_value"], standardized
    )
    assert math.isclose(result["effective_sample_size"], 180)
unsupported = [
    (g, j, 0 if (g, j) == ("occasional", "A") else p, n, y)
    for g, j, p, n, y in exposures
]
try:
    constant_policy_value(unsupported, "A")
except ValueError:
    pass
else:
    raise AssertionError("An unsupported target policy must be rejected")
print(json.dumps(r, indent=2))
```

6.15 mHealth: Prior Activity and Experiment Precision

A manager wants an experiment to detect a useful change in logging with fewer participants. Users differ persistently in how often they record food. Random assignment balances those differences in expectation, while a particular assignment can still place more frequent loggers in one arm. A pre-treatment activity measure can explain part of the outcome variation and improve precision.

For a planned June experiment, retain users with at least 30 recorded days before June 1. Define X_i as the fraction of May days with a log and Y_i as the fraction of June days with a log. All 327 eligible users remain in both monthly denominators, including users with zero observed logging days. Fit a coefficient using March logging fractions to predict April fractions, and freeze it before June. The CUPED-adjusted outcome is

$$Y_i^{\text{adj}} = Y_i - \theta(X_i - \overline{X}), \qquad \widehat{\tau}_{\text{adj}} = overline Y_1^{\text{adj}} - overline Y_0^{\text{adj}}.$$

The subscripts 1 and 0 denote randomized assignment arms. With a fixed coefficient and a pre-treatment covariate unaffected by assignment, the adjustment removes a mean-zero difference between arms in expectation. It preserves the pooled outcome mean. Randomization supplies the treatment-effect comparison; covariate adjustment changes its precision. The arithmetic is the control-variate approach underlying CUPED, introduced by Deng, Xu, Kohavi, and Walker (2013).

For a fixed coefficient, the adjusted variance equals

$$\operatorname{Var}(Y^{\text{adj}}) = \operatorname{Var}(Y) + \theta^2 \operatorname{Var}(X) - 2\theta \operatorname{Cov}(X, Y).$$

At the population-optimal coefficient $\theta^* = \operatorname{Cov}(X, Y)/\operatorname{Var}(X)$, this becomes $\operatorname{Var}(Y)(1 - \rho^2)$. The historical coefficient used here is learned from different months, so its gain must be measured in the later comparison. A poorly transferred coefficient can increase variance. A covariate affected by treatment can also change the estimand and bias the treatment-effect calculation.

Historical Planning Quantity	Unadjusted	CUPED
Variance of June Outcome	0.12546	0.06315
Exact Sharp-Null Randomization SD	0.03918	0.02779
SD Across 2,000 Random Assignments	0.04032	0.02840
Approximate Users per Arm for 0.05	788	397

The March–April coefficient is 0.55386. The table compares the same June users and outcomes, with an adjusted-to-unadjusted variance ratio of 0.50334. The prospective sample-size calculation assumes independent users, equal arm sizes, a two-sided 5% test, 80% power, and a 0.05 absolute increase in logging fraction. It uses historical variance as a planning input and a normal approximation. Actual effects, variance shifts, interference, and attrition can change the needed sample.

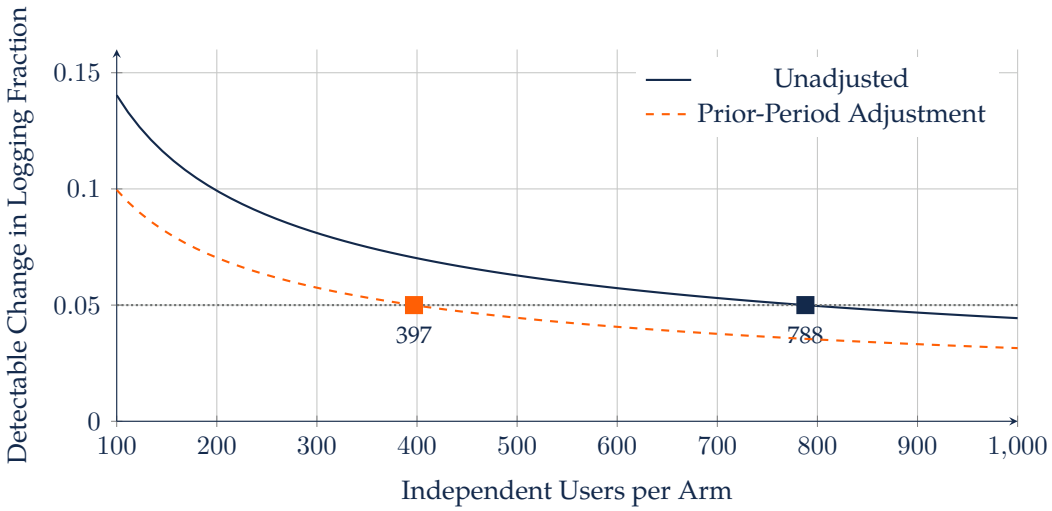


Figure 6.6. Historical covariate adjustment lowers the detectable change at a given sample size; both curves use the same outcome, significance level, and power.

No experimental assignment is observed in mHealth. The randomization exercise imposes a sharp zero-effect null, holding every recorded June outcome fixed while repeatedly assigning 163 users to treatment and 164 to control. Its dispersion illustrates the design's sampling variability. The analytical finite-population variance is $S^2(1/n_1 + 1/n_0)$ for each fixed outcome, and provides an independent comparison with the simulation.

FeedX combines experimental design with prior-period adjustment for shopping-feed tests. Its application also raises a choice of randomization unit. A test of generated product descriptions may

randomize products, whereas an mHealth reminder experiment could randomize users. Shared exposure and spillovers must be addressed at the unit actually assigned. Repeated fixed-horizon significance tests during monitoring require a sequentially valid procedure or a prespecified analysis time; software output does not resolve that design choice.

Python source: cuped.py. From its directory, run `python cuped.py`.

```
"""Historical precision planning with a coefficient frozen before June."""

import json
import math
import numpy as np
import pandas as pd
from scipy.stats import norm
from common import SEED, load_food, make_daily, save_result


def adjust(outcome, baseline, theta):
    return outcome - theta * (baseline - baseline.mean())


if __name__ == "__main__":
    daily = make_daily(load_food())
    prior = daily.loc[daily.date < "2016-06-01"].groupby("Userid").size()
    users = prior.index[prior >= 30]
    monthly = {}
    for month in ["2016-03", "2016-04", "2016-05", "2016-06"]:
        period = pd.Period(month, freq="M")
        part = daily.loc[daily.date.dt.to_period("M").eq(period)]
        monthly[month] = (
            part.groupby("Userid").size().reindex(users, fill_value=0).to_numpy()
            / period.days_in_month
        )
    march, april = monthly["2016-03"], monthly["2016-04"]
    theta = np.cov(march, april, ddof=1)[0, 1] / np.var(march, ddof=1)
    baseline, outcome = monthly["2016-05"], monthly["2016-06"]
    adjusted = adjust(outcome, baseline, theta)
    assert np.isclose(adjusted.mean(), outcome.mean())
    n, n_treated = len(users), len(users) // 2
    rng = np.random.default_rng(SEED)
    differences = []
    for _ in range(2000):
        treated = np.zeros(n, dtype=bool)
        treated[rng.choice(n, n_treated, replace=False)] = True
        # Imposed sharp null: these recorded outcomes do not change with assignment.
        differences.append(
            [
                values[treated].mean() - values[~treated].mean()
                for values in [outcome, adjusted]
            ]
        )
```

```
differences = np.asarray(differences)
power_multiplier = 2 * (norm.ppf(0.975) + norm.ppf(0.8)) ** 2 / 0.05**2
result = {
    "users": n,
    "coefficient_from_march_april": float(theta),
    "may_june_correlation": float(np.corrcoef(baseline, outcome)[0, 1]),
    "june_mean": float(outcome.mean()),
    "sharp_null_draws": len(differences),
    "methods": [],
}
for column, (name, values) in enumerate(
    [("unadjusted", outcome), ("cuped", adjusted)]
):
    variance = np.var(values, ddof=1)
    result["methods"].append(
        {
            "method": name,
            "outcome_variance": float(variance),
            "simulated_effect_mean": float(differences[:, column].mean()),
            "simulated_effect_sd": float(differences[:, column].std(ddof=1)),
            "exact_randomization_sd": float(
                np.sqrt(variance * (1 / n_treated + 1 / (n - n_treated)))
            ),
            "approximate_users_per_arm_for_0_05": math.ceil(
                power_multiplier * variance
            ),
        }
    )
result["variance_ratio"] = float(np.var(adjusted, ddof=1) / np.var(outcome, ddof=1))
save_result("cuped", result)
print(json.dumps(result, indent=2))
```

6.16 McDonald's: Repair Policy and the Choice of a Comparison Group

A franchise operator evaluating access to repair services wants to know whether a policy reduces interruptions and improves the return to maintenance. The 2024 final rule introduced a limited circumvention exemption for diagnosis, maintenance, and repair of retail-level commercial food-preparation equipment. Its effective date is October 28, 2024, as stated on page 85437; the relevant provision appears on pages 85448–85449. Announcement timing, the legal effective date, and actual adoption of new repair practices are distinct dates. A change in an ordering indicator can arise through several channels described in Section 2.16.

The weekly country data permit a concrete calculation of the comparison that a difference-in-differences design would use. Retain the 12 common weekly snapshots before October 28 and the 12 after the implementation week. Exclude that week from the pre/post means. Require at least 2,000 US records and 500 records in each comparison country. These imposed thresholds retain

all 24 pre/post weeks for the USA, Canada, and the UK in this window. Compute each weekly percentage from its numerator and denominator, then weight weeks equally.

For comparison country c, the descriptive contrast is

$$\widehat{D}_c = (\overline{p}_{US,post} - \overline{p}_{US,pre}) - (\overline{p}_{c,post} - \overline{p}_{c,pre}).$$

Here p is the percentage of observed records reporting unavailability. The contrast has percentage-point units. A causal interpretation would require the US outcome to follow the comparison country's change in the absence of the policy, together with comparable measurement, a suitable exposure definition, and no consequential spillovers or simultaneous country-specific changes. The national policy supplies one treated country. Repeated stores and weeks do not create additional independent policy assignments.

Country	Pre-Period Mean (%)	Post-Period Mean (%)	Change (Percentage Points)
USA	14.80	16.87	+2.07
Canada	6.09	7.52	+1.42
UK	19.36	8.06	-11.29

The US-minus-Canada contrast is +0.65 percentage points. The US-minus-UK contrast is +13.37 points. The UK pre-period contains a large temporary rise in recorded unavailability; its subsequent decline dominates the second contrast. The time paths therefore matter for selecting a comparison, even when a regression can readily produce a coefficient. Canadian record counts are constant at 1,428 in this window. That regularity calls for checking the source's refresh process alongside the apparent stability of the percentage. A constant count alone does not establish a stale feed.

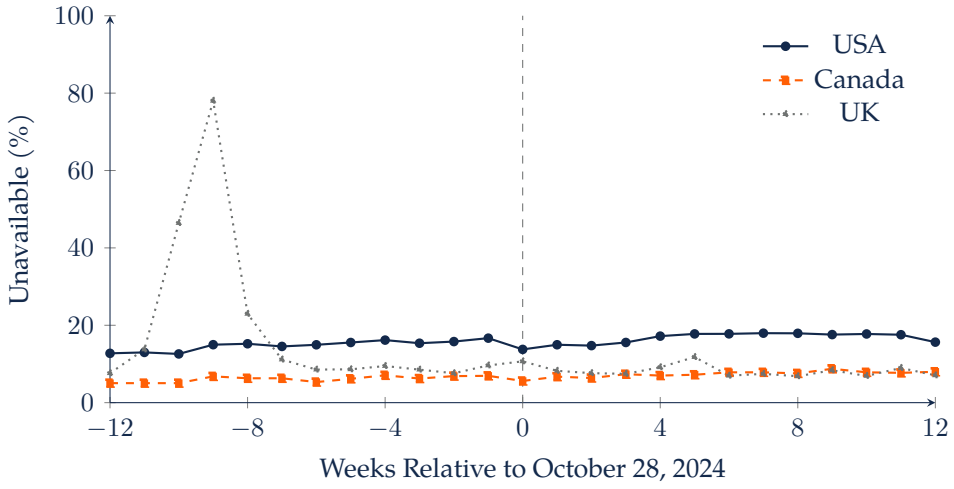


Figure 6.7. Weekly recorded unavailability varies sharply in the UK before the US policy date. The country paths expose the sensitivity of a pre/post contrast to the chosen comparison.

These data support an audit of a proposed policy evaluation. The positive contrasts do not establish that the exemption worsened availability. Stronger evidence would connect a stable set of

outlets to policy exposure, actual repair adoption, verified mechanical interruptions, and comparable collection times. Pre-treatment event coefficients can diagnose particular discrepancies, while a nonsignificant pre-trend test supplies no proof of the counterfactual assumption. Customer demand, service hours, and repair implementation would further distinguish an operational policy effect from a change in the ordering measure.

Python source: policy.py. From its directory, run `python policy.py`.

```
"""Calculate descriptive country contrasts around the October 2024 exemption."""
import json
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parent


def main():
    data = pd.read_csv(ROOT / "data/policy_window.csv")
    assert not data.duplicated(["country", "event_week"]).any()
    data["percent"] = 100 * data.n_broken / data.n_checked
    # These are sensitivity screens, not a claim of representative coverage.
    data = data[data.n_checked.ge(data.country.map({"USA": 2000, "UK": 500, "CA": 500}))]
    wide = data.pivot(index="event_week", columns="country", values="percent")
    common = wide[["USA", "UK", "CA"]].dropna()
    pre = common.loc[common.index < 0].mean()
    post = common.loc[common.index > 0].mean()  # omit the implementation week
    if len(common.loc[common.index < 0]) != 12 or len(common.loc[common.index > 0]) != 12:
        raise ValueError("Expected 12 common pre weeks and 12 common post weeks")
    change = post - pre
    table = pd.DataFrame({"pre_percent": pre, "post_percent": post,
                          "change_pp": change})
    contrasts = {c: float(change.USA - change[c]) for c in ["CA", "UK"]}
    results = ROOT / "results"
    results.mkdir(exist_ok=True)
    table.to_csv(results / "policy_means.csv", index_label="country")
    (results / "policy.json").write_text(json.dumps({
        "effective_date": "2024-10-28", "pre_weeks": 12, "post_weeks": 12,
        "omitted_week": 0, "weighting": "Equal weight per common weekly snapshot",
        "us_change_minus_comparator_change_pp": contrasts,
        "interpretation": "Descriptive contrasts; parallel counterfactual trends and stable measurement
        ↪  are not established."}, indent=2) + "\n")
    figure = r"""\begin{tikzpicture}
\begin{axis}[width=12.5cm,height=6.7cm,xmin=-12,xmax=12,ymin=0,ymax=100,
 xlabel={Weeks Relative to October 28, 2024},ylabel={Unavailable (\%)},
 xtick={-12,-8,-4,0,4,8,12},legend style={at={(0.98,0.98)},anchor=north east}]
"""
    for country, style in [("USA", "IlliniBlue,mark=*"),
                           ("CA", "IlliniOrange,dashed,mark=square*"),
                           ("UK", "Storm,dotted,mark=triangle*")]:
        points = " ".join(f"({i},{v:.6f})" for i, v in common[country].items())
```

```
        figure += f"\\addplot[{style},mark size=1.4pt] coordinates {{{points}}};\n"
    figure += r"\legend{USA,Canada,UK}" + "\n"
    figure += r"\draw[Storm,dashed] (axis cs:0,0)--(axis cs:0,100);" + "\n"
    figure += "\\end{axis}\n\\end{tikzpicture}\n"
    (ROOT.parents[1] / "06_Lecture_Notes/figures/M06_mcbroken_policy.tex").write_text(figure)
    print(table.to_string())
    print(json.dumps(contrasts, indent=2))


if __name__ == "__main__":
    main()
```

Chapter 7

Prediction, Machine Learning, and Marketing Actions

A retailer wants to anticipate next month's customer purchases and allocate limited service capacity. A model that predicts historical purchases accurately may still fail when deployed: its features may contain future information, its test set may reuse the same customers as training, or its objective may reward errors that are costly for the business. Prediction should therefore be designed around an explicit deployment situation.

7.1 Specify the Forecast Origin and Target

A prediction task needs a target variable, a forecast origin, a horizon, and an eligible population. For customer churn, define what counts as churn and when that status becomes observable. In a subscription business, cancellation may be explicit. In noncontractual retail, "no purchase for 90 days" is an operational label, not proof that a relationship has ended permanently.

Construct predictors using information available at the forecast origin. If predicting purchases during November, features might use transactions through October 31. November support contacts, returns, or campaign interactions are unavailable at that origin. A database may contain them when the analyst extracts the file, making leakage easy to overlook. Store feature timestamps or document their measurement windows.

The deployment population determines the validation design. Predicting future behavior for existing customers differs from predicting behavior for newly acquired customers. A random row split can place the same customer in training and test data, giving a favorable assessment for a task that actually requires generalization to new customers. A time split addresses future periods, while a customer-group split addresses new customers. Some tasks require both.

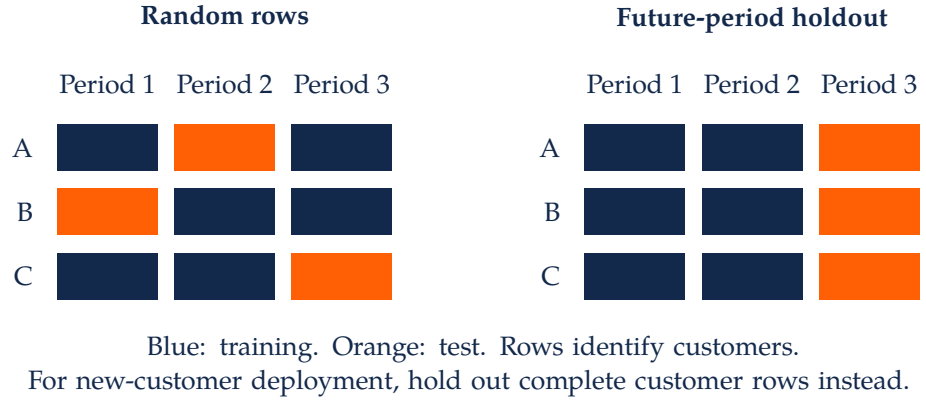


Figure 7.1. The validation split should reproduce the information available at deployment.

A random row split can reuse a customer's history in training and test data. Holding out complete customers evaluates generalization to new customers; holding out later periods evaluates future performance. The intended use determines which comparison is relevant.

7.2 Baselines, Loss Functions, and Regularization

A baseline should be operationally meaningful. For sales forecasting, compare with the previous period, the same period last year, or a simple customer-history average when those quantities are available. For classification, compare with a constant probability and a transparent logistic regression. A complex model's incremental value is easier to assess when these comparisons are reported on the same test observations.

A supervised learner minimizes a loss that quantifies prediction error. Squared error penalizes large deviations strongly; absolute error gives a different compromise under skewness. For a binary outcome, log loss is

$$L = -\frac{1}{n}\sum_{i=1}^{n}\{y_i \log \hat{p}_i + (1 - y_i)\log(1 - \hat{p}_i)\}.$$

Here y_i is the observed binary outcome and $\hat{p}_i$ is its predicted probability. Confident mistakes receive large penalties. The loss evaluates probability forecasts; it does not by itself encode the cost of a marketing action. Select a loss appropriate to the prediction task, then evaluate how the forecasts support the downstream decision.

Regularization constrains a flexible model to reduce overfitting. Ridge regression adds a squared-coefficient penalty, while lasso adds an absolute-coefficient penalty and can set some coefficients to zero. Standardize features appropriately when the penalty depends on coefficient scale, and fit that transformation using training data. The regularization strength is a tuning choice selected with validation data, not with the final test sample.

Bias and variance describe an important tradeoff. A rigid model may systematically miss a pattern; a very flexible model may fit noise specific to the training sample. More data can help, but only if it represents the deployment problem and adds relevant information. A million duplicated or highly dependent rows can provide less useful information than their count suggests.

7.3 Logistic Regression, Trees, and Ensembles

Logistic regression maps a linear index to a probability:

$$\hat{p}_i = \frac{1}{1 + \exp[-(\beta_0 + x_i^\top \beta)]}.$$

The coefficients describe changes in log odds under the model. A coefficient's effect on probability depends on the starting probability and other predictors. Predicted probabilities are often easier to communicate than odds ratios, especially when the business decision uses expected counts or contribution. Nonlinear features and interactions can extend the model while preserving a relatively transparent structure.

A decision tree partitions the feature space into regions using sequential splits. It can represent interactions without specifying them manually: a spending threshold might matter only among recent customers. Deep trees can fit small idiosyncratic groups, making pruning, minimum leaf sizes, and validation important. A split point near an observed value should not automatically be interpreted as a behavioral threshold.

Random forests average trees fitted with randomness in observations and candidate features, reducing instability relative to a single tree in many settings. Gradient boosting builds an ensemble sequentially to improve a chosen loss, with tuning choices such as tree depth, learning rate, and number of iterations. These methods can be effective on tabular data, but performance remains empirical. More complexity also increases the burden of checking leakage, calibration, computation, and reproducibility.

Neural networks learn layered representations and can be especially useful for text, images, or large complex datasets. Their flexibility does not eliminate the need for baselines or careful evaluation. In a modest customer table, a well-tuned simpler model may be competitive and easier to maintain. Choose the method based on task performance and practical constraints rather than the apparent modernity of the algorithm.

7.4 Validation, Cross-Validation, and Leakage

Use training data to fit parameters, validation data or cross-validation to select models and tuning choices, and a final test sample to estimate performance after those choices are fixed. Reusing the test sample repeatedly turns it into a validation sample, even if the file is still named "test." Report the procedure so the claimed holdout remains meaningful.

All learned preprocessing belongs inside the training process. This includes imputation, scaling, feature selection, vocabulary construction, dimensionality reduction, and target-based encodings. In cross-validation, fit each transformation on the training fold and apply it to that fold's validation observations. A pipeline makes the sequence explicit. The scikit-learn pitfalls guide documents implementation patterns for this separation.

Time-series and grouped data need adapted splits. Rolling-origin evaluation trains on earlier periods and predicts later ones, potentially repeating this across several origins. Grouped validation holds out complete customers, households, stores, or other relevant units. If a target window overlaps a training outcome window, a temporal gap may be necessary. Document the dates and units assigned to each split.

A useful audit asks when each feature became observable. In a marketing-call dataset, call duration can be useful after a call but unavailable when deciding whom to call. A model using that feature answers a different prediction problem. The feature's legitimacy follows the decision time, not whether a package accepts it or whether it improves accuracy.

7.5 Case Study: Accuracy Can Conceal a Useless Model

Suppose 5% of 1,000 customers make a purchase in the target window. A model predicting no purchase for everyone achieves 95% accuracy while identifying no purchasers. A different model

flags 100 customers, including 30 purchasers and 70 nonpurchasers. It has 30 true positives, 20 false negatives, 70 false positives, and 880 true negatives.

The second model's precision is 30%, recall is 60%, and accuracy is 91%. Its lower accuracy coexists with a potentially useful ranking for a limited-capacity task. Whether it improves profit depends on the action, costs, and whether the purchase probabilities under that action are the relevant quantities. Accuracy alone cannot make that decision.

ROC curves compare true-positive and false-positive rates across thresholds. Precision-recall curves focus on the tradeoff between finding positives and the composition of the flagged group, which can be informative when positives are rare. Both depend on the evaluation setting; precision is especially sensitive to prevalence. Lift at a specified contact fraction compares the selected group's outcome rate with the population rate, but observed lift is not automatically an incremental treatment effect.

Report metrics with their operational units. For a service team that can review only 100 cases per week, precision among the top 100 and the number of important cases missed may be more relevant than a global ranking metric. Use a holdout that reflects the weekly population and inspect uncertainty in the metric when counts are small.

7.6 Ranking and Calibration Answer Different Questions

A ranking model orders cases from more likely to less likely. A calibrated probability model assigns probabilities that correspond to observed frequencies in relevant groups. If customers assigned probability 0.20 purchase about 20% of the time in a suitable evaluation population, that group is calibrated. Calibration can vary across segments and over time even when overall performance looks satisfactory.

A strictly increasing transformation of scores preserves their ranking. Squaring probabilities between zero and one therefore leaves a ranking-based AUC unchanged, apart from practical handling of ties or finite precision, while generally changing calibration. A predicted probability of 0.50 becomes 0.25. If the original 0.50 group purchases half the time, the transformed probabilities understate its frequency.

For calibrated forecasts, predicted probabilities equal observed frequencies. After squaring an initially calibrated score p, the reported probability is $q = p^2$, while the observed frequency remains $p = \sqrt{q}$. Expected counts and profit calculations depend on the probability scale, so unchanged ranking performance does not guarantee unchanged decision quality.

Inspect calibration plots and proper scoring rules, such as log loss or the Brier score, on held-out data. Calibration methods themselves require fitting and should use data separate from the final evaluation. If the deployment population changes, recalibration may be necessary. A single global correction may be insufficient when the relationship changes differently across groups.

7.7 Thresholds Should Reflect the Decision

Consider a classification action in which a false positive costs C_{FP} and a false negative costs C_{FN}, with zero cost for correct classifications. For a calibrated probability p, predicting positive has

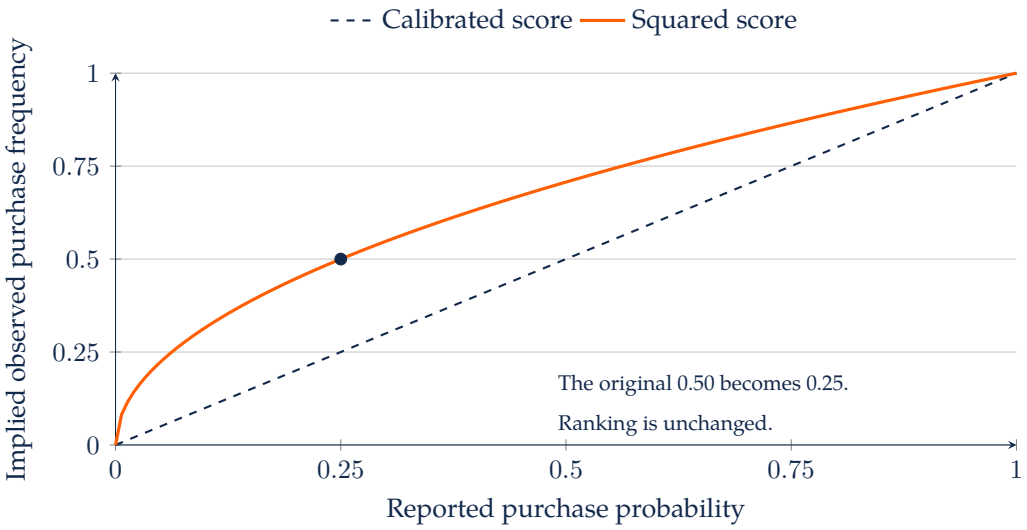


Figure 7.2. A monotonic score transformation preserves ranking while changing probability calibration.

expected cost $C_{FP}(1-p)$ and predicting negative has expected cost $C_{FN}p$. The cost-minimizing rule predicts positive when

$$p > \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

If an unnecessary review costs one unit and missing a true issue costs nine units, the threshold is 0.10 under this simplified loss. A threshold of 0.50 would encode a different cost balance. Capacity constraints, review effectiveness, and heterogeneous costs require a more detailed rule. The formula is useful because it makes the decision assumptions explicit.

A marketing intervention usually needs a causal comparison beyond this classification loss. For a coupon, let $p_i(1)$ and $p_i(0)$ be purchase probabilities with and without the coupon, m the contribution before the discount, d the discount paid on a treated purchase, and c the contact cost. A simple incremental expected contribution is

$$\Delta\pi_i = m\{p_i(1) - p_i(0)\} - d\,p_i(1) - c.$$

The discount is paid on all treated purchases in this setup, including purchases that would have happened anyway. The equation assumes one relevant purchase opportunity and a fixed discount and contribution. It can be extended to spending, multiple purchases, returns, and heterogeneous margins.

7.8 Case Study: Targeting the Most Responsive Customers Can Lose Money

Take $m = 40$, $d = 5$, and $c = 2$. Customer A has purchase probabilities 0.80 with the coupon and 0.78 without it. Customer B has probabilities 0.30 and 0.15. A has the higher treated response probability, but incremental contribution is \$-5.20 for A and \$2.50 for B.

Customer	Purchase With Coupon	Purchase Without Coupon	Incremental Contribution
A	0.80	0.78	-$5.20
B	0.30	0.15	$2.50



Figure 7.3. Baseline purchasing and discount expense can reverse a response-based targeting rule.

Incremental purchasing contributes $0.80 for A and $6.00 for B, while discount and contact costs are $6.00 and $3.50, respectively. These values follow from the assumed purchase probabilities. Estimating their difference from customer data requires treatment variation and an identification strategy. Uplift or heterogeneous-treatment-effect models address that causal target under their conditions; they are not ordinary response models with a different name.

Evaluate a targeting policy on data that support its value comparison. A randomized holdout or a carefully designed policy evaluation can estimate incremental contribution for the selected group. Estimating a model and reporting its value on the same observations used to choose the policy can overstate performance. Budget constraints also matter: a customer with positive expected value may still be excluded when another customer offers greater value per unit of scarce capacity.

7.9 Interpretation, Fairness, and Deployment

Feature importance describes a relationship between a fitted model and its inputs under a chosen method. Permutation importance measures the change in predictive performance after disrupting a feature, with complications when predictors are correlated. Partial-dependence plots average predictions over a reference distribution and can involve unrealistic combinations of correlated features. Local explanation methods describe the fitted prediction under their conventions. These tools do not automatically identify causal effects of changing a feature.

Evaluate performance across groups relevant to the application and examine how errors translate into service or opportunity. A model can be accurate on average while performing poorly for new customers, small stores, or a language group with sparse training data. Protected or sensitive

attributes and proxies require appropriate institutional and legal review for the intended use. The consequences of errors depend on the decision and the affected customers.

Deployment requires monitoring the input distribution, missingness, outcome frequency, calibration, and decision value. A change in model accuracy can reflect changing customer behavior, instrumentation, or policy. Record versions and retain a reproducible baseline. If an intervention changes the behavior that generates future training data, the model's environment becomes endogenous to its use, because its recommendations change subsequent exposure and behavior.

7.10 Recommendation Models and the Candidate Set

A recommendation decision chooses a small set of products for a particular customer at a particular time. Candidate generation first identifies eligible items, scoring predicts their relevance, and ranking selects their order. A final adjustment may impose stock availability, repetition limits, category variety, or a customer-stated goal. An accurate score cannot recover a useful product that candidate generation has already excluded. Evaluation should therefore record both the eligible set and the final list.

Content-based models use item attributes and the customer's history. A grocery system might represent foods by ingredients, category, price, or an embedding of their descriptions. Collaborative models instead use patterns across customer-item interactions. Matrix factorization represents a customer and an item with short vectors and scores their compatibility through an inner product:

$$\hat{r}_{uj} = \mu + b_u + b_j + p_u^\top q_j.$$

Here u indexes a customer, j an item, μ is an overall level, b_u and b_j are customer and item offsets, and the vectors p_u and q_j summarize interaction patterns in a common latent space. The score's units depend on the fitted target. With squared-error training on star ratings, it approximates rating units. A ranking score fitted to implicit interactions has no automatic probability interpretation. Popularity offsets capture broad appeal, while the inner product permits customers with different histories to prefer different products.

Ratings are explicit feedback. Clicks, views, and purchases are implicit feedback whose meaning depends on exposure and opportunity. A missing interaction is not a verified rejection. Treating every missing customer-item pair as a negative can encode the previous recommender's display policy. Negative sampling changes the training distribution, so the sampling rule and any probability correction belong in the analysis. Logging impressions and eligible alternatives makes these distinctions more tractable.

New customers and new products create a cold-start problem. Content features, initial preference questions, and popularity baselines can provide information before interaction histories become informative. A hybrid model combines these sources. Sequential models additionally condition on the order of past events, allowing a recent purchase to change what is useful next. The validation sample must preserve that order and exclude later purchases, ratings, and catalog information from the forecast origin.

7.11 Ranking Metrics Depend on Position and Available Labels

Precision at K is the fraction of the first K recommendations labeled relevant. Recall at K is the fraction of all labeled relevant items retrieved in that list. Hit rate records whether at least one relevant item appears. These metrics use different denominators and can disagree. A hit rate of one can coexist with low precision when a long list contains one relevant item.

Normalized discounted cumulative gain gives greater weight to relevant items near the top. For nonnegative relevance labels $r_1, \ldots, r_K$, one convention is

$$\text{DCG@}K = \sum_{k=1}^{K} \frac{2^{r_k} - 1}{\log_2(k+1)}, \qquad \text{NDCG@}K = \frac{\text{DCG@}K}{\text{IDCG@}K}.$$

The position discount reflects the greater value assigned to placing useful items earlier. The ideal DCG uses the best ordering of the labeled eligible items at the same cutoff. Report the gain convention and the treatment of customers with no relevant labels, for whom this ratio has a zero denominator. Averaging only over customers with positive labels changes the evaluation population.

Consider a synthetic three-item candidate set in which A is irrelevant and B and C are relevant. The lists A, B, C and B, C, A both have precision $2/3$ and recall one. The first list has DCG $1/\log_2(3) + 1/\log_2(4) \approx 1.131$. The second has DCG $1 + 1/\log_2(3) \approx 1.631$, which is also the ideal value. Their NDCG values are approximately 0.693 and 1.000. The same number of relevant items can produce different ranking quality because their positions differ.

Historical labels describe items observed under earlier exposure and consumption decisions. High NDCG on those labels can coexist with poor discovery of items rarely displayed. Evaluate a common candidate set, retain customer and time boundaries, compare with a popularity baseline, and report performance for customers and products with little history. The policy-value calculation in M6, Section 6.7, addresses selected exposure under different assumptions from an ordinary held-out ranking calculation. Neither calculation alone establishes customer satisfaction or the value of changing the deployed system.

7.12 Case Study: Food Recommendations, Variety, and Customer Goals

A food app can predict what a customer usually eats while learning little about what would help that customer pursue a stated goal. The distinction changes the training sample, the model inputs, and the relevant evaluation outcome. Ling, Nie, Nielsen, Knäuper, Yang, and Dubé (2022) develop a recommendation system using daily food histories and information about calories and body-mass-index groups. Their 2016 app sample restricts users to ages 18–65 with substantial logging histories and removes sparse daily records. Table 2 reports 7,457 users in the resulting model sample.

The model represents both the foods within a day and the sequence of days. Separate models learn from general histories, particular body-mass-index groups, and periods associated with the relevant weight outcome. Their predictions can be combined. The last basket supplies the test observation, the preceding basket supplies validation, and earlier baskets supply training. This de-

sign evaluates whether the model predicts foods eaten during periods associated with the intended outcome. It does not randomize customers to receive the recommendations, so the reported ranking improvements do not identify a causal effect on weight.

Selecting successful periods can help define a target pattern while also selecting motivation, physical activity, reporting behavior, or other determinants of the outcome. A customer who follows a similar food pattern may differ on those other dimensions. For deployment, the goal and all required inputs must be available when recommendations are made. A later realized weight outcome can define a retrospective evaluation stratum; it cannot serve as a known input for a new customer unless supplied as an explicit scenario rather than an observed future fact.

Nielsen, Yang, Dubé, Knäuper, Ling, and Nie (2022) examine variety in food logs and develop FOODVAR. Their empirical analysis relates daily variety to subsequent changes in calories and the composition of food consumption. Variety measures include item counts and a reverse-coded concentration index based on calorie shares. Two days with the same item count can therefore differ in variety if one is dominated by a single food. The reported relationships are conditional associations, and the denominators and sample restrictions differ across the total-calorie, vegetable, and snack analyses.

FOODVAR incorporates a variety-change indicator into sequential food recommendation. Its evaluation uses 30 days of history, seven subsequent validation days, and a final test day, with the indicator representing the intended variety direction. Table 5 compares hit-rate and NDCG measures against the baseline across weight-outcome groups. The results establish predictive differences within that evaluation design. A prospective test would elicit the variety goal before displaying recommendations and measure subsequent behavior. The synthetic category-bonus example in M8, Section 8.2, is a simpler list-selection exercise with a different algorithm.

7.12.1 A Goal Weight Can Change the Preferred Item Abruptly

Suppose three products have preference scores and goal-compatibility scores on a common illustrative scale. A scores $(0.90, 0.10)$, B scores $(0.75, 0.60)$, and C scores $(0.60, 0.90)$. The scores are imposed inputs, not estimated purchase probabilities or health effects. A weighted ranking rule is

$$s_j(\alpha) = (1-\alpha)p_j + \alpha g_j, \qquad 0 \le \alpha \le 1,$$

where p_j measures preference compatibility, g_j measures compatibility with the stated goal, and α determines their relative importance. At $\alpha = 0$, A ranks first. At $\alpha = 0.25$, the scores are 0.700, 0.7125, and 0.675, so B ranks first. At $\alpha = 0.50$, C ranks first with 0.750. Solving the pairwise score equalities gives the optimal switches at $\alpha = 3/13$ and $\alpha = 1/3$.

Every score changes continuously, yet the chosen item changes discretely. A small adjustment near a crossing can therefore replace the displayed recommendation. B is the preferred compromise only over a narrow weight interval. A useful audit varies the weight and inspects which customers experience large changes in their lists. The chosen scales and weights define the tradeoff; combining scores does not itself establish that a goal will be achieved.

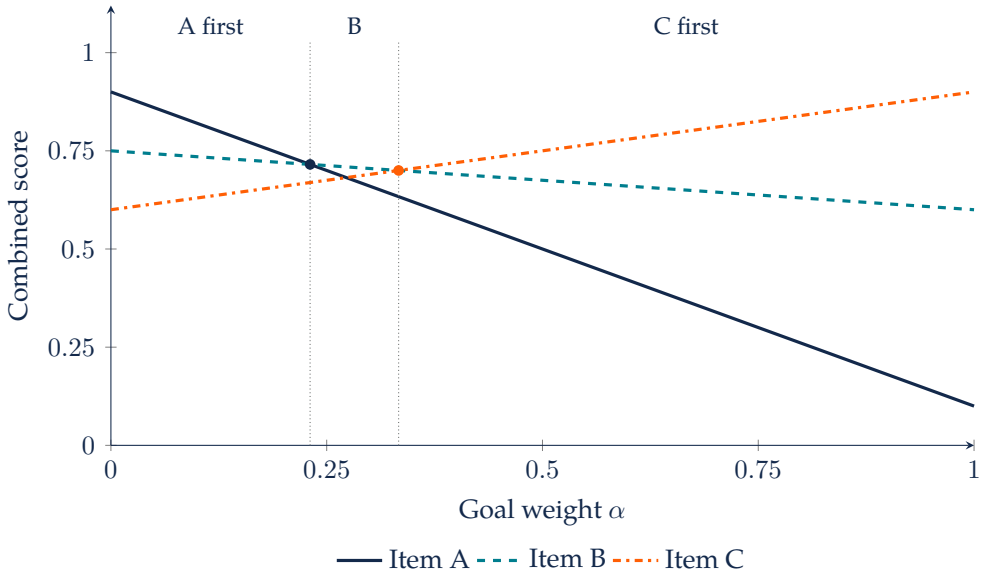


Figure 7.4. Smooth changes in the objective weight produce discrete changes in the top-ranked item.

7.13 LLM Explanations Need Factual and Behavioral Evaluation

A shopping assistant may generate a persuasive reason for recommending a product after the product has already been selected. That text can be useful, but its fluency does not establish that it describes the ranking model's actual reason. Three questions remain distinct: whether the explanation contains correct product facts, whether it corresponds to the system's decision process, and whether it helps the customer make a satisfactory choice.

Wang, Li, and Chen's RecPIE study, in its May 2026 version, jointly trains recommendation prediction and natural-language explanation. Prediction performance informs explanation generation, and explanation representations feed into subsequent predictions. The empirical setting includes Google Maps point-of-interest recommendations and a separate human assessment of explanations. Reported improvements in ranking and explanation preference concern those evaluations. They do not by themselves estimate the effect of deploying the system on purchases, retention, or customer welfare.

Qin, Huang, Deng, Lei, and Chua (2024) study credibility in conversational recommendation. Their PC-CRS approach uses credibility-aware persuasive strategies and subsequent revision of explanations. The comparison motivates separate evaluation of persuasive success and support for factual claims. An assistant can increase acceptance by asserting an appealing attribute that the product does not possess. Acceptance alone would reward that error.

Consider a synthetic catalog with prices, delivery times, and verified product attributes. A customer asks for delivery within two days at a price below $30. First enforce those eligibility constraints using the catalog. Then rank eligible products and generate explanations that refer to specific supported attributes. "Arrives tomorrow" is unsupported when the recorded delivery window is three to five days, even if the rest of the explanation is relevant. "You value quiet operation" is unsupported when the customer has expressed no such preference and no permitted history supplies it.

An audit should inspect factual support, constraint violations, ranking performance, and customer outcomes separately. Remove or alter a cited attribute and examine whether the explanation and ranking change as claimed; this provides a behavioral diagnostic of correspondence, with scope limited to the perturbations tested. Hold out customers, products, or time periods according to deployment. Keep the catalog version and the evidence used for each explanation so the claim can be checked after product information changes.

7.14 AI-Assisted Workflow, Practice, and Readings

Ask an assistant to create an evaluation plan before choosing a model. Specify forecast origin, horizon, unit, grouping, baseline, loss, and final holdout. Require a feature-availability table and a pipeline that fits transformations only on training data. Have it produce both a metric table and an explanation of the operational decision each metric supports.

For practice, verify the 91% accuracy and 60% recall in the rare-purchase example. Square a small set of predicted probabilities and confirm that their order is unchanged while expected purchase counts change. Recalculate the coupon case with a $1 discount and explain which customer becomes more attractive. Finally, identify why a model trained on all customers' latest outcomes cannot honestly claim to predict those same outcomes out of sample.

For a recommender exercise, have an assistant build a popularity baseline, a simple content score, and the stated goal-weight rule on synthetic items. Require a chronological split, a documented candidate set, and a report of excluded or unavailable products. Verify the two NDCG calculations by hand, then enumerate the goal-weight switches independently. Ask why an unseen item was labeled irrelevant and require the code to distinguish an observed rejection from absent exposure. A working implementation of the simple score is not a replication of FOODVAR, HHFR, or RecPIE.

For an explanation exercise, give the assistant a small catalog and a customer's explicit constraints. Request a recommendation with an attribute-level source for each factual claim. Include one unavailable attribute and one product that violates a constraint. Check whether the system acknowledges missing information and excludes the ineligible product. Score factual support separately from how persuasive the explanation sounds.

An Introduction to Statistical Learning develops the central learning methods. The scikit-learn evaluation guide provides metric definitions and implementation references. Hagen and colleagues (2020) connect machine-learning tools to behavioral marketing questions.

7.15 mHealth: Predicting a Returning User's Next Basket

The manager needs a recommendation baseline that uses only information available when suggestions are chosen. Global category popularity is easy to implement. A personal frequency rule can exploit repeated food choices without estimating a complex model. The comparison should preserve complete baskets and evaluate both rules on the same users, candidate set, and future outcomes.

A basket is the set of distinct source categories recorded by a user on a day; missing category labels are excluded. Freeze the candidate set at the 312 categories observed before July 1. For each forecast origin, require at least 30 prior observed basket days and select the first observed basket during the following month as the target. Validation starts July 1 and testing starts September 1. Every score uses dates strictly before its forecast origin. The test concerns returning users who subsequently log a basket, conditional on that observation; it does not evaluate people who never return.

Personal frequency is the fraction of a user's observed days in a specified history window containing a category. It does not divide by all calendar days. A tiny popularity term resolves most frequency ties; exact remaining ties use alphabetical order. If a user has no observed day in a short window, personal frequencies are zero and the popularity term supplies the order. Global popularity uses the corresponding fraction across all prior observed user-days. Scores produce a list of ten distinct categories.

Choose among 30-day, 90-day, and all-prior-history windows using July mean NDCG at ten. Their validation scores are 0.5666, 0.5661, and 0.5501, respectively, among 297 evaluated users. The 30-day rule wins by a very small margin over 90 days. The code preserves the specified tie order and carries the chosen window to September. A further claim that 30 days is generally superior would need repeated or independent validation.

September Rule	Evaluated Users	Recall at 10	NDCG at 10
Global Popularity	299	0.2799	0.3270
Personal 30-Day Frequency	299	0.4759	0.5862

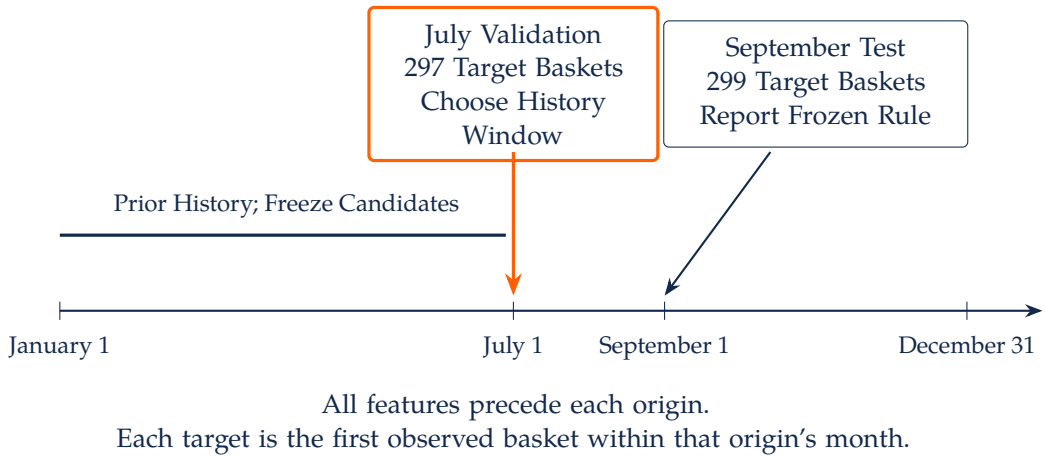


Figure 7.5. The candidate universe is fixed before validation. The September rule can use updated pre-September histories, while its chosen history-window length remains fixed from July.

There are 362 eligible users at the September origin, but only 299 have a target basket in that month. Reporting both counts exposes the conditioning introduced by the target definition. All observed target categories in this evaluation are covered by the frozen candidate set. The implementation nevertheless retains all target categories in the recall denominator and ideal DCG, so an unsupported category would count against performance.

The personal rule recovers more recorded categories than the popularity baseline. This supports personalization for predicting the observed basket under the historical environment. It does not show that displaying those suggestions changes choices, improves nutrition, or generates incremental app value. Logging may omit foods, and the basket can occur several days after the fixed origin. A same-day or event-time recommender would need a corresponding evaluation design.

7.15.1 Histories, Candidate Sets, and Ranking Metrics

`ranking.py` defines the common evaluation so that M8 changes the ranking objective while holding the target baskets fixed. Recall divides the number of recovered categories by the target basket size. NDCG discounts relevant categories by position and divides by the best possible ranking for that target. Each evaluated user contributes one metric value, and the final result averages users equally.

Python source: ranking.py. From its directory, run `python ranking.py`.

```
"""Whole-basket chronological evaluation with a frozen category universe."""

import numpy as np
import pandas as pd


def candidate_categories(baskets):
    prior = baskets.loc[baskets.date.lt("2016-07-01")]
    return sorted({category for basket in prior.basket for category in basket})


def evaluation_cases(baskets, cutoff, candidates):
    cutoff = pd.Timestamp(cutoff)
    prior = baskets.loc[baskets.date.lt(cutoff)]
    counts = prior.groupby("Userid").size()
    eligible = counts[counts >= 30].index
    end = cutoff + pd.offsets.MonthBegin(1)
    target = baskets.loc[baskets.date.ge(cutoff) & baskets.date.lt(end)]
    target = target.loc[target.Userid.isin(eligible)].sort_values(["Userid", "date"])
    target = target.drop_duplicates("Userid")
    histories = dict(tuple(prior.groupby("Userid")))
    assert prior.date.max() < cutoff
    global_counts = (
        prior.basket.explode().value_counts().reindex(candidates, fill_value=0)
    )
    popularity = global_counts.to_numpy(dtype=float) / len(prior)
    cases = [(histories[row.Userid], set(row.basket)) for row in target.itertuples()]
    return cases, popularity, len(eligible)


def personal_frequency(history, cutoff, candidates, window):
    if window is not None:
        start = pd.Timestamp(cutoff) - pd.Timedelta(days=window)
        history = history.loc[history.date.ge(start)]
```

```
    counts = history.basket.explode().value_counts().reindex(candidates, fill_value=0)
    return counts.to_numpy(dtype=float) / max(1, len(history))


def rank_categories(scores, candidates, k=10):
    # Stable sorting resolves exact score ties using alphabetically sorted candidates.
    positions = np.argsort(-scores, kind="stable")[:k]
    return [candidates[index] for index in positions]


def ranking_metrics(ranking, target):
    relevance = np.array([category in target for category in ranking], dtype=float)
    discounts = 1 / np.log2(np.arange(2, len(ranking) + 2))
    ideal = discounts[: min(len(target), len(ranking))].sum()
    return float(relevance.sum() / len(target)), float(
        (relevance * discounts).sum() / ideal
    )


def evaluate(
    baskets, cutoff, candidates, window=None, popularity_only=False, novelty_weight=0
):
    cases, popularity, eligible = evaluation_cases(baskets, cutoff, candidates)
    rows = []
    for history, target in cases:
        frequency = personal_frequency(history, cutoff, candidates, window)
        recent = personal_frequency(history, cutoff, candidates, 30)
        unseen = recent == 0
        scores = popularity if popularity_only else frequency + 1e-6 * popularity
        ranking = rank_categories(scores + novelty_weight * unseen, candidates)
        recall, ndcg = ranking_metrics(ranking, target)
        recent_set = {candidates[j] for j in np.flatnonzero(recent)}
        novelty = sum(category not in recent_set for category in ranking) / len(ranking)
        coverage = len(target & set(candidates)) / len(target)
        rows.append([recall, ndcg, novelty, coverage])
    means = np.mean(rows, axis=0)
    return {
        "eligible_users": eligible,
        "evaluated_users": len(cases),
        "recall_at_10": float(means[0]),
        "ndcg_at_10": float(means[1]),
        "novelty_at_10": float(means[2]),
        "target_candidate_coverage": float(means[3]),
    }
```

7.15.2 Validation Before Testing

The execution order matters. Fit the history-window choice using July, freeze it, and report September performance without changing the rule after seeing the test. The small metric counterexample confirms that placing both relevant categories first yields NDCG of one, while placing an irrele-

vant category first lowers it. A separate test supplies a category that occurs only in the future and verifies that it never enters personal history.

Python source: m07.py. From its directory, run `python m07.py`.

```
"""Choose history length in July and evaluate the frozen rule in September."""

import json
from common import load_inputs, save_result
from ranking import candidate_categories, evaluate, ranking_metrics

food, daily, baskets = load_inputs()
candidates = candidate_categories(baskets)
windows = {"30_days": 30, "90_days": 90, "all_prior_days": None}
validation = {
    name: evaluate(baskets, "2016-07-01", candidates, window)
    for name, window in windows.items()
}
# Ties retain the first listed specification; September plays no role in selection.
chosen = max(validation, key=lambda name: validation[name]["ndcg_at_10"])
test = {
    "popularity": evaluate(baskets, "2016-09-01", candidates, popularity_only=True),
    "personal_frequency": evaluate(baskets, "2016-09-01", candidates, windows[chosen]),
}
assert ranking_metrics(["B", "C", "A"], {"B", "C"}) == (1.0, 1.0)
assert ranking_metrics(["A", "B", "C"], {"B", "C"})[1] < 1
result = {
    "candidate_categories": len(candidates),
    "validation": validation,
    "chosen_window": chosen,
    "test": test,
}
save_result("M07", result)
print(json.dumps(result, indent=2))
```

7.16 Python Calculations for the Synthetic Cases

The inputs below are imposed teaching values. Running the program creates the labeled synthetic CSV files and prints the corresponding calculations. They are separate from the mHealth estimates. Keep the script beside `example_helpers.py` in `03_Data_and_Code/Lecture_Examples`.

Python source: m07_synthetic.py. From its directory, run `python m07_synthetic.py`.

```
"""Synthetic numerical examples for M7; run from any working directory."""

import itertools
import json
import math
from example_helpers import write_csv, ols_slope, shares
```

```
r = {}
r["M07"] = {
    "precision": 30 / 100,
    "recall": 30 / 50,
    "accuracy": (30 + 880) / 1000,
    "all_negative_accuracy": 950 / 1000,
    "cost_threshold": 1 / (1 + 9),
    "coupon_A": 40 * (0.8 - 0.78) - 5 * 0.8 - 2,
    "coupon_B": 40 * (0.3 - 0.15) - 5 * 0.3 - 2,
}
write_csv(
    "M07_coupon_synthetic.csv",
    [
        "customer",
        "purchase_with_coupon",
        "purchase_without_coupon",
        "margin_dollars",
        "discount_dollars",
        "contact_cost_dollars",
    ],
    [("A", 0.8, 0.78, 40, 5, 2), ("B", 0.3, 0.15, 40, 5, 2)],
)

rankings = {
    "ABC": [("A", 0), ("B", 1), ("C", 1)],
    "BCA": [("B", 1), ("C", 1), ("A", 0)],
}


def dcg(labels):
    return sum(
        (2**label - 1) / math.log2(rank + 1)
        for rank, label in enumerate(labels, start=1)
    )


ideal = dcg([1, 1, 0])
r["M07"]["rankings"] = {
    name: {
        "precision_at_3": sum(v for _, v in vals) / 3,
        "recall_at_3": sum(v for _, v in vals) / 2,
        "dcg_at_3": dcg([v for _, v in vals]),
        "ndcg_at_3": dcg([v for _, v in vals]) / ideal,
    }
    for name, vals in rankings.items()
}
write_csv(
    "M07_rankings_synthetic.csv",
    ["ranking", "position", "item", "binary_relevance"],
    [
        (name, pos, item, rel)
```

```
        for name, vals in rankings.items()
        for pos, (item, rel) in enumerate(vals, start=1)
    ],
)
assert math.isclose(r["M07"]["rankings"]["BCA"]["ndcg_at_3"], 1)
assert 0.693 < r["M07"]["rankings"]["ABC"]["ndcg_at_3"] < 0.694

goal_items = {"A": (0.9, 0.1), "B": (0.75, 0.6), "C": (0.6, 0.9)}
goal_score = (
    lambda item, alpha: (1 - alpha) * goal_items[item][0] + alpha * goal_items[item][1]
)
r["M07"]["goal_blend"] = {
    "scores": {
        str(alpha): {item: goal_score(item, alpha) for item in goal_items}
        for alpha in [0, 0.25, 0.5, 1]
    },
    "optimal_switches": [3 / 13, 1 / 3],
}
write_csv(
    "M07_goals_synthetic.csv",
    ["item", "preference_score", "goal_score"],
    [(item, *scores) for item, scores in goal_items.items()],
)
assert math.isclose(goal_score("A", 3 / 13), goal_score("B", 3 / 13))
assert math.isclose(goal_score("B", 1 / 3), goal_score("C", 1 / 3))
for i in range(1001):
    alpha = i / 1000
    expected = "A" if alpha < 3 / 13 else ("B" if alpha < 1 / 3 else "C")
    assert max(goal_items, key=lambda j: goal_score(j, alpha)) == expected
print(json.dumps(r, indent=2))
```

7.17 mHealth: Basket Associations and Future Validation

A recommendation manager wants to identify categories that commonly appear together in a recorded day. An association rule can generate candidates for a basket-completion feature. Its commercial value still depends on when a suggestion is shown, whether the customer has already recorded the item, and whether the suggestion changes behavior. A high co-occurrence rate alone does not establish incremental demand.

Treat each observed user-day with at least one classified category as a basket. Deduplicate categories within the day, so repeated servings contribute one presence indicator. For categories A and B, define

$$\text{support}(A, B) = P(A \cap B), \qquad \text{confidence}(A \to B) = P(B \mid A), \qquad \text{lift}(A \to B) = \frac{P(B \mid A)}{P(B)}.$$

Probabilities are empirical fractions of recorded baskets. Confidence is directional, whereas lift is symmetric for a two-category pair. A popular category can have high confidence without an

unusually strong association. Lift compares the conditional rate with the marginal rate and therefore supplies the necessary benchmark. The label "confidence" here is a conditional frequency, separate from a statistical confidence interval.

Use January–June data to select the 20 most frequent categories. Among their pairs, require at least 100 joint baskets and 30 distinct supporting users. Rank eligible pairs by training lift and fix the largest before examining July–August. These prespecified restrictions avoid selecting a pair supported by only a few repeated entries, but they do not make users' repeated baskets independent. The training sample has 40,727 baskets from 358 users. The selected pair is oil and tomato, supported jointly by 192 users.

Metric	January–June	July–August
Recorded Baskets	40,727	12,956
Baskets With Oil	7,283	2,174
Baskets With Tomato	4,708	1,930
Baskets With Both	1,985	658
Tomato Given Oil	27.26%	30.27%
Marginal Tomato Rate	11.56%	14.90%
Lift	2.358	2.032

The later evaluation includes observed baskets from the previously known users; 334 of them contribute baskets in July–August. Users with no later basket supply no basket-level outcome. In the table, conditional tomato frequency rises while lift falls because tomato also becomes more common overall. Judging only the conditional frequency would miss that change in the comparison benchmark.

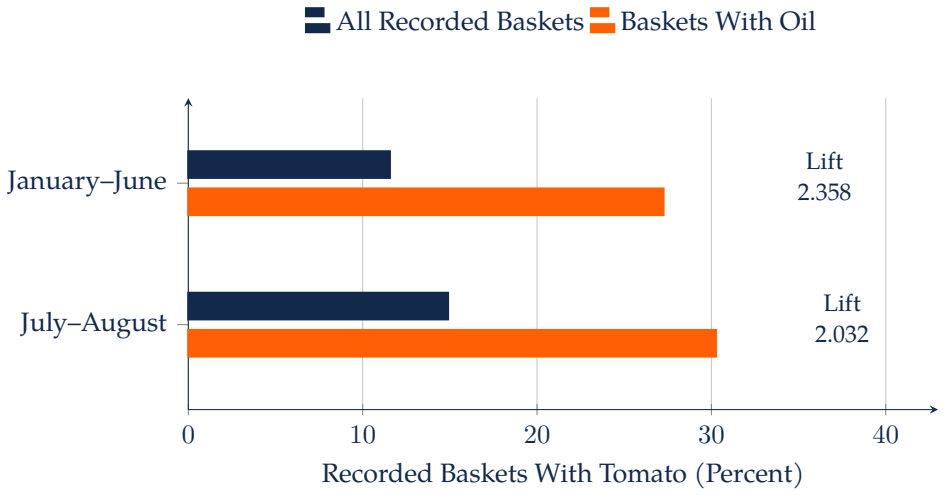


Figure 7.6. Conditional tomato frequency rises in the later period, but the marginal tomato rate rises proportionally more, reducing lift.

A 500-draw bootstrap resamples the 358 training users, carrying each user's complete later basket history into the draw, including zero later baskets. The percentile interval for later lift is $[1.762, 2.411]$. It describes sampling variation for the already selected pair, with dependence within each user preserved. It does not address common calendar shocks, uncertainty over the category-

selection procedure, or the effect of displaying a recommendation. The recorded category names also do not establish nutritional complementarity.

The mlxtend association-rule guide defines support, confidence, and lift and provides scalable itemset-mining tools. The code below enumerates pairs from a bounded candidate set so every numerator is inspectable. Its results can be compared with that library after the same basket and candidate definitions are imposed.

The implicit recommendation examples and Surprise examples address different feedback structures. Food logging supplies observed occurrences; missing entries do not provide explicit dislike ratings. For next-basket prediction, retain the chronological evaluation and known candidate set from Section 7.15. For within-basket completion, the source would additionally need an information rule specifying which recorded categories are visible when a suggestion is made.

Python source: association_rules.py. From its directory, run `python association_rules.py`.

```
"""Select a category pair in January-June; evaluate it in July-August."""

from itertools import combinations
import json
import numpy as np
import pandas as pd
from common import SEED, load_food, make_baskets, save_result


def metrics(n, a, b, both):
    if min(n, a, b) <= 0:
        raise ValueError("Positive basket and marginal counts are required")
    return {
        "baskets": int(n),
        "a_baskets": int(a),
        "b_baskets": int(b),
        "joint_baskets": int(both),
        "support": both / n,
        "confidence_a_to_b": both / a,
        "baseline_b": b / n,
        "lift": both * n / (a * b),
    }


def presence(baskets, categories):
    return np.array(
        [[c in basket for c in categories] for basket in baskets.basket], dtype=np.int64
    )


if __name__ == "__main__":
    baskets = make_baskets(load_food())
    train = baskets.loc[baskets.date < "2016-07-01"]
    frequency = train.explode("basket").groupby("basket").size()
    categories = sorted(
        frequency.sort_values(ascending=False, kind="stable").head(20).index
```

```
)
matrix = presence(train, categories)
joint, marginal = matrix.T @ matrix, matrix.sum(axis=0)
rules = []
for a, b in combinations(range(len(categories)), 2):
    mask = (matrix[:, a] * matrix[:, b]).astype(bool)
    supporting_users = train.loc[mask, "Userid"].nunique()
    if joint[a, b] >= 100 and supporting_users >= 30:
        rule = metrics(len(train), marginal[a], marginal[b], joint[a, b])
        rules.append(
            {
                "a": categories[a],
                "b": categories[b],
                "supporting_users": int(supporting_users),
                **rule,
            }
        )
rules.sort(key=lambda r: (-r["lift"], r["a"], r["b"]))
selected = rules[0]
users = sorted(train.Userid.unique())
future = baskets.loc[
    baskets.Userid.isin(users) & baskets.date.between("2016-07-01", "2016-08-31")
].copy()
binary = presence(future, [selected["a"], selected["b"]])
future["a"], future["b"] = binary[:, 0], binary[:, 1]
future["both"], future["n"] = binary.prod(axis=1), 1
counts = (
    future.groupby("Userid")[["n", "a", "b", "both"]]
    .sum()
    .reindex(users, fill_value=0)
    .to_numpy()
)
evaluation = metrics(*counts.sum(axis=0))
rng = np.random.default_rng(SEED)
lifts = []
for _ in range(500):
    total = counts[rng.integers(0, len(users), len(users))].sum(axis=0)
    if min(total[:3]) > 0:
        lifts.append(metrics(*total)["lift"])
evaluation["user_bootstrap_ci95"] = np.quantile(lifts, [0.025, 0.975]).tolist()
evaluation["bootstrap_valid_draws"] = len(lifts)
result = {
    "candidate_categories": len(categories),
    "eligible_pairs": len(rules),
    "training_users": len(users),
    "evaluation_users": int(future.Userid.nunique()),
    "selected_training_pair": selected,
    "july_august": evaluation,
}
save_result("association_rules", result)
print(json.dumps(result, indent=2))
```

7.18 McDonald's: Forecasting Availability With Missing Future Labels

A service team with limited time can prioritize verification calls using recent order availability. The forecast origin is the selected snapshot on date $t-1$; the target is the same address key's status in the selected snapshot on the next calendar date t. The key must be observed at the origin. For this exercise, an eligible origin has at least 2,000 US records in its snapshot, taken within three hours of 18:00 UTC. A scored target must meet the same conditions and contain the same key. This is a forecast for the next sampled observation. It does not measure whether the outlet experiences any interruption during the entire intervening day.

Section 2.16's preparation program preserves eligible origins whose target is absent. Those records lack a label. It also retains the original address keys, so address changes can remove a match. The resulting forecast population consists of observed, linkable records under the stated timing and coverage rules. A larger rollout would require checking performance and outcome observation beyond this population.

7.18.1 Compare Simple Forecasts on Later Dates

Fix August 1, 2026, as the first test target date. Estimate probabilities using earlier target dates only. The training mean assigns the same estimated probability to every origin. A second forecast conditions on whether the previous snapshot reported unavailability. Its estimate for previous status s is

$$\widehat{q}_s = \frac{\sum_{t<T}\sum_i A_{it}\mathbf{1}\{U_{i,t-1}=s\}U_{it}}{\sum_{t<T}\sum_i A_{it}\mathbf{1}\{U_{i,t-1}=s\}},$$

where T is the fixed cutoff and A_{it} indicates a usable observed pair under the stated rules. The denominator counts training pairs in status group s. This is an empirical conditional-frequency model, equivalent to a saturated probability model with one binary predictor. No hyperparameter is chosen using test outcomes. The benchmark that always predicts availability provides a further check on the usefulness of accuracy.

The preparation retains 1,484,356 labeled training pairs. In the later evaluation period, 580,473 of 801,841 eligible origins have usable labels, giving 72.39% label coverage. Label coverage is approximately 72.40% for previously available records and 72.38% for previously unavailable records. Similar coverage across these two groups does not establish that other sources of missingness are harmless.

Forecast	Test Brier Score	Test Accuracy at a 0.5 Cutoff
Always available	0.1465	85.35%
Training mean	0.1253	85.35%
Previous status	0.0851	89.16%

The Brier score averages $(U-\widehat{q})^2$ across labeled test pairs; smaller values indicate more accurate probability forecasts. The training-mean forecast improves probability error while making the same binary classification as the always-available rule. Conditioning on previous status improves

both measures. These results concern later observations for existing address keys; they do not establish performance at newly entering restaurants. Repeated keys and common dates also make individual rows dependent, so the large pair count should not be treated as an independent-sample precision calculation.

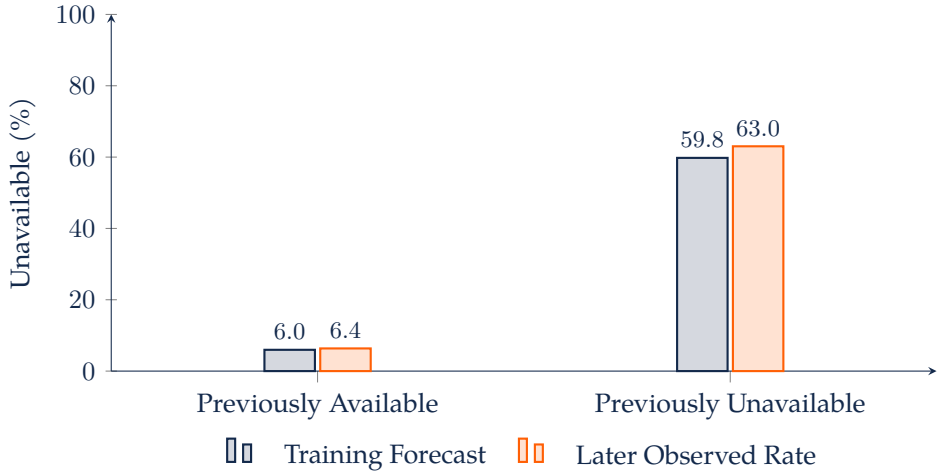


Figure 7.7. Training probabilities and later observed unavailability are much higher after a previously unavailable observation. Later rates exceed both training forecasts, making calibration drift visible.

The conditional model predicts 5.97% after an available observation and 59.78% after an unavailable observation. The corresponding later frequencies are 6.35% and 63.03%. Recent status is informative, while both groups exhibit higher later frequencies. A manager can use this ranking to prioritize verification and monitor calibration as new labels arrive. A dispatch decision would additionally require confirmation of the operational problem, the expected benefit of repair, and the cost of a visit.

7.18.2 Preserve the Information Available at the Forecast Origin

The full-year dimension table contains `n_days` and `last_seen`. Those quantities summarize future observation patterns and would leak information into a historical forecast. The next snapshot's count and check time are also unavailable at the origin; they define the evaluation sample here and do not enter the probability estimates as features. Restricting labels after collection must be reported because operational forecasts are issued before that future coverage is known. Origin-only features and exact calendar joins make the time sequence explicit.

The course table aggregates pairs by target date and previous status. It retains counts of eligible origins, observed labels, and unavailable outcomes, which are sufficient for these specified forecasts and scores. Grouping avoids repeating identical probability calculations across millions of rows. For a group with n labels, b unavailable outcomes, and forecast q, its total squared error is $b(1-q)^2 + (n-b)q^2$. The test verifies that this grouped expression equals the individual-row calculation on a hand-checkable example.

Python source: predict.py. From its directory, run `python predict.py`.

```
"""Test pre-specified availability forecasts on later calendar dates."""
import json
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parent
CUTOFF = pd.Timestamp("2026-08-01")


def fit_probabilities(train):
    counts = train.groupby("previous_unavailable")[["n_labeled", "n_unavailable"]].sum()
    if (counts.n_labeled <= 0).any() or set(counts.index) != {0, 1}:
        raise ValueError("Both prior-status groups require training observations")
    return (train.n_unavailable.sum() / train.n_labeled.sum(),
            counts.n_unavailable / counts.n_labeled)


def brier_score(n, successes, probability):
    """Exact row-level squared probability error from grouped binary counts."""
    return float(np.sum(successes * (1 - probability)**2
                        + (n - successes) * probability**2) / np.sum(n))


def main():
    data = pd.read_csv(ROOT / "data/transitions.csv", parse_dates=["date"])
    train, test = data[data.date < CUTOFF], data[data.date >= CUTOFF]
    assert train.date.max() < test.date.min()
    base, by_status = fit_probabilities(train)
    forecasts = {"Always available": np.zeros(len(test)),
                 "Training mean": np.full(len(test), base),
                 "Previous status": test.previous_unavailable.map(by_status).to_numpy()}
    n, y = test.n_labeled.to_numpy(), test.n_unavailable.to_numpy()
    scores = []
    for name, probability in forecasts.items():
        correct = np.where(probability >= 0.5, y, n - y)
        scores.append({"model": name, "brier_score": brier_score(n, y, probability),
                       "accuracy": float(correct.sum() / n.sum())})
    calibration = test.groupby("previous_unavailable")[["n_origins", "n_labeled", "n_unavailable"]].sum()
    calibration["forecast"] = by_status
    calibration["observed"] = calibration.n_unavailable / calibration.n_labeled
    calibration["label_coverage"] = calibration.n_labeled / calibration.n_origins
    results = ROOT / "results"
    results.mkdir(exist_ok=True)
    pd.DataFrame(scores).to_csv(results / "prediction_scores.csv", index=False)
    calibration.to_csv(results / "prediction_calibration.csv")
    summary = {"cutoff": str(CUTOFF.date()), "train_labeled_pairs": int(train.n_labeled.sum()),
               "test_labeled_pairs": int(n.sum()), "test_origins": int(test.n_origins.sum()),
```

```
                "test_unavailable": int(y.sum()), "baseline_probability": float(base),
                "test_label_coverage": float(n.sum() / test.n_origins.sum()),
                "train_first_target": str(train.date.min().date()),
                "train_last_target": str(train.date.max().date()),
                "test_last_target": str(test.date.max().date()), "scores": scores}
    (results / "prediction.json").write_text(json.dumps(summary, indent=2) + "\n")
    figure = r"""\begin{tikzpicture}
\begin{axis}[ybar,bar width=19pt,width=12.5cm,height=6.3cm,ymin=0,ymax=100,
 ylabel={Unavailable (\%)},xtick={0,1},xticklabels={Previously Available,Previously Unavailable},
 enlarge x limits=0.5,legend style={at={(0.5,-0.17)},anchor=north,legend columns=2,column sep=10pt},
 nodes near coords,every node near coord/.append style={font=\footnotesize,/pgf/number
 ↪  format/fixed,/pgf/number format/fixed zerofill,/pgf/number format/precision=1}]
"""
    for column, color in [("forecast", "IlliniBlue"), ("observed", "IlliniOrange")]:
        points = " ".join(f"({int(i)},{100*v:.6f})" for i, v in calibration[column].items())
        figure += f"\\addplot[draw={color},fill={color}!18] coordinates {{{points}}};\n"
    figure += "\\legend{Training Forecast,Later Observed Rate}\n\\end{axis}\n\\end{tikzpicture}\n"
    (ROOT.parents[1] / "06_Lecture_Notes/figures/M07_mcbroken_prediction.tex").write_text(figure)
    print(json.dumps(summary, indent=2))
    print(calibration.to_string())


if __name__ == "__main__":
    main()
```

Chapter 8

Current Topics and Integrated Marketing Decisions

A marketing team can now generate campaign copy, build a prediction model, and summarize customer feedback within a short working session. The remaining difficulty is deciding whether the resulting system improves a business decision. A recommendation engine can reproduce past exposure, a media model can attribute seasonal demand to advertising, and an AI-generated analysis can calculate an invalid comparison precisely. Evaluating these systems requires a clear account of their information, incentives, assumptions, and consequences.

8.1 Recommendation Systems Learn From Selected Exposure

A retailer wants to recommend products that customers will find useful. A common starting point is past interaction: purchases, clicks, ratings, or time spent. These observations combine preferences with the opportunities customers received. A product displayed prominently has more chances to be clicked than an otherwise similar product that was rarely shown. An absent interaction can therefore mean dislike, lack of exposure, lack of need, or an unavailable product.

Content-based methods use product attributes and customer histories to identify similar items. Collaborative methods use patterns across users and items, including matrix factorization, which approximates an interaction matrix with lower-dimensional user and item representations. Hybrid systems combine these sources. Each method needs a strategy for new customers and new products that have little interaction history. Popularity baselines and simple category rules provide useful comparisons.

Recommendation systems also change the data used to train the next model. A product that receives more exposure can accumulate more interactions, become easier to estimate, and receive still more exposure. This process can amplify initial differences without establishing that the repeatedly displayed product would be preferred under equal exposure.

Chaney, Stewart, and Engelhardt (2018) study this feedback through simulations in which recommendations shape the next training sample. Their results show that consumption can become more homogeneous without corresponding utility improvements. The result depends on the simulated preferences, recommendation rules, and updating process. It identifies a mechanism to examine in deployed systems rather than a universal effect of personalization.

Rankings determine exposure, exposure affects observed behavior, and that behavior enters subsequent training data. Customer preferences also affect behavior, while availability and position affect the opportunity to respond. Logging impressions, positions, eligible alternatives, and the policy that selected them helps distinguish these components. Purchase logs alone often omit the alternatives a customer could actually have encountered.

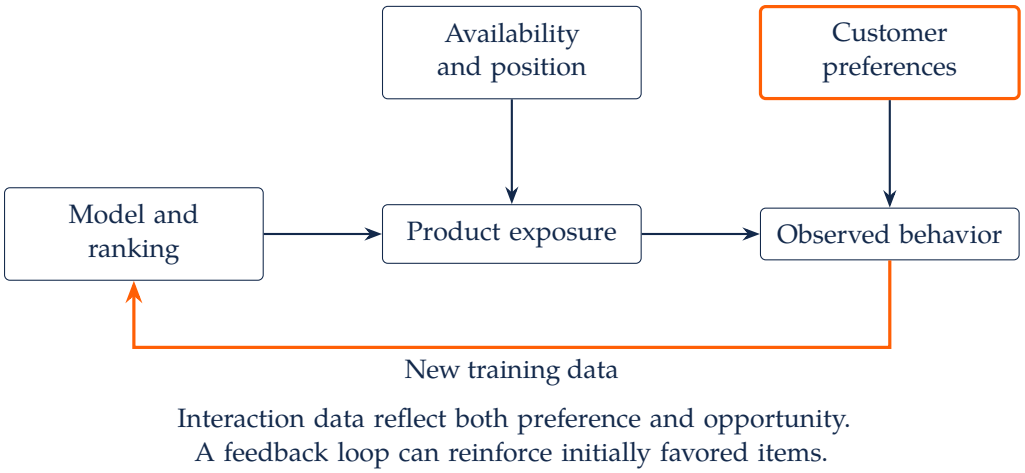


Figure 8.1. A recommender changes the exposure that generates its next training sample.

Evaluate recommendation quality at the unit relevant to the decision. Precision among the top recommended items and ranking metrics assess aspects of relevance. Coverage, novelty, diversity, repeat engagement, returns, and contribution describe other consequences. A system can improve one metric while reducing another. Offline evaluation on historical logs is particularly sensitive to the policy that generated those logs. An experiment that changes exposure can supply information that a more flexible model of the same selected data cannot supply.

8.1.1 Case Study: A Temporary Campaign Can Change the Learned Policy

A platform may stop boosting a category while continuing to use the behavior generated during that boost. Luo, Yao, and Zhang's 2026 working paper studies a celebrity sleep-reminder campaign on a short-video platform. During the campaign, a ranking adjustment increased the chance that treated users would see a reminder during the late-night window. Exposure remained dependent on the ranking process. The analysis uses a balanced panel of 200,000 users active on the experiment's first day, with 100,000 drawn from each assignment group.

Table 6 reports assignment effects of 14.75% higher late-night usage and 2.18% higher overall app usage relative to the respective control means during the campaign. These are relative percentage changes in usage, not percentage-point changes in exposure or measured effects on sleep. The paper also reports persistent engagement changes and changes in the content delivered afterward. The authors interpret the evidence through discovery of previously underexposed preferences and subsequent recommendation updates. The experiment estimates effects of the assignment regime; attributing persistence specifically to learning requires the additional mechanism evidence. The first-day activity restriction also makes sample inclusion consequential when extending the result to all assigned users.

The temporary boost ends, but its information can remain in the learned model. Freezing the model after the campaign would interrupt that particular channel while leaving any direct changes in customer behavior possible. Updating it incorporates the new interactions into future exposure. These policies can have different effects even when the campaign content and immediate responses

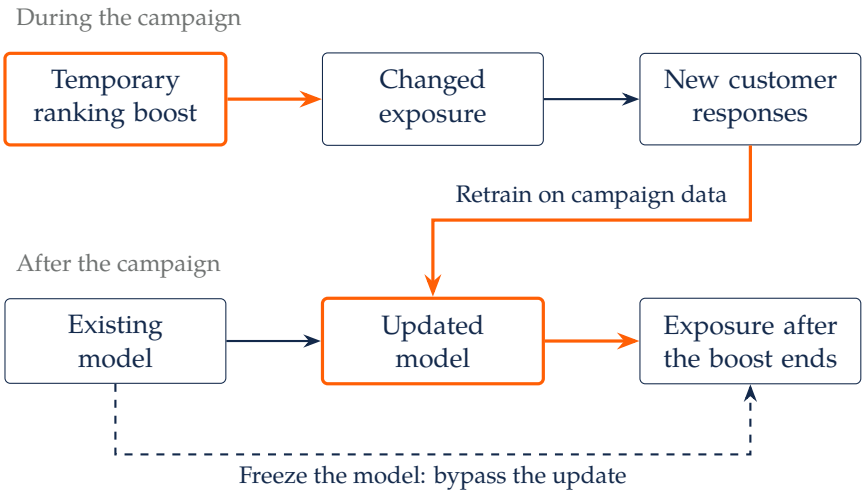


Figure 8.2. A temporary ranking intervention can affect later recommendations through the training data it creates.

are identical. Separating their contributions requires variation in the learning policy as well as the initial intervention.

An evaluation should retain the assignment, exposure, model version, retraining schedule, and outcomes after the intervention ends. A shared model can transmit treatment-generated information to comparison customers. Report the assignment unit and use uncertainty calculations that account for repeated observations of the same unit. Measure the intended customer outcome directly where possible, because increased engagement cannot reveal whether a campaign achieved a goal such as less late-night use or greater satisfaction.

8.1.2 Case Study: Netflix Personalization and Incremental Engagement

A streaming service wants to distinguish the appeal of its catalog from the value created by matching titles to viewers. Zielnicki, Aridor, Bibaut, Tran, Chou, and Kallus, in the supplied June 2026 paper, estimate a choice model using Netflix viewership and recommendation exposure. The model incorporates heterogeneous tastes and dependence on viewing history. Its identification treats exposure as conditionally exogenous given the included user, title, and state information. A separate randomized recommendation perturbation provides diversion patterns used to assess the model.

Under that model, replacing the current recommender with matrix factorization reduces engagement by approximately 4%, and replacing it with a popularity-based rule reduces engagement by approximately 12%. These are model-based counterfactual reductions relative to the existing recommender, not direct A/B estimates of those complete replacements. Their interpretation depends on the exposure condition, maintained choice model, and counterfactual implementation.

For a subscription business, engagement can affect retention and hence customer lifetime value, but the relationship requires evidence. A 12% engagement reduction does not imply a 12% decline in revenue, retention, or consumer welfare. Evaluate those outcomes separately, account for substitution across titles, and compare the incremental value with the cost of maintaining the rec-

ommendation system. The model's experimental check is useful because it evaluates responses to changed exposure, beyond fit to historical viewing alone.

8.2 Case Study: Relevance, Variety, and Customer Goals

A customer may prefer a useful collection of products even when some items in that collection have lower individual relevance scores. Category repetition, past purchases, and the customer's stated goal affect the value of the complete list. The food-recommendation studies in M7, Section 7.12, motivate these distinctions. A simple list objective makes the relevance-variety tradeoff explicit.

Suppose a retail customer requests three products and values variety across categories. Assign four candidate items the relevance scores below. The scores are comparable across these items and measure relevance rather than purchase probability.

Item	Category	Relevance Score
A	Coffee	0.90
B	Coffee	0.85
C	Tea	0.75
D	Fruit	0.65

Choosing the three highest scores gives A, B, and C, with total relevance 2.50 and two categories. Choosing A, C, and D gives relevance 2.30 and three categories. To make the tradeoff explicit, define a list objective

$$J(L) = \sum_{j \in L} s_j + \lambda K(L),$$

where L is a three-item list, s_j is relevance, $K(L)$ counts distinct categories, and λ is the value placed on an additional category in the same units as the relevance score. With $\lambda = 0.30$, A, B, C has objective 3.10, while A, C, D has objective 3.20. The latter list is preferred under this stated objective. The threshold at which these two lists tie is $\lambda = 0.20$.

The variety weight must come from an explicit objective, user preference, or evaluation process. Category counts are also a coarse measure: they treat all category differences equally and ignore which products the customer already owns. A deployed system would need better measurements and an evaluation of whether the resulting lists improve the intended customer experience.

For an experiment, randomize eligible customers to the existing and revised recommendation policies. Choose a primary outcome such as contribution per assigned customer over a fixed horizon, alongside measures of customer satisfaction and unwanted repetition. Analyze assignment rather than only customers who clicked a recommendation. Clicking occurs after the policy is assigned and can differ between policies. Preserve the list shown and the alternatives eligible at each impression so the mechanism can be examined.

8.2.1 Engagement, Considered Preferences, and the Objective

A customer can click an item and later wish it had not been recommended. Kleinberg, Mullainathan, and Raghavan model inconsistent preferences to explain why behavior alone can be insufficient for a platform seeking to improve user utility. The problem can arise even when the platform wants to serve the user: the observed action may reflect an immediate impulse that differs from a considered preference. A useful evaluation therefore combines behavior with explicit goals, satisfaction, and measures of regretted use, with their measurement limitations stated.

Stray and coauthors (2021) discuss recommender design around objectives such as well-being, diversity, and user control. For a retailer, a concrete formulation could maximize expected contribution subject to customer-selected spending limits, a maximum repetition rate, and a minimum satisfaction criterion. These constraints make the acceptable tradeoffs explicit. A weighted sum is another choice, but its conclusions depend on the scales and weights. Estimates of the component outcomes remain necessary under either formulation.

A list with more categories need not better serve every customer. Someone replenishing a familiar weekly basket may value reliability more than novelty. Elicit the relevant goal, preserve a way to revise it, and evaluate whether the system responds appropriately. A policy that increases variety on average can still repeatedly ignore the preferences of a particular group.

8.2.2 Case Study: Content Exposure and Political Attitudes Are Different Outcomes

Milli and coauthors (2025) conduct a preregistered audit with 806 Twitter users, comparing posts selected by engagement ranking with recent posts from accounts each user follows. They collect item-level stated preferences and perceptions. Engagement ranking selects more emotionally charged and out-group-hostile material, and the political posts selected are not preferred by users on the stated-preference measure. The outcomes describe selected content and users' evaluations within the audit's exposure window.

Liu and coauthors (2025) conduct four experiments with nearly 9,000 participants on a YouTube-like interface. Manipulating recommendation sequences produces limited short-term polarization effects in the studied settings. The intervention, exposure duration, and attitude outcomes differ from the Twitter audit. The studies therefore address different links between recommendation, consumption, perception, and opinion. A change in the composition of displayed content does not determine the magnitude of an attitude effect.

For a managerial comparison, state the population, assignment, exposure window, outcome, and uncertainty before combining findings across settings. A nonsignificant attitude effect supplies no general assurance of zero long-run effects, just as a difference in displayed content supplies no estimate of long-run polarization. The relevant evidence is the estimated effect and its uncertainty for the intervention actually evaluated.

8.2.3 Platform Incentives and Preference Changes

The platform's return from a recommendation can differ from the customer's value. Chen and Tsai (2024) examine Amazon's dual role as retailer and intermediary using product-level recommenda-

tion data and variation from Amazon stockouts. The study finds that Amazon's seller status affects the receipt of "Frequently Bought Together" recommendations. The design uses within-product changes, with accompanying controls and checks, to distinguish seller identity from stable product appeal. Its causal interpretation depends on the treatment of other changes accompanying stockouts.

An analyst evaluating a marketplace should record whether an item is sold by the platform, sponsored, or commission-bearing, alongside availability and customer relevance. Contribution should use the appropriate margin or fee for each item. A common ranking score can conceal those different incentives. A policy change may improve platform profit while reallocating sales among sellers, so report effects at the customer, seller, and platform levels relevant to the decision.

Zhu and coauthors develop a benchmark for distinguishing better matching from changes in users' preferences under a recommendation process. Their experiments use synthetic and real-world datasets within the proposed framework. They motivate measuring preference change separately from click-through performance. They do not establish that every online performance gain represents manipulation. A customer may learn to appreciate an unfamiliar product; whether that change serves the customer requires a stated criterion and evidence beyond a higher click count.

8.3 Marketing-Mix Models Combine Carryover and Saturation

A marketing-mix model relates aggregate outcomes to media activity and other determinants across time or markets. Its appeal is practical: many firms observe weekly sales and channel spending even when individual customer paths are unavailable. The analytical challenge is that media decisions often respond to expected demand. Advertising, promotions, holidays, distribution, and competitor actions can move together.

A simple carryover variable is

$$a_{ct} = s_{ct} + \rho_c a_{c,t-1}, \qquad 0 \le \rho_c < 1,$$

where s_{ct} is spending in channel c during period t, a_{ct} is its accumulated exposure proxy, and ρ_c controls geometric decay. This unnormalized definition gives the same units as spending, but its long-run level under constant spending is $s_c/(1-\rho_c)$. Some software normalizes the weights. Coefficients and interpretations must follow the selected convention.

If one spending pulse of 100 occurs at period zero and $\rho = 0.5$, the subsequent accumulated values are 100, 50, 25, and 12.5. A pulse can therefore coincide with later sales even when current spending is zero. A seasonal demand pattern can also create persistence. The model needs enough information and a credible design to distinguish these explanations.

Saturation describes declining additional response at higher exposure. A saturating response curve is $R(a) = A[1-\exp(-a/b)]$, where A is the limiting response and $b > 0$ controls how quickly the curve approaches that limit. A basic outcome model may combine channel response curves, controls, trend, seasonality, and an error term. The fitted components are model-dependent; their causal interpretation requires assumptions about how spending was chosen and which demand determinants were measured.

The official Meridian documentation describes a modern framework for marketing-mix modeling. The PyMC Marketing examples provide another implementation route. Use those materials to inspect definitions, priors, and diagnostics. A software package cannot supply a missing source of identifying variation. Experiments and external evidence can help calibrate a model, subject to whether their populations, interventions, and time horizons match the intended use.

8.4 Case Study: Allocate the Next Dollar Using Marginal Returns

Consider two channels with imposed contribution response curves

$$R_A(s_A) = 100[1 - \exp(-s_A/20)], \qquad R_B(s_B) = 80[1 - \exp(-s_B/40)].$$

Both spending and contribution are measured in thousands of dollars over the same horizon. These response functions omit cross-channel interactions and are assumed known. At a total budget of 60, suppose the current allocation is 40 to A and 20 to B. The question is whether moving a small amount between channels increases contribution while holding the total budget fixed.

Average contribution per unit spent is $R_c(s_c)/s_c$. At the current allocation, it is approximately 2.16 for A and 1.57 for B. Marginal contribution is the derivative of the response curve. It is approximately 0.68 for A and 1.21 for B. A has the higher average return, while B offers the greater return on the next small unit of spending.

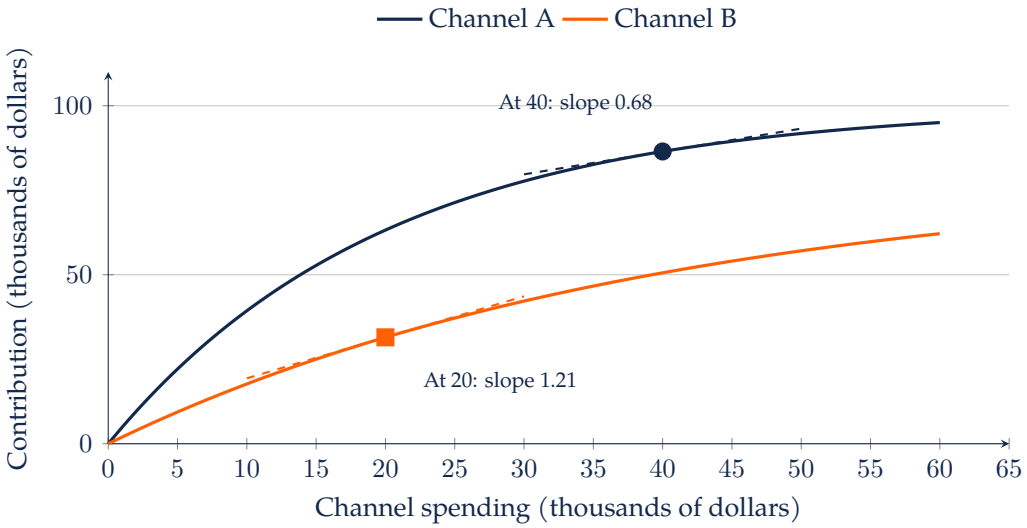


Figure 8.3. Average returns can favor one channel while marginal returns favor the other.

At the current allocations, B has the steeper response curve. Moving a small amount from A to B therefore raises total contribution. The comparison depends on the slopes at the current allocation, whereas average returns summarize all spending up to that point. The budget is held fixed, so the transfer does not change total spending expense.

For a differentiable, concave response with an interior optimum and no other constraints, marginal returns are equal across channels. Here the condition is

$$5\exp(-s_A/20) = 2\exp(-s_B/40), \qquad s_A + s_B = 60.$$

Solving gives $s_A \approx 32.22$ and $s_B \approx 27.78$. Total contribution rises from approximately 117.94 to 120.09, or about 2.14 thousand dollars. If total budget were also a choice, marginal contribution would be compared with the marginal cost of spending; exhausting a fixed budget and choosing its size are different optimization problems.

A real decision should also examine parameter uncertainty, operational minimums, saturation outside observed spending ranges, delayed effects, and interactions across channels. Report how the recommended allocation changes under plausible response curves. A sharply stated optimum based on weakly identified slopes can conceal substantial decision uncertainty. Prefer an allocation that is supported by observed variation and specify an experiment that would resolve the most consequential uncertainty.

8.5 Generative AI Changes the Cost of Producing and Checking Work

Generative AI can assist with code, text classification, documentation, and communication. The business value depends on the complete workflow. Faster generation can be offset by additional review, integration failures, or errors that reach downstream decisions. Measure the output that matters: a correct, reproducible analysis completed with an appropriate understanding of its limitations.

Separate several uses. A coding assistant produces or edits a program that can be inspected and executed. An annotation model produces labels that require a measurement evaluation. A retrieval system selects source passages and may generate a summary that needs factual support. A content system produces customer-facing material whose effect should be evaluated in its deployment context. Each use has a different unit of quality and a different failure mechanism.

For coding, begin with an analytical specification and a small example with a known result. Ask the assistant to implement a limited step, explain the data transformations, and identify assumptions. Inspect the code before running it. Then compare intermediate outputs with independent checks. A program that executes successfully can still use an incorrect denominator, join on the wrong key, or include future information. Those errors require substantive checks rather than syntax repair.

For retrieval and summarization, maintain the connection between a claim and its source. A citation should support the particular statement next to it. Ask the model to distinguish an observed result from a proposed explanation, and check that distinction against the original text. If a passage or file is unavailable, state that limitation. Fluent reconstruction is not a substitute for access to the evidence.

8.6 Case Study: Evaluate an AI-Assisted Analytics Workflow

Suppose a firm wants to know whether access to a coding assistant improves analysts' work on customer-retention tasks. Define treatment as access under a specified protocol and version, and define an outcome that incorporates both completion and validity. A useful primary outcome

could be the fraction of assigned tasks meeting a prespecified analytical rubric within a fixed time budget. Secondary outcomes could include completion time, independent error severity, and a later explanation task completed without assistance.

Randomize analysts or teams according to the likely spillovers. Repeated tasks by the same analyst require attention to learning, carryover, and dependence. If a crossover design is used, randomize order and consider whether experience in the first condition changes performance in the second. A task-level comparison with no accounting for analyst ability or task difficulty can confound the treatment effect with who received which work.

Quality review should use a common rubric and, where feasible, reviewers unaware of treatment assignment. The rubric can check observation units, joins, leakage, estimation, uncertainty, and interpretation. Record the assigned task, the eligible inputs, the tool configuration, and the evaluation procedure. Avoid interpreting a favorable result for one task type as a universal effect across all analytical work.

Suppose a manual workflow uses 90 minutes to produce an analysis and 20 minutes to review it, totaling 110 minutes. An assisted workflow uses 45 minutes to produce a draft and 40 minutes to review it, totaling 85 minutes. The reduction in total time is 25 minutes, or about 22.7%, while the drafting-time reduction is 50%. The total-time comparison is informative only if the completed outputs meet comparable quality requirements.

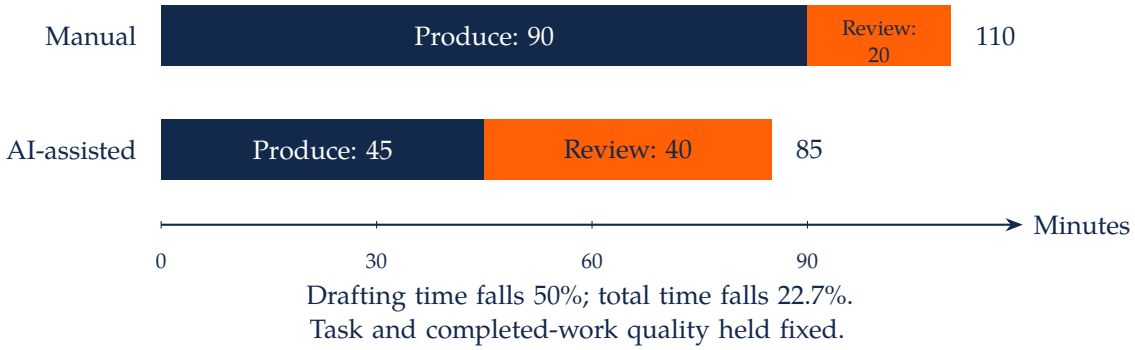


Figure 8.4. The relevant time saving includes generation, review, and correction.

This comparison holds the task and required quality fixed. Review is part of production because an undetected error can alter a marketing decision. In empirical work, report the completion and quality outcomes alongside time so that faster abandonment or lower-quality work is not counted as an equivalent improvement.

8.7 Integrated Case: A Retail Subscription Launch

A retailer is considering a subscription service with recommendations and delivery benefits. Begin with the decision: which offer to launch, to which eligible customers, at what price, with what evidence of incremental contribution? Define the evaluation horizon, capacity constraints, and the alternatives to launch.

Construct the customer and order tables with explicit keys and dates. Describe customer activity and purchasing cohorts, keeping unavailable future retention separate from observed inac-

tivity. Use regression to summarize conditional patterns while examining confounding. Design a conjoint exercise to learn stated tradeoffs between delivery, assortment, and price, then assess whether its predictions generalize to an actual offer.

Use segmentation to organize meaningful differences in needs and behavior, and calculate customer value from contribution rather than revenue. Analyze reviews to identify the service attributes customers mention, checking annotation quality against human labels. Develop a prediction model using only information available at the targeting date. Its forecasts can help plan capacity, while a randomized offer evaluates incremental take-up, contribution, and retention.

Finally, evaluate the recommendation component and the acquisition budget. Log exposure to recommended items, distinguish average from marginal media returns, and use AI assistance within a documented verification process. The management recommendation should identify the supported action, the uncertainty that could change it, and the next observation or experiment with the greatest decision value. A complicated collection of models is useful only if these pieces support a coherent choice.

8.8 Practice and Further Reading

Recommendation exercise. Compare all four three-item lists in the variety example at $\lambda = 0$, 0.20, and 0.30. At zero, A, B, C is optimal. At 0.20, it ties A, C, D. At 0.30, A, C, D is optimal. Explain how a different category definition could change that conclusion without changing any relevance score.

Dynamic-policy exercise. Design a comparison between a temporary recommendation boost with a frozen model and the same boost followed by retraining. Specify how training data are separated across conditions, when outcomes are measured, and how shared content or customers could transmit effects. Explain which comparison identifies the effect of allowing the model to learn from campaign-period behavior.

Evidence exercise. Compare the MovieLens field experiment, the Netflix model-based replacement counterfactuals, and the short-term YouTube experiments. State the intervention and estimand for each. Explain why their reported outcomes cannot be combined into a single percentage measuring the value or harm of recommendation systems.

Budget exercise. Calculate the gain from moving one thousand dollars from channel A to channel B at the original allocation. Compare the exact change with the difference between the two marginal returns. The derivative is a local approximation; the exact finite transfer also reflects curvature. Then explain why an unobserved seasonal demand shock could invalidate a causal interpretation of an estimated response curve.

AI evaluation exercise. Draft a rubric for an assistant-generated retention analysis. Include at least one check that detects a substantive error in code that executes successfully, such as an order-line denominator used for average order value. Define a hand-checkable example that exposes the error and explain how a reviewer would determine whether it occurred.

Read Nielsen and coauthors, "Consumption Variety in Food Recommendation", and Ling and coauthors, "Following Good Examples", for applications connecting recommendation objectives with customer behavior. The official marketing-mix documentation linked above develops implementation details. Consult the AI-coding evidence and workflow resources for further study.

8.9 mHealth: Familiarity, Novelty, and the Decision Objective

A manager may value introducing unfamiliar categories even when the most accurate prediction repeats a customer's routine. That preference changes the ranking objective. It should be stated as a decision choice before interpreting any performance comparison. A category absent from recent logs is novel relative to recorded history; it may be familiar to the person outside the app.

Retain the September users, target baskets, candidate set, and 30-day frequency rule from M7. Add a bonus λ to a candidate category if it has not appeared in the user's prior 30 calendar days of logs:

$$S_{ic}(\lambda) = f_{ic} + 10^{-6} p_c + \lambda \mathbf{1}\{c \text{ absent from the prior 30-day log}\}.$$

Here f_{ic} is the fraction of observed history days containing category c, and p_c is global popularity used for tie resolution. The bonus is measured in the same score units as frequency. It is an imposed objective weight, not an estimated willingness to try a food or a nutritional benefit. Rank all candidates by this score and count the fraction of the ten recommendations absent from recent history.

Novelty Bonus	Recall at 10	NDCG at 10	Novel Categories in the List
0.00	0.4759	0.5862	4.78%
0.05	0.4759	0.5862	4.82%
0.10	0.4759	0.5863	5.28%
0.20	0.4661	0.5809	9.83%

The grid is a specified sensitivity comparison on the same 299 test users. It is not used to select a winning policy and then claim an independent test of that winner. At a bonus of 0.20, novelty roughly doubles while recall falls by about 0.98 percentage points. At smaller bonuses, changes in the list can occur among categories that are all absent from the target, leaving recall unchanged. NDCG can move without a recall change when relevant categories exchange positions.

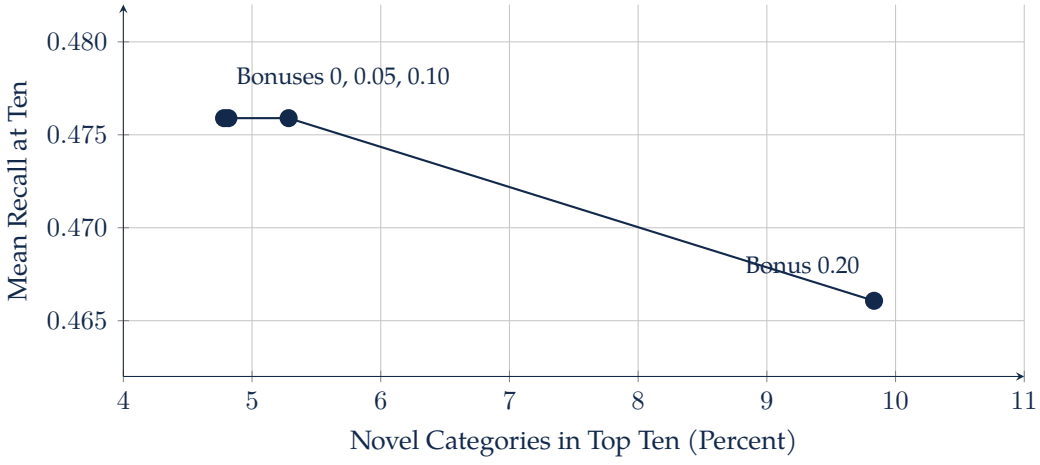


Figure 8.5. A novelty bonus changes list composition before it substantially changes basket recovery.

The first three points lie close together in recall even though novelty rises. The fourth trades some recovery of the recorded basket for greater novelty. This is an offline relationship between a ranking rule and historical labels. It does not measure whether customers prefer the new list, accept the suggestions, or change diet.

A nutrition objective would need a validated nutrient database, appropriate portions, restrictions supplied by the customer, and an outcome definition. Source categories and calorie-share variety alone do not establish healthy choices. The data-cleaning distinctions in M2 therefore affect what the manager can optimize. Dropping low-calorie days might improve an apparent fit by selecting more complete logs, while also excluding the customers for whom the interface is hardest to use.

A concrete next decision is whether to run the M6 user-randomized comparison, using a prespecified novelty setting and separate measures of logging, acceptance, and any validated nutrition outcome. The experiment should preserve the assignment population and record exposure. An AI assistant can implement the frozen ranking rule and audit category descriptions, while unsupported health claims remain outside the factual explanation supplied with the recommendation.

Python source: m08.py. From its directory, run `python m08.py`.

```
"""Measure an imposed novelty-relevance tradeoff on the same September baskets."""

import json
from common import OUTPUT, load_inputs, save_result
from ranking import candidate_categories, evaluate

food, daily, baskets = load_inputs()
selection = json.loads((OUTPUT / "M07.json").read_text())
windows = {"30_days": 30, "90_days": 90, "all_prior_days": None}
window = windows[selection["chosen_window"]]
candidates = candidate_categories(baskets)
# This grid is a specified sensitivity analysis, not a test-set model search.
weights = [0.0, 0.05, 0.10, 0.20]
tradeoff = [
    {
        "novelty_weight": weight,
        **evaluate(baskets, "2016-09-01", candidates, window, novelty_weight=weight),
    }
    for weight in weights
]
assert (
    abs(
        tradeoff[0]["recall_at_10"]
        - selection["test"]["personal_frequency"]["recall_at_10"]
    )
    < 1e-12
)
result = {"history_window": selection["chosen_window"], "tradeoff": tradeoff}
save_result("M08", result)
print(json.dumps(result, indent=2))
```

8.10 Python Calculations for the Synthetic Cases

The inputs below are imposed teaching values. Running the program creates the labeled synthetic CSV files and prints the corresponding calculations. They are separate from the mHealth estimates. Keep the script beside `example_helpers.py` in `03_Data_and_Code/Lecture_Examples`.

Python source: m08_synthetic.py. From its directory, run `python m08_synthetic.py`.

```
"""Synthetic numerical examples for M8; run from any working directory."""

import itertools
import json
import math
from example_helpers import write_csv, ols_slope, shares

r = {}
items = {
    "A": ("Coffee", 0.90),
    "B": ("Coffee", 0.85),
    "C": ("Tea", 0.75),
    "D": ("Fruit", 0.65),
}
list_values = {}
for weight in [0, 0.2, 0.3]:
    list_values[str(weight)] = {
        "".join(choice): sum(items[j][1] for j in choice)
        + weight * len({items[j][0] for j in choice})
        for choice in itertools.combinations(items, 3)
    }
ra = lambda s: 100 * (-math.expm1(-s / 20))
rb = lambda s: 80 * (-math.expm1(-s / 40))
sa = (40 * math.log(2.5) + 60) / 3
sb = 60 - sa
r["M08"] = {
    "recommendation_objectives": list_values,
    "carryover": [100 * 0.5**t for t in range(4)],
    "average_A": ra(40) / 40,
    "average_B": rb(20) / 20,
    "marginal_A": 5 * math.exp(-40 / 20),
    "marginal_B": 2 * math.exp(-20 / 40),
    "current_total_contribution": ra(40) + rb(20),
    "optimal_spend_A": sa,
    "optimal_spend_B": sb,
    "optimal_total_contribution": ra(sa) + rb(sb),
    "contribution_gain": ra(sa) + rb(sb) - ra(40) - rb(20),
    "one_unit_transfer_gain": ra(39) + rb(21) - ra(40) - rb(20),
    "manual_total_minutes": 110,
    "assisted_total_minutes": 85,
    "time_reduction_fraction": 25 / 110,
}
assert math.isclose(5 * math.exp(-sa / 20), 2 * math.exp(-sb / 40))
```

```
# Independent grid check of the constrained optimum, to the grid's precision.
grid_best = max((ra(i / 100) + rb(60 - i / 100), i / 100) for i in range(6001))
assert abs(grid_best[1] - sa) < 0.01
write_csv(
    "M08_recommendation_synthetic.csv",
    ["item", "category", "relevance_score"],
    [(key, *value) for key, value in items.items()],
)
print(json.dumps(r, indent=2))
```

8.11 Evidence Checks for Generated Marketing Explanations

An analytics manager may accept a fluent business review whose numbers and interpretation describe different populations. Structured output helps software locate a metric, period, value, unit, and denominator. Each field also needs evidence. A correctly formatted statement can contain the wrong number or describe a causal effect that the underlying analysis never estimated.

Use the fixed-cohort mHealth review from Section 1.10 as the evidence source. June contains 6,331 observed user-days out of 10,200 eligible calendar user-days, giving a logging fraction of 0.620686. A structured claim must identify that month, denominator, unit, and descriptive interpretation. Four constructed candidates retain the same field structure: one reproduces the supported result, one adds 0.10 to its value, one substitutes the number of logging users for the denominator, and one labels the result a causal health benefit.

Constructed Candidate	Format	Evidence Check
Supported Result	Valid Fields	Accepted
Changed Number	Valid Fields	Value Mismatch
Changed Denominator	Valid Fields	Denominator Mismatch
Causal Health Claim	Valid Fields	Unsupported Interpretation

All four candidates pass the same basic structural requirement. Only the supported claim passes the evidence checks. They are deliberate teaching inputs, not observed outputs from a particular LLM. The code makes no external model request and sends no customer data to an AI service. Its checks concern the declared fields. Free-form prose, omitted qualifications, and advice require a separate substantive review; an "interpretation" field cannot validate an unrestricted paragraph.

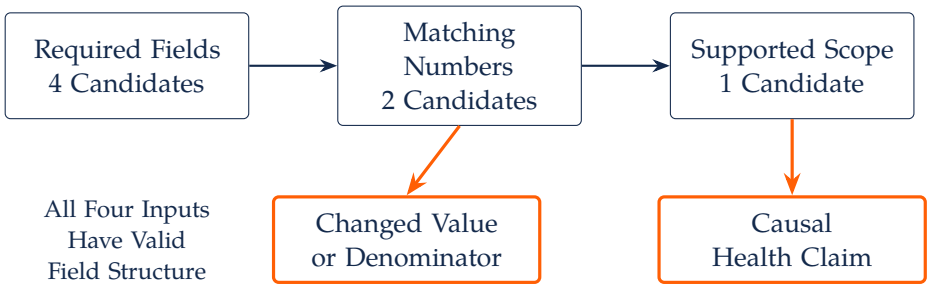


Figure 8.6. A generated statement passes through separate checks for structure, numerical evidence, and substantive interpretation before informing a decision.

Copycat demonstrates generation of advertising text from brand examples and evaluates similarity and memorization. Those diagnostics concern the generated content. Incremental sales or engagement requires an outcome comparison under an appropriate exposure design, such as a randomized test of the generated and existing alternatives. The Gemini and Google Cloud marketing notebooks in the resource collection supply further examples of generation and structured extraction; model credentials, costs, and data permissions are separate requirements for executing those reference notebooks.

Synthetic consumer panels raise a related distinction. The semantic-similarity-rating package maps generated text to distributions over response anchors. The mapping can standardize how generated answers are summarized. Validation against a target population still requires human outcomes, a sampling definition, and comparisons held out from prompt or anchor selection. Generating more answers under the same misspecified prompt reduces simulation noise around that prompt's distribution while preserving its systematic difference from consumers. Agreement among generated respondents is therefore a property to examine alongside external validity.

A useful AI coding request is: "Compare every numerical field with its computed evidence; reject unknown metrics, periods, units, and denominators; retain the analysis's descriptive interpretation; and test a correctly formatted false claim." The checking rule should be written before inspecting a favored generated answer. Report the factual error rate and the time spent checking, alongside any reduction in drafting time.

Python source: claim_checks.py. From its directory, run `python claim_checks.py`.

```
"""Check constructed structured claims against computed mHealth evidence."""

import json
import math
from copy import deepcopy
from common import load_food, save_result
from reporting import monthly_review


def check_claim(claim, evidence):
    required = {"metric", "period", "value", "unit", "denominator", "interpretation"}
    if not isinstance(claim, dict) or set(claim) != required:
        return ["schema"]
    rows = [row for row in evidence["periods"] if row["period"] == claim["period"]]
    if len(rows) != 1 or claim["metric"] != evidence["metric"]:
        return ["unknown metric or period"]
    row = rows[0]
    errors = []
    if type(claim["value"]) not in (int, float) or not math.isfinite(claim["value"]):
        errors.append("numeric value")
    elif not math.isclose(
        claim["value"], row["logging_fraction"], rel_tol=0, abs_tol=1e-10
    ):
        errors.append("value disagrees with evidence")
    if (
        type(claim["denominator"]) is not int
        or claim["denominator"] != row["calendar_user_days"]
```

```
    ):
        errors.append("denominator")
    if claim["unit"] != evidence["unit"]:
        errors.append("unit")
    if claim["interpretation"] != evidence["interpretation"]:
        errors.append("unsupported interpretation")
    return errors


if __name__ == "__main__":
    evidence = monthly_review(load_food())
    june = evidence["periods"][1]
    valid = {
        "metric": "logging_fraction",
        "period": "2016-06",
        "value": june["logging_fraction"],
        "unit": evidence["unit"],
        "denominator": june["calendar_user_days"],
        "interpretation": "descriptive",
    }
    candidates = {"supported": valid}
    changes = {
        "wrong_number": ("value", valid["value"] + 0.1),
        "wrong_denominator": ("denominator", june["users_with_logs"]),
        "causal_claim": ("interpretation", "causal health benefit"),
    }
    for name, (field, value) in changes.items():
        candidates[name] = deepcopy(valid)
        candidates[name][field] = value
    result = {
        "candidate_origin": "constructed teaching examples; no LLM request",
        "checks": {
            name: check_claim(claim, evidence) for name, claim in candidates.items()
        },
    }
    assert result["checks"]["supported"] == []
    assert all(result["checks"][name] for name in changes)
    save_result("claim_checks", result)
    print(json.dumps(result, indent=2))
```

8.12 An Executable Chart Can Still Misstate the Evidence

A manager asks an AI assistant to plot monthly customer engagement. The generated program can run successfully and produce an attractive chart while changing the population, labeling a fraction as a percentage, or describing a temporal association as an intervention effect. A chart therefore needs checks of execution, fidelity to the analytical question, and readability. VisEval separates these dimensions when evaluating generated visualizations.

The mHealth example provides an independent numerical reference. The requested measure is the percentage of all eligible calendar user-days with a log. Its two numerators and denominators are computed before plotting. The following constructed mistakes retain valid numerical inputs and executable drawing instructions. Each fails a different substantive requirement.

Constructed Chart Error	Consequence	Check
Plot a fraction under a percent label	Values are 100 times too small	Recompute units
Keep only users who logged	Population changes with the outcome	Reconcile denominators
Start rate bars at 60%	Bar lengths distort magnitudes	Inspect baseline
Declare the time difference causal	Claim exceeds the design	Inspect interpretation

The checker below accepts the correct specification and rejects all four constructed mistakes. Its numerator and denominator checks use the saved monthly evidence, rather than the plotted value as its own reference. This is a narrow validator for declared fields. It does not interpret unrestricted natural-language titles, inspect pixels, or guarantee that a rendered chart is readable. Review the displayed title, labels, clipping, color contrast, and all interactive selections separately. A chart can pass these field checks while still misleading through an omitted subgroup or an unsupported narrative.

Ask a coding assistant to return the aggregate plotting table, units, eligibility rule, axis domains, and one hand-calculated plotted value with its code. A second review should compare those objects with the original analysis specification. Passing the assistant's own test is informative only when the expected answer was derived independently. The Financial Times vocabulary supports the choice of display, while Rougier's visualization code provides examples of figure construction and layout. The managerial claim must remain tied to the evidence and design.

Run `visualization.py` first to create the evidence table, then run the audit. The false charts are constructed teaching cases; their rejection rates do not measure the reliability of any particular language model.

Python source: chart_audit.py. From its directory, run `python chart_audit.py`.

```
"""Check a narrow chart specification against the monthly evidence table."""

from copy import deepcopy
import json
import math
from common import OUTPUT, save_result


def check_chart(chart, evidence):
    errors = []
    expected = {row["period"]: row for row in evidence["months"]}
    if chart.get("unit") != "percent of eligible user-days":
        errors.append("axis unit")
    if chart.get("interpretation") != "descriptive":
```

```
        errors.append("interpretation")
    if chart.get("mark") == "bar" and chart.get("y_domain", [None])[0] != 0:
        errors.append("bar baseline")
    rows = chart.get("rows", [])
    if sorted(row["period"] for row in rows) != sorted(expected):
        errors.append("period coverage")
    for row in rows:
        ref = expected.get(row["period"])
        if ref is None:
            continue
        value = row["value"]
        target = 100 * ref["observed_user_days"] / ref["calendar_user_days"]
        if not math.isfinite(value) or not math.isclose(value, target, abs_tol=1e-8):
            errors.append("plotted value")
        if row["denominator"] != ref["calendar_user_days"]:
            errors.append("denominator")
    return sorted(set(errors))


def main():
    evidence = json.loads((OUTPUT / "visualization.json").read_text())
    correct = {
        "unit": "percent of eligible user-days", "interpretation": "descriptive",
        "mark": "bar", "y_domain": [0, 100],
        "rows": [{"period": r["period"], "value": r["logging_percent"],
                  "denominator": r["calendar_user_days"]} for r in evidence["months"]],
    }
    cases = {"correct": correct}
    for name in ["fraction_as_percent", "conditional_denominator", "truncated_bar", "causal_title"]:
        cases[name] = deepcopy(correct)
    cases["fraction_as_percent"]["rows"][0]["value"] /= 100
    row = cases["conditional_denominator"]["rows"][0]
    row["denominator"] = evidence["months"][0]["logger_calendar_days"]
    row["value"] = evidence["months"][0]["conditional_logging_percent"]
    cases["truncated_bar"]["y_domain"] = [60, 75]
    cases["causal_title"]["interpretation"] = "causal"
    result = {name: check_chart(chart, evidence) for name, chart in cases.items()}
    assert result["correct"] == []
    assert all(result[name] for name in cases if name != "correct")
    save_result("chart_audit", {"constructed_cases": result})
    print(json.dumps(result, indent=2))


if __name__ == "__main__":
    main()
```

8.13 Competition, Demand Measurement, and Generative AI

A provider deciding whether to cut its subscription price needs to anticipate where customers come from and how long they remain. New subscribers can expand the paid category, replace a competing provider, or add a second service. These responses have different implications for market size and competitive pressure. Posted prices and public attention help describe the available offers and interest in them, while demand analysis requires choices linked to the prices and alternatives available when those choices occurred.

Egan, Russel, Shi, and Yao (2026) study paid consumer subscriptions using card transactions. Their abstract reports 2.9% paid adoption by April 2026, substantial turnover, and growing substitutability among providers. Fradkin (2025) studies applications using models through OpenRouter and documents variation in whether releases expand usage or displace other models. The population and choice object differ: consumer subscriptions, application-level model use, and token demand answer different managerial questions. A descriptive retention gap alone does not identify a structural substitution parameter.

8.13.1 A Price Coefficient With a Known Interpretation

To connect a subscription probability to price, consider an imposed binary-choice benchmark. A homogeneous consumer chooses whether to hold one provider in a month. Let p_{jt} be its price in dollars per month, δ_j its provider intercept, and τ_t a common month effect. A logistic utility difference with price coefficient $-\alpha$ implies

$$s_{jt} = \frac{\exp(\delta_j + \tau_t - \alpha p_{jt})}{1 + \exp(\delta_j + \tau_t - \alpha p_{jt})}, \qquad \log\left(\frac{s_{jt}}{1 - s_{jt}}\right) = \delta_j + \tau_t - \alpha p_{jt}.$$

The share s_{jt} is the choice probability for this binary offer. The coefficient α measures the reduction in log odds per additional dollar. Homogeneous tastes, the binary choice structure, and the absence of lagged subscription states make the inversion exact. With exact model-implied shares and price variation remaining after provider and month controls, the regression recovers the imposed coefficient. With sampled choices, finite-sample uncertainty must also be assessed.

Holding the intercepts fixed, the benchmark's own-price elasticity is $-\alpha p_{jt}(1-s_{jt})$. That calculation describes a response within the specified binary model. Cross-provider substitution requires an explicit joint choice model and variation that supports its parameters. Consumers holding several subscriptions need a choice structure that permits those combinations; a single-choice nested logit would change that behavior.

8.13.2 Why Subscriber Stocks and New Subscriptions Differ

The supplied simulator draws heterogeneous consumer affinity and monthly logistic shocks, adds provider quality and inertia from last month's holdings, and selects a set of subscriptions subject to a cost of additional providers. Premium status is assigned after the provider-holding decision. A common utility shift is recalibrated each month to approach the target adoption path. These rules produce a useful dynamic teaching panel, but they differ from the homogeneous binary benchmark.

Construct one row for every available provider-month using the price table, including combinations with zero subscribers. Divide the number of basic-plan holders by the full roster to form a stock share. Divide new basic-plan holders by that same roster to form a flow share. The current data contain 179 available provider-months; all have positive stock and flow shares, so the two regressions use the same observations. Each regression includes provider and month effects.

Share Used in the Regression	Price Coefficient per Dollar	Interpretation
Basic-plan stock divided by the consumer roster	-0.0300	Descriptive log-odds projection of current holdings
New basic-plan holders divided by the consumer roster	-0.0499	Descriptive log-odds projection of acquisition flow
Exact homogeneous binary-model probability	-0.0600	Recovery of the imposed coefficient under the benchmark

The simulator also imposes a utility price coefficient of -0.0600. The flow slope is numerically closer to it in this realization. That proximity does not establish recovery: average choices across heterogeneous consumers generally do not obey the representative-consumer logit formula, and changing the outcome from holdings to acquisitions leaves selection, inertia, competing offers, and premium assignment in the data-generating process. The common utility recalibration also fixes aggregate adoption near its target as prices and qualities vary. A promotion counterfactual must specify which of those objects remains fixed.

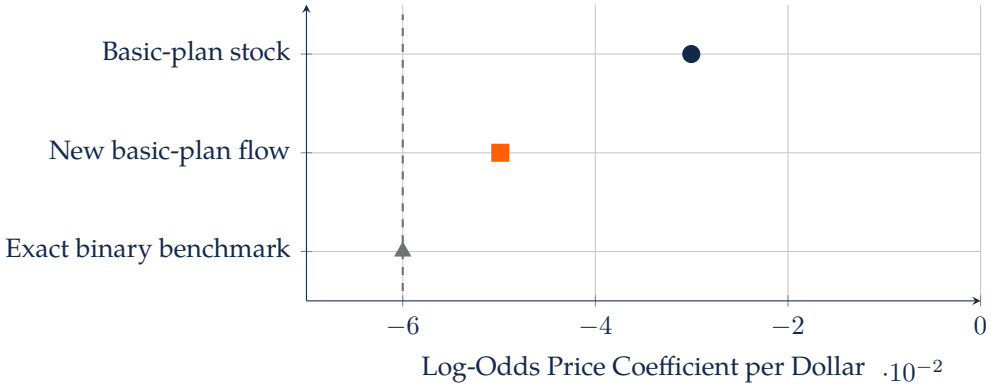


Figure 8.7. Three price coefficients calculated on the same 179 provider-month price observations. The dynamic simulation gives a stock slope of -0.0300 and a flow slope of -0.0499; exact binary-logit probabilities recover -0.0600. The dots are point calculations without confidence intervals and do not measure real-market price sensitivity.

An acquisition flow divided by all consumers also differs from an acquisition probability among eligible nonsubscribers. For a provider-level holding indicator D_{ijt}, its acquisition hazard conditions on $D_{ij,t-1} = 0$. A new-holder count divided by N combines that hazard with the proportion eligible to acquire the provider. Changes in the eligible population can therefore change the flow share even when its conditional response is stable. Plan-level flows require an additional definition of upgrades, downgrades, and new provider relationships.

8.13.3 Auditing an AI-Generated Demand Analysis

A useful coding specification states the consumer roster, available alternatives, monthly price units, subscription-history definition, and target coefficient before requesting a model. Require the code to retain zero-count provider-months, check whether prices vary after the fixed effects, and reject undefined log odds. A pseudocount changes the transformed outcome and should follow an explicit statistical model rather than conceal zero shares.

For a simulation, compare the estimator with the generator line by line. The price appearing in a utility equation, the recorded price of a chosen premium plan, and the average amount paid are different objects. Verify whether the program holds category attractiveness fixed or recalibrates it when prices change. Then test the estimator on exact probabilities from a model for which its equation is correct. The benchmark below passes that test; the dynamic-panel regressions retain their descriptive interpretation.

The regression uses NumPy least squares and checks the price slope independently by removing provider and month effects from both price and the transformed share. Agreement verifies the linear calculation. Identification of a price response additionally requires justified price variation, a suitable choice model, and uncertainty that respects the level at which prices and shocks vary. The 60,000 simulated consumers do not create 60,000 independent price assignments.

Python source: demand.py. From the script's folder, run `python3 demand.py`. Inputs are read from the supplied local bundle; output tables are written beside the course scripts.

```
"""Contrast descriptive price regressions with an exact binary-logit benchmark."""
from pathlib import Path
import json

import numpy as np
import pandas as pd

HERE = Path(__file__).resolve().parent
SOURCE = HERE.parents[1] / "01_Resources/08_M8_Current_Topics_in_Marketing/genai_demand_lab/data"
OUTPUT = HERE / "results"


def design(frame):
    # All provider intercepts plus all but one month effect; no extra intercept.
    fixed = pd.concat([pd.get_dummies(frame.provider, dtype=float),
                       pd.get_dummies(frame.month, drop_first=True, dtype=float)], axis=1)
    x = np.column_stack([fixed.to_numpy(), frame.basic_price.to_numpy()])
    if np.linalg.matrix_rank(x) != x.shape[1]:
        raise ValueError("Price is not separately identified from the fixed effects")
    return x


def price_projection(frame, shares):
    shares = np.asarray(shares, dtype=float)
    if not np.isfinite(shares).all() or not ((shares > 0) & (shares < 1)).all():
        raise ValueError("Log-odds require shares strictly between zero and one")
    x = design(frame)
```

```
    y = np.log(shares) - np.log1p(-shares)
    beta = np.linalg.lstsq(x, y, rcond=None)[0]
    # Independent Frisch-Waugh-Lovell check of the price coefficient.
    controls, price = x[:, :-1], x[:, -1]
    price_residual = price - controls @ np.linalg.lstsq(controls, price, rcond=None)[0]
    y_residual = y - controls @ np.linalg.lstsq(controls, y, rcond=None)[0]
    fwl = float(price_residual @ y_residual / (price_residual @ price_residual))
    np.testing.assert_allclose(beta[-1], fwl, atol=1e-10)
    return {"price_coefficient": float(beta[-1]), "rows": len(frame),
            "residual_price_sum_squares": float(price_residual @ price_residual)}


def main():
    subs = pd.read_csv(SOURCE / "synthetic_subscriptions.csv.gz")
    consumers = pd.read_csv(SOURCE / "synthetic_consumers.csv")
    prices = pd.read_csv(SOURCE / "synthetic_basic_prices.csv")
    dgp = json.loads((SOURCE / "synthetic_dgp.json").read_text())
    keys = ["month", "provider"]
    assert consumers.consumer_id.is_unique and not prices.duplicated(keys).any()
    assert not subs.duplicated(["consumer_id"] + keys).any()
    basic = subs.loc[subs.tier.eq("basic")]
    stocks = basic.groupby(keys).size().rename("basic_holders")
    flows = basic.loc[basic["new"]].groupby(keys).size().rename("new_basic_holders")
    frame = prices.merge(stocks, on=keys, how="left", validate="one_to_one")
    frame = frame.merge(flows, on=keys, how="left", validate="one_to_one")
    frame[["basic_holders", "new_basic_holders"]] = frame[["basic_holders",
    ↪  "new_basic_holders"]].fillna(0)
    # Zero filling is justified by the complete simulated population and price support.
    frame["stock_share"] = frame.basic_holders / len(consumers)
    frame["flow_share"] = frame.new_basic_holders / len(consumers)
    common = frame.loc[frame.stock_share.between(0, 1, inclusive="neither") &
                       frame.flow_share.between(0, 1, inclusive="neither")].copy()
    results = []
    for name in ["stock_share", "flow_share"]:
        results.append({"outcome": name, **price_projection(common, common[name])})
    # Exact model-implied shares: homogeneous binary choice, no inertia or multihoming.
    # Provider/time intercepts vary, but the same price design and alpha are retained.
    x = design(common)
    intercepts = np.linspace(-4.0, -3.0, common.provider.nunique())
    time_effects = np.linspace(0.0, 0.2, common.month.nunique() - 1)
    controls = np.r_[intercepts, time_effects]
    alpha = dgp["alpha_price"]
    utility = x[:, :-1] @ controls - alpha * common.basic_price.to_numpy()
    benchmark_share = 1 / (1 + np.exp(-utility))
    benchmark = price_projection(common, benchmark_share)
    np.testing.assert_allclose(benchmark["price_coefficient"], -alpha, atol=1e-10)
    results.append({"outcome": "exact_binary_logit", **benchmark})
    result = {"data_type": "synthetic", "available_provider_months": len(frame),
              "common_positive_share_rows": len(common), "excluded_zero_share_rows": len(frame) -
              ↪  len(common),
              "imposed_utility_price_coefficient": -alpha, "projections": results,
```

```
            "interpretation": "Stock and flow slopes are descriptive projections of a heterogeneous
            ↪  dynamic simulation; only the exact binary benchmark recovers its imposed coefficient
            ↪  by construction"}
    OUTPUT.mkdir(exist_ok=True)
    frame.to_csv(OUTPUT / "provider_months.csv", index=False, float_format="%.10f")
    pd.DataFrame(results).to_csv(OUTPUT / "price_projections.csv", index=False, float_format="%.10f")
    (OUTPUT / "demand.json").write_text(json.dumps(result, indent=2) + "\n")
    print(json.dumps(result, indent=2))


if __name__ == "__main__":
    main()
```